Continuous-Variable Quantum Error Correction using Topological Gottesman–Kitaev–Preskill (GKP) Codes

A THESIS SUBMITTED FOR
THE DEGREE OF

Master of Science

IN THE
DEPARTMENT OF PHYSICS

BY

PRITAM GHOSH



DEPARTMENT OF PHYSICS
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

JUNE, 2026

Certificate

This is to certify that the thesis entitled

"Continuous-Variable Quantum Error Correction using Topological Gottesman–Kitaev–Preskill (GKP) Codes"

submitted by **Mr. Pritam Ghosh** to the Indian Institute of Technology Madras in partial fulfillment of the requirements for the award of the degree of **Master of Science**, is a bona fide record of the original research work carried out by him under the supervision of **Prof. Shayan S. Garani**, Department of Electronic Systems Engineering, Indian Institute of Science, Bangalore, and the co-supervision of **Prof. Prabha Mandayam**, Department of Physics, Indian Institute of Technology Madras.

The work reported in this thesis has not been submitted, either in part or in full, to any other university or institute for the award of any degree, diploma, or other academic qualification. Collaborations and contributions from external researchers and institutions have been appropriately acknowledged wherever applicable.

Chennai – 600036

Date: 11.06.2026

Prof. Prabha Mandayam
Professor
Department of Physics
Indian Institute of Technology Madras

Prof. Shayan S. Garani
Professor
Department of Electronic Systems Engineering
Indian Institute of Science

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Prof. Shayan S. Garani, for giving me the opportunity to work on this project and for introducing me to the field of quantum error correction. His guidance, encouragement, and insightful discussions have been invaluable throughout this work.

I am equally grateful to Prof. Prabha Mandayam for her mentorship, support, and valuable suggestions during this project. Her lectures, discussions, and feedback greatly enriched my understanding of quantum information and quantum error correction.

I would like to extend my heartfelt thanks to Mr. Mrinmoy Karmakar for his significant contributions to the development of the Multi-Genus Rotated Toric code architecture employed in this thesis. I am also grateful for the many discussions and exchanges of ideas that greatly enhanced this work. My sincere thanks also go to Dr. Pavan Kumar for his guidance, encouragement, and constant willingness to help.

I would like to thank the members of the PNSIL Lab, Indian Institute of Science, Bangalore, for their support and for creating a welcoming and collaborative research environment. I am especially grateful to Karthik, Sudhir, and other members of the group for the many discussions, suggestions, and interactions that enriched both this work and my research experience.

I am grateful to the Department of Physics, Indian Institute of Technology Madras, for providing an excellent academic and research environment. The coursework, faculty, and research culture of the department have played an important role in my academic development.

I would also like to thank my friends and classmates at IIT Madras for their encouragement, companionship, and memorable discussions throughout my Master's program.

Finally, I express my deepest gratitude to my parents and family for their unwavering love, sacrifices, patience, and constant encouragement. Their faith in me has been my greatest source of strength, and this work would not have been possible without their support.

Pritam Ghosh
Department of Physics
Indian Institute of Technology Madras

Abstract

Quantum error correction is an essential requirement for the realization of large-scale fault-tolerant quantum computation. While conventional qubit-based quantum error-correcting codes have been studied extensively, continuous-variable (CV) quantum systems provide an alternative framework in which quantum information can be protected using bosonic degrees of freedom. Among the various CV quantum error-correcting codes, the Gottesman–Kitaev–Preskill (GKP) code is particularly significant due to its ability to convert small displacement errors in phase space into correctable discrete errors.

In this thesis, we investigate the performance of concatenated GKP-topological codes under Gaussian displacement noise. After reviewing the theoretical foundations of stabilizer and topological quantum codes, we study the concatenation of square GKP states with topological quantum error-correcting codes. Particular emphasis is placed on the closest-point decoding framework, which maps GKP decoding to a closest vector problem on an associated lattice.

Building upon this framework, we introduce and analyze Multi-Genus Rotated Toric-GKP codes, a family of higher-genus topological constructions that generalize the conventional rotated toric code while encoding multiple logical qubits within a single connected architecture. The decoding problem is formulated using a polynomial-time closest-point decoder based on minimum-weight perfect matching, and extensive Monte Carlo simulations are performed to evaluate logical performance under Gaussian displacement noise.

Numerical results show that the Multi-Genus Rotated Toric-GKP codes exhibit threshold values consistently higher than those obtained for the Rotated Surface-GKP code. For the genera investigated in this work, threshold estimates ranging from approximately 0.6025 to 0.6040 are obtained. In addition, detailed fidelity comparisons for the genus-2 construction reveal logical-fidelity improvements of up to approximately 20% relative to equivalent logical-capacity Rotated Surface-GKP constructions while remaining competitive against higher-distance surface-code counterparts.

A further advantage of the proposed architecture is its improved encoding efficiency. Whereas the rotated surface code encodes a single logical qubit, a genus-g Rotated Toric code encodes $2g$ logical qubits while preserving favorable fault-tolerance properties. These results demonstrate the potential of Multi-Genus Rotated Toric-GKP codes for CV fault-tolerant quantum computation and motivate future studies of tensor-network and machine-learning decoders.

Contents

Certificate i

Acknowledgement iii

Abstract v

1 Introduction 1
- 1.1 Quantum Computation and the Need for Error Correction 1
- 1.2 Topological Quantum Error Correction 2
- 1.3 Continuous-Variable Quantum Error Correction and GKP Codes . . 2
- 1.4 Concatenated GKP-Topological Codes 3
- 1.5 Motivation for Higher-Genus Topological Codes 3
- 1.6 Objectives and Contributions of this Thesis 3

2 Background 5
- 2.1 Overview of Quantum Error Correction 5
- 2.2 Discrete-Variable Quantum Error Correction 6
  - 2.2.1 Quantum Noise 6
  - 2.2.2 The No-Cloning Obstacle 7
  - 2.2.3 Encoding Quantum Information 7
  - 2.2.4 Error Detection and Syndrome Extraction 8
  - 2.2.5 Discretization of Quantum Errors 8
  - 2.2.6 The Three-Qubit Repetition Code 9
  - 2.2.7 Stabilizer Formalism 11
  - 2.2.8 CSS Codes 21
  - 2.2.9 Homological Perspective of CSS Codes 23
- 2.3 Topological Quantum Codes 32
  - 2.3.1 Surface Codes 32
  - 2.3.2 Rotated Surface Codes 39
  - 2.3.3 Toric Codes 42
  - 2.3.4 Rotated Toric Codes 44
- 2.4 Continuous-Variable (CV) Quantum Error Correction 45
  - 2.4.1 Quantum Harmonic Oscillator and Phase Space 46
  - 2.4.2 Displacement Operator 49

2.4.3 Coherent States . . . 49
2.4.4 Generation of Coherent States . . . 50
2.5 Construction of Approximate GKP Codewords . . . 51
2.5.1 Vacuum State in Position Representation . . . 51
2.5.2 Translation of Wavepackets . . . 51
2.5.3 Approximate GKP Logical Codewords . . . 52
2.5.4 Logical Plus State . . . 52
2.5.5 Momentum-Space Interpretation . . . 53
2.6 Ideal GKP Codewords and their relation with Approximate GKP codewords . . . 54
2.6.1 From Approximate to Ideal GKP States . . . 54
2.6.2 Approximate GKP States as Smeared Ideal Codewords . . . 55
2.6.3 Choice of Smearing Function . . . 56
2.6.4 Orthogonality of Ideal GKP Codewords . . . 57
2.6.5 Non-Orthogonality of Approximate Codewords . . . 58
2.6.6 Choice of Spacing . . . 58
2.6.7 Alternative Geometries . . . 59
2.7 Square GKP and their stabilizers and Logicals . . . 60
2.7.1 Position Space Representation . . . 60
2.7.2 Momentum Space Representation . . . 60
2.7.3 Phase-Space Translations and Lattice Structure . . . 60
2.7.4 Stabilizer Operators . . . 61
2.7.5 Logical Operators . . . 62
2.8 Multimode Bosonic Formalism . . . 63
2.8.1 Multimode Quadrature Operators . . . 63
2.8.2 Symplectic Matrix . . . 63
2.8.3 Multimode Displacement Operators . . . 64
2.8.4 Weyl Commutation Relation . . . 64
2.8.5 Gaussian Unitary Operations . . . 65
2.9 Multimode GKP Stabilizers and Lattice Structure . . . 65
2.9.1 Stabilizer Generator Vectors . . . 65
2.9.2 Generator Matrix Representation . . . 66
2.9.3 Stabilizer Group and Phase-Space Lattice . . . 66
2.9.4 Commutation Requirement and Symplectic Integral Lattices 67
2.9.5 Canonical Form of the Commutation Matrix . . . 68
2.9.6 Normalizer Lattice and Logical Operators . . . 68
2.9.7 Generator Matrix of the Dual Lattice . . . 69
2.9.8 Logical Operators and Cosets . . . 69
2.9.9 Encoded Dimension . . . 70
2.9.10 Code Distance in Multimode GKP Codes . . . 71
2.10 Gaussian Transformations Between GKP Codes . . . 72
2.10.1 Decomposition into Square Lattice GKP Codes . . . 73
2.11 Decoding of Multimode GKP Codes . . . 74

2.11.1 Physical Noise Model . . . 74
2.11.2 Displacement Representation of Errors . . . 74
2.11.3 Syndrome Extraction . . . 75
2.11.4 Syndrome Cosets and Lattice Ambiguity . . . 75
2.11.5 Maximum-Likelihood Decoding and Closest Vector Problem 76
2.11.6 Recovery Operation and Decoding Outcome . . . 77
2.11.7 Minimum-Energy Interpretation . . . 78
2.11.8 Decoding Complexity and the Closest Vector Problem . . . 78
2.11.9 Computational Complexity of CVP . . . 79
2.11.10 Structured Lattices for Efficient Decoding . . . 80
2.12 Examples of Multimode GKP Lattice Constructions . . . 80
2.12.1 Hypercubic ($\mathbb{Z}_{2N}$) Lattice . . . 80
2.12.2 Rectangular GKP Lattice . . . 81
2.12.3 Relation to Square GKP via Squeezing . . . 81
2.12.4 Logical Distances of Rectangular GKP Codes . . . 82
2.12.5 Dense Lattice Packing and Decoding Performance . . . 83
2.12.6 Checkerboard Lattice D_n . . . 83
2.12.7 Tesseract Lattice GKP Code . . . 84
2.12.8 Concatenation Interpretation . . . 84
2.12.9 Logical Distances of the Tesseract Code . . . 85
2.12.10 Generalized Repetition-Rectangular Codes . . . 85
2.13 Lattice Structure as the Computational Backbone of GKP Decoding 86
2.13.1 Direct Sum Structure . . . 86
2.13.2 Union of Cosets . . . 87
2.13.3 Glue Lattices . . . 88
2.13.4 From Exponential Search to Practical Decoders . . . 88

3 Higher genus Toric code **89**
3.1 Stabilizers . . . 89
3.2 Logical Operator . . . 90
3.3 Generalized Toric code . . . 91
3.4 Construction of generalized Rotated Toric code . . . 91
3.5 Summary . . . 92

4 Concatenation with GKP Codes **93**
4.1 Surface-GKP Construction . . . 93
4.1.1 General Standard Form Construction . . . 94
4.1.2 Topological Scrambling and the RREF Approach . . . 95

5 Structure of the decoder **98**
5.1 Polynomial-Time Closest-Point Decoder for the Surface-GKP Code 98
5.2 Decoder for the Multi-Genus Toric-GKP Code . . . 102

6 Simulation Methodology **103**
6.1 Noise Model . . . 103
6.2 Syndrome Extraction . . . 104
6.3 Construction of the Concatenated GKP Code . . . 105
6.4 Monte Carlo Simulation Procedure . . . 106
6.5 Logical Success Criterion . . . 107

7 Results and Discussion **109**
7.1 Surface-GKP Code Performance . . . 109
7.1.1 Logical Fidelity as a Function of Noise Strength . . . 110
7.1.2 Threshold Estimation . . . 111
7.2 Multi-Genus Rotated Toric-GKP Code Performance . . . 112
7.2.1 Representative Logical Fidelity Curves . . . 112
7.2.2 Threshold Estimation for Different Genera . . . 113
7.3 Comparison with the Rotated Surface-GKP Code . . . 116
7.3.1 Comparison at Equal Logical Capacity . . . 116
7.3.2 Comparison with Higher-Distance Surface-GKP Codes . . . 118
7.3.3 Implications for Higher-Genus Codes . . . 119
7.3.4 Encoding Rate Improvement . . . 120

8 Conclusion and Future Work **122**
8.1 Summary and Conclusions . . . 122
8.2 Future Work . . . 123

References **125**

Chapter 1

Introduction

1.1 Quantum Computation and the Need for Error Correction

Quantum computation exploits uniquely quantum mechanical phenomena such as superposition and entanglement to process information in ways that are fundamentally inaccessible to classical computers. Quantum algorithms have demonstrated theoretical advantages for several important computational tasks, including integer factorization, unstructured database search, quantum simulation, and optimization problems. These developments have motivated significant efforts toward the realization of large-scale quantum computers.

Despite these advantages, quantum systems are inherently susceptible to environmental noise and imperfections in quantum operations. Interaction with the surrounding environment leads to decoherence, while imperfect control introduces operational errors. Unlike classical digital systems, where information can be protected through straightforward redundancy, quantum information cannot be copied arbitrarily due to the no-cloning theorem. Consequently, protecting quantum information against noise remains one of the central challenges in quantum computing.

Quantum error correction (QEC) provides a framework for overcoming this challenge by encoding logical quantum information into a larger physical system [1] [2] [3]. Instead of storing information in a single physical qubit, QEC distributes the information across multiple physical degrees of freedom, enabling the detection and correction of errors without directly measuring the encoded quantum state. The development of efficient quantum error-correcting codes is therefore essential for realizing fault-tolerant quantum computation.

1.2 Topological Quantum Error Correction

Among the various approaches to quantum error correction, topological quantum codes [4] have emerged as one of the most promising architectures for scalable fault-tolerant quantum computation. In topological codes, logical information is stored nonlocally in global topological features of a lattice, making it intrinsically robust against local errors.

One of the most extensively studied examples is the surface code [5], which arranges physical qubits on a two-dimensional lattice and employs local stabilizer measurements to detect errors. Surface codes possess several attractive properties, including high error thresholds, compatibility with nearest-neighbor interactions, and relatively simple syndrome extraction procedures. These features have made them a leading candidate for implementation on current quantum hardware platforms.

The closely related toric code provides a topological interpretation of quantum error correction through homology and non-contractible loops on closed surfaces. Logical operators correspond to topologically distinct paths that cannot be continuously deformed into one another. This connection between topology and quantum information has inspired numerous generalizations of surface and toric codes to more complex geometries.

1.3 Continuous-Variable Quantum Error Correction and GKP Codes

While most quantum error-correcting codes are formulated using discrete-variable qubits, an alternative paradigm employs continuous-variable quantum systems such as harmonic oscillators. Continuous-variable platforms naturally arise in several experimental settings, including superconducting microwave cavities and optical modes.

A particularly important continuous-variable code is the Gottesman–Kitaev–Preskill (GKP) code [6], which encodes a qubit into an oscillator by constructing lattice-like states in phase space. The GKP code converts small displacement errors in position and momentum into correctable shift errors and thereby provides protection against Gaussian noise.

An attractive feature of GKP encoding is its ability to serve as a first layer of error correction. Residual errors after GKP correction can subsequently be handled by a higher-level qubit code, leading to concatenated error-correction architectures. Such concatenation combines the strengths of bosonic and qubit-based error correction and has become an active area of research in fault-tolerant quantum computing.

1.4 Concatenated GKP-Topological Codes

Recent studies have demonstrated that concatenating GKP codes with topological stabilizer codes can significantly improve error-correction performance [7]. In these schemes, each physical qubit of a topological code is replaced by a GKP-encoded oscillator. The GKP layer suppresses continuous displacement errors, while the outer stabilizer code corrects the remaining logical errors.

Among these constructions, concatenated GKP-surface codes have attracted considerable attention due to their favorable threshold behavior and compatibility with experimentally relevant Gaussian noise models. Numerical and theoretical investigations have shown that the use of GKP encoding can substantially enhance the effective threshold of topological codes and reduce logical error rates.

However, despite these advantages, conventional surface-code architectures possess a relatively low encoding rate. A standard planar surface code encodes only a single logical qubit while requiring a large number of physical qubits to achieve practical code distances. As quantum processors scale, improving the ratio of encoded logical information to physical resources becomes an increasingly important objective.

1.5 Motivation for Higher-Genus Topological Codes

The encoding properties of topological codes are closely connected to the topology of the underlying surface. For a closed orientable surface of genus g, the corresponding toric code encodes $k = 2g$ logical qubits.

Consequently, increasing the genus of the underlying surface naturally increases the number of encoded logical qubits without requiring independent copies of the code. Higher-genus topological codes therefore provide a promising route toward improving encoding efficiency while preserving the favorable properties of topological error correction.

Although higher-genus topological codes have been studied from a theoretical perspective, comparatively little work has explored their behavior when combined with bosonic encodings such as the GKP code. In particular, it remains unclear whether the increased encoding rate can be achieved without sacrificing error-correction performance, and whether higher-genus constructions may offer advantages in logical fidelity and threshold behavior.

1.6 Objectives and Contributions of this Thesis

The primary objective of this thesis is to investigate the performance of multigenus toric codes concatenated with GKP encoding under Gaussian displacement noise.

Specifically, this work aims to:

1. Construct multigenus toric-code families capable of encoding multiple logical qubits.
2. Develop a concatenated GKP-topological coding framework based on these constructions.
3. Implement numerical decoding and Monte Carlo simulation methods for evaluating logical performance.
4. Compare the logical fidelity and threshold behavior of multigenus toric-GKP codes with conventional surface-GKP codes.
5. Analyze the encoding efficiency and resource utilization associated with higher-genus topological architectures.

The results presented in this thesis demonstrate that multigenus toric-GKP codes can achieve improved logical fidelity and threshold performance while simultaneously providing substantially higher encoding rates. These findings suggest that higher-genus topological constructions constitute a promising direction for scalable fault-tolerant quantum information processing.

Chapter 2

Background

2.1 Overview of Quantum Error Correction

Quantum computers promise significant computational advantages over their classical counterparts for a variety of problems, including integer factorization, quantum simulation, and optimization. However, quantum information is highly susceptible to noise arising from environmental interactions, imperfect control operations, and measurement errors. Even small amounts of noise can accumulate during a computation and ultimately corrupt the stored quantum information. Consequently, the development of quantum error correction (QEC) has become a fundamental requirement for realizing large-scale fault-tolerant quantum computation [1, 3].

Quantum error correction seeks to protect quantum information by encoding logical states into larger physical systems in such a way that errors can be detected and corrected without disturbing the encoded information. Over the past several decades, two major paradigms of quantum error correction have emerged.

The first paradigm is *discrete-variable quantum error correction*, in which quantum information is encoded into finite-dimensional systems such as qubits. Errors are typically represented using combinations of Pauli operators, and protection is achieved through the use of stabilizer codes, CSS codes, and topological codes. Notable examples include the surface code and toric code, which have become leading candidates for fault-tolerant quantum computation due to their high error thresholds and locality properties [4, 8, 5].

The second paradigm is *continuous-variable quantum error correction*, where information is encoded into infinite-dimensional quantum systems such as harmonic oscillators. In this framework, noise is naturally described by continuous displacements in phase space. Among the various continuous-variable codes, the Gottesman–Kitaev–Preskill (GKP) code occupies a particularly important position because it enables the correction of small displacement errors while providing an interface between continuous-variable and qubit-based quantum information processing [6].

These two approaches are not mutually exclusive. In recent years, concate-

nated architectures combining continuous-variable and discrete-variable codes have attracted considerable attention. In such schemes, a bosonic code such as the GKP code serves as the first layer of protection against physical noise, while a higher-level stabilizer code corrects the residual logical errors. Surface-GKP codes constitute one prominent example of this approach and have demonstrated substantial improvements in fault-tolerant performance [7].

The primary focus of this thesis is the study of multigenus toric codes concatenated with GKP encoding. To develop the necessary theoretical framework, this chapter first reviews the principles of discrete-variable quantum error correction, including quantum noise, syndrome extraction, stabilizer codes, and topological codes. Subsequently, the fundamentals of continuous-variable quantum error correction and GKP encoding are introduced. Finally, the concatenation of topological and GKP codes is discussed, providing the foundation for the multigenus toric-GKP constructions investigated in later chapters.

2.2 Discrete-Variable Quantum Error Correction

Quantum information is inherently fragile due to unavoidable interactions between a quantum system and its surrounding environment. Such interactions lead to decoherence and operational errors that corrupt the stored quantum information. Unlike classical information, which can be protected using simple redundancy schemes, quantum information requires specialized techniques due to the fundamental principles of quantum mechanics. The development of quantum error correction (QEC) has therefore become one of the central requirements for realizing large-scale fault-tolerant quantum computation [1, 3].

2.2.1 Quantum Noise

The evolution of an isolated quantum system is governed by a unitary operator. In practice, however, quantum systems are open and interact with their environment. Such interactions are described by quantum channels, which are mathematically represented by completely positive trace-preserving (CPTP) maps,

$$\mathcal{E} : \mathcal{D}(\mathcal{H}) \to \mathcal{D}(\mathcal{H}), \tag{2.1}$$

where $\mathcal{D}(\mathcal{H})$ denotes the set of density operators acting on the Hilbert space $\mathcal{H}$. Any CPTP map admits a Kraus decomposition of the form

$$\mathcal{E}(\rho) = \sum_i E_i \rho E_i^\dagger, \tag{2.2}$$

where the Kraus operators satisfy the completeness relation

$$\sum_i E_i^\dagger E_i = I. \tag{2.3}$$

Since an arbitrary quantum channel can contain infinitely many possible error processes, directly correcting every possible error appears infeasible. Quantum error correction overcomes this challenge by encoding quantum information into a larger Hilbert space and exploiting the structure of quantum noise.

2.2.2 The No-Cloning Obstacle

Classical error correction relies heavily on redundancy. A classical bit may be copied several times and recovered through majority voting when errors occur. One may therefore wonder whether a similar strategy can be applied to quantum information.

Consider an arbitrary qubit

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \tag{2.4}$$

where

$$|\alpha|^2 + |\beta|^2 = 1. \tag{2.5}$$

Ideally, one might wish to duplicate the state in order to protect it against noise. However, the no-cloning theorem states that there exists no universal unitary operation capable of performing

$$|\psi\rangle|0\rangle \longrightarrow |\psi\rangle|\psi\rangle \tag{2.6}$$

for an arbitrary unknown quantum state $|\psi\rangle$ [1]. Consequently, quantum error correction must protect quantum information without directly copying the encoded state.

2.2.3 Encoding Quantum Information

The central idea of quantum error correction is to encode a logical qubit into a larger physical Hilbert space.

Instead of storing the quantum state

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \tag{2.7}$$

in a single physical qubit, an encoding operation maps it into a larger system,

$$|\psi\rangle \longmapsto |\psi\rangle_L = \alpha|0_L\rangle + \beta|1_L\rangle, \tag{2.8}$$

where $|0_L\rangle$ and $|1_L\rangle$ denote logical basis states.

The logical states span a protected subspace,

$$\mathcal{C} = \text{span}\left\{|0_L\rangle, |1_L\rangle\right\}, \tag{2.9}$$

known as the *code space*. The code space occupies only a small portion of the full physical Hilbert space. Errors acting on physical qubits generally move encoded states out of the code space and into distinguishable error subspaces.

2.2.4 Error Detection and Syndrome Extraction

A major challenge in quantum error correction is determining whether an error has occurred without disturbing the encoded quantum information. Direct measurement of the data qubits would collapse the quantum state and destroy the encoded superposition.

To circumvent this problem, auxiliary ancilla qubits are introduced. The ancilla qubits interact with the encoded data through controlled operations and are subsequently measured. The resulting classical outcomes provide information about the error while preserving the logical quantum state.

The collection of measurement outcomes is known as the *error syndrome*. Distinct correctable errors produce distinct syndrome patterns, allowing the error to be identified and corrected without revealing information about the logical state itself.

2.2.5 Discretization of Quantum Errors

An important insight due to Gottesman is that arbitrary quantum errors can be reduced to combinations of Pauli errors [2].

The single-qubit Pauli operators are

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \tag{2.10}$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{2.11}$$

These operators form a basis for the vector space of all 2×2 complex matrices. Consequently, any operator acting on a single qubit can be expressed as

$$E = c_0 I + c_1 X + c_2 Y + c_3 Z, \tag{2.12}$$

where $c_i \in \mathbb{C}$.

Therefore, the correction of arbitrary physical noise can be reduced to the correction of combinations of Pauli errors. This observation forms the foundation of modern quantum error correction and motivates the stabilizer formalism, which will be discussed in subsequent sections.

2.2.6 The Three-Qubit Repetition Code

To illustrate the basic principles of quantum error correction, we consider the three-qubit repetition code. This code protects quantum information against a single bit-flip error and serves as one of the simplest examples of a quantum error-correcting code.

Logical Encoding

Consider a logical qubit

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle. \tag{2.13}$$

The three-qubit repetition code encodes the logical basis states as

$$|0_L\rangle = |000\rangle, \tag{2.14}$$

$$|1_L\rangle = |111\rangle. \tag{2.15}$$

An arbitrary logical state is therefore encoded as

$$|\psi\rangle_L = \alpha|000\rangle + \beta|111\rangle. \tag{2.16}$$

The logical states span the code space

$$\mathcal{C} = \text{span}\left\{|000\rangle, |111\rangle\right\}. \tag{2.17}$$

Since three physical qubits possess an 8-dimensional Hilbert space,

$$\mathcal{H} = (\mathbb{C}^2)^{\otimes 3}, \tag{2.18}$$

the code space occupies only a two-dimensional subspace of the full Hilbert space.

Bit-Flip Noise Model

The three-qubit repetition code is designed to correct bit-flip errors.

A single-qubit bit-flip channel is described by

$$\mathcal{E}(\rho) = (1-p)\rho + pX\rho X, \tag{2.19}$$

where p denotes the probability of a bit-flip error.

Equivalently, the channel may be represented using Kraus operators

$$E_0 = \sqrt{1-p}\, I, \tag{2.20}$$

$$E_1 = \sqrt{p}\, X. \tag{2.21}$$

Assuming independent noise on the three physical qubits, the overall channel is

$$\mathcal{E}^{\otimes 3}. \tag{2.22}$$

The corresponding error operators are

$$III, \tag{2.23}$$

$$XII, \quad IXI, \quad IIX, \tag{2.24}$$

$$XXI, \quad XIX, \quad IXX, \tag{2.25}$$

$$XXX. \tag{2.26}$$

These correspond respectively to zero, one, two, and three bit-flip errors.

Correctable Errors

The three-qubit repetition code is capable of correcting any single bit-flip error.

Consider the logical state

$$|\psi\rangle_L = \alpha|000\rangle + \beta|111\rangle. \tag{2.27}$$

A bit-flip acting on the first qubit produces

$$XII|\psi\rangle_L = \alpha|100\rangle + \beta|011\rangle. \tag{2.28}$$

Similarly,

$$IXI|\psi\rangle_L = \alpha|010\rangle + \beta|101\rangle, \tag{2.29}$$

and

$$IIX|\psi\rangle_L = \alpha|001\rangle + \beta|110\rangle. \tag{2.30}$$

The states generated by these single-qubit errors are mutually orthogonal and therefore distinguishable.

Consequently, the error can be identified without disturbing the encoded quantum information.

Error Subspaces

Each correctable error maps the code space into a distinct error subspace.

The code space is

$$\mathcal{C}_0 = \text{span}\{|000\rangle, |111\rangle\}. \tag{2.31}$$

The three single-error subspaces are

$$\mathcal{C}_1 = \text{span}\{|100\rangle, |011\rangle\}, \tag{2.32}$$

$$\mathcal{C}_2 = \text{span}\{|010\rangle, |101\rangle\}, \tag{2.33}$$

$$\mathcal{C}_3 = \text{span}\{|001\rangle, |110\rangle\}. \tag{2.34}$$

These subspaces satisfy

$$\mathcal{C}_i \cap \mathcal{C}_j = \varnothing, \qquad i \neq j, \tag{2.35}$$

and are mutually orthogonal.

The ability to distinguish between these subspaces forms the basis of error correction.

Syndrome Extraction

To determine which error has occurred, syndrome measurements are performed using ancilla qubits.

For the repetition code, two parity checks are sufficient:

$$Z_1 Z_2, \tag{2.36}$$

$$Z_2 Z_3. \tag{2.37}$$

The measurement outcomes uniquely identify whether no error, a first-qubit error, a second-qubit error, or a third-qubit error has occurred.

Once the syndrome has been obtained, the corresponding recovery operation is applied to return the state to the code space.

Logical Error Probability

A logical failure occurs when two or more physical qubits experience bit-flip errors.

The probability of successful correction is therefore

$$P_{\text{succ}} = (1-p)^3 + 3p(1-p)^2. \tag{2.38}$$

The probability of logical failure is

$$P_L = 3p^2(1-p) + p^3. \tag{2.39}$$

Simplifying,

$$P_L = 3p^2 - 2p^3. \tag{2.40}$$

For sufficiently small physical error rates,

$$P_L \approx 3p^2, \tag{2.41}$$

which is significantly smaller than the unencoded error probability p.

The three-qubit repetition code therefore demonstrates the central idea of quantum error correction: by encoding information into a larger Hilbert space, the logical error probability can be suppressed below the physical error probability.

2.2.7 Stabilizer Formalism

The stabilizer formalism, introduced by Gottesman, provides a powerful algebraic framework for describing and analyzing a large class of quantum error-correcting codes [2]. Most modern quantum codes, including CSS codes, surface codes, toric codes, and color codes, can be naturally expressed within this formalism.

A quantum error-correcting code is typically specified by the parameters

$$[[n, k, d]], \tag{2.42}$$

where n denotes the number of physical qubits, k denotes the number of encoded logical qubits, and d denotes the distance of the code.

The distance d determines the error-correcting capability of the code. A code of distance d can correct

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor \tag{2.43}$$

arbitrary qubit errors and can detect up to $d-1$ errors [1].

Stabilized States

The central idea of the stabilizer formalism is to characterize quantum states through operators that leave them invariant.

Definition 2.1 *A quantum state $|\psi\rangle$ is said to be stabilized by an operator S if*

$$S|\psi\rangle = |\psi\rangle. \tag{2.44}$$

Equivalently, $|\psi\rangle$ is a $+1$ eigenstate of S.

As an example, consider the Bell state

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}. \tag{2.45}$$

One readily verifies that

$$(X \otimes X)|\Phi^+\rangle = |\Phi^+\rangle, \tag{2.46}$$

and

$$(Z \otimes Z)|\Phi^+\rangle = |\Phi^+\rangle. \tag{2.47}$$

Therefore the Bell state is simultaneously stabilized by the operators

$$X \otimes X \qquad \text{and} \qquad Z \otimes Z. \tag{2.48}$$

More generally, if two states $|\psi_1\rangle$ and $|\psi_2\rangle$ are stabilized by an operator S, then any linear combination

$$\alpha|\psi_1\rangle + \beta|\psi_2\rangle \tag{2.49}$$

is also stabilized by S. Consequently, the set of states stabilized by S forms a vector space, namely the $+1$ eigenspace of S.

The Pauli Group

The stabilizer formalism is built upon the Pauli operators

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \tag{2.50}$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{2.51}$$

The single-qubit Pauli group is defined as

$$\mathcal{P}_1 = \{\pm I, \pm iI, \pm X, \pm iX, \pm Y, \pm iY, \pm Z, \pm iZ\}. \tag{2.52}$$

For a system of n qubits, the Pauli group is

$$\mathcal{P}_n = \{i^m P_1 \otimes P_2 \otimes \cdots \otimes P_n\}, \tag{2.53}$$

where

$$m \in \{0, 1, 2, 3\} \tag{2.54}$$

and each

$$P_j \in \{I, X, Y, Z\}. \tag{2.55}$$

Every element of $\mathcal{P}_n$ either commutes or anticommutes with every other element, a property that plays a crucial role in syndrome extraction and error detection.

Stabilizer Groups

Let $\mathcal{C} \subseteq (\mathbb{C}^2)^{\otimes n}$ denote a quantum code space.

Definition 2.2 *The stabilizer group associated with the code space $\mathcal{C}$ is*

$$S(\mathcal{C}) = \{s \in \mathcal{P}_n \,|\, s|\psi\rangle = |\psi\rangle \;\forall\; |\psi\rangle \in \mathcal{C}\}. \tag{2.56}$$

Thus every element of the stabilizer group leaves every codeword invariant.

The stabilizer group forms an Abelian subgroup of the n-qubit Pauli group. Since all stabilizer elements must possess a common set of eigenvectors, they necessarily commute with one another. Furthermore, the element $-I$ cannot belong to a stabilizer group because the condition

$$-I|\psi\rangle = |\psi\rangle \tag{2.57}$$

implies

$$|\psi\rangle = 0, \tag{2.58}$$

which is not a physical quantum state.

Generators and Code Spaces

In practice, it is unnecessary to specify every element of a stabilizer group. Instead, one specifies a set of generators

$$S = \langle g_1, g_2, \ldots, g_r \rangle. \tag{2.59}$$

Every stabilizer element can then be expressed as

$$g_1^{a_1} g_2^{a_2} \cdots g_r^{a_r}, \tag{2.60}$$

where

$$a_i \in \{0, 1\}. \tag{2.61}$$

The code space is defined as the common $+1$ eigenspace of all stabilizer generators,

$$\mathcal{C} = \{|\psi\rangle \mid g_i|\psi\rangle = |\psi\rangle, \quad i = 1, \ldots, r\}. \tag{2.62}$$

Equivalently, the code space may be viewed as the intersection of the $+1$ eigenspaces of all stabilizer generators. This perspective provides a particularly useful geometric interpretation of stabilizer codes and forms the foundation of topological quantum error correction.

Dimension of a Stabilizer Code

Let

$$S = \langle g_1, g_2, \ldots, g_r \rangle \tag{2.63}$$

be a stabilizer group generated by r independent commuting Pauli operators acting on n physical qubits.

Each stabilizer generator g_i has eigenvalues ± 1. Consequently, the Hilbert space may be decomposed into the direct sum of the corresponding $+1$ and -1 eigenspaces.

Imposing the stabilizer condition

$$g_i|\psi\rangle = |\psi\rangle \tag{2.64}$$

restricts the allowed states to the $+1$ eigenspace of g_i, thereby reducing the dimension of the available Hilbert space by a factor of two.

Since the generators are independent and mutually commuting, each additional stabilizer generator imposes an independent constraint. Starting from the full Hilbert space

$$\dim(\mathcal{H}) = 2^n, \tag{2.65}$$

the dimension of the code space becomes

$$\dim(\mathcal{C}) = \frac{2^n}{2^r} = 2^{n-r}. \tag{2.66}$$

For a stabilizer code encoding k logical qubits,

$$\dim(\mathcal{C}) = 2^k. \tag{2.67}$$

Comparing the two expressions yields the important relation

$$k = n - r. \tag{2.68}$$

Thus an $[[n, k]]$ stabilizer code possesses

$$r = n - k \tag{2.69}$$

independent stabilizer generators. [2]

Projectors onto the Code Space

For each stabilizer generator g_i, the projector onto its $+1$ eigenspace is

$$P_i^{(+)} = \frac{I + g_i}{2}. \tag{2.70}$$

To verify that this operator is indeed a projector, observe that

$$\left(P_i^{(+)}\right)^2 = \frac{(I + g_i)^2}{4}. \tag{2.71}$$

Using

$$g_i^2 = I, \tag{2.72}$$

one obtains

$$\left(P_i^{(+)}\right)^2 = \frac{I + 2g_i + I}{4} = \frac{I + g_i}{2} = P_i^{(+)}. \tag{2.73}$$

Since the code space is the common $+1$ eigenspace of all stabilizer generators, the projector onto the code space is given by

$$P = \prod_{i=1}^{r} \frac{I + g_i}{2}. \tag{2.74}$$

The image of this projector is precisely the stabilizer code space,

$$P\mathcal{H} = \mathcal{C}. \tag{2.75}$$

This projector representation plays a central role in the derivation of error-correction conditions and the construction of logical operators. [2]

The Normalizer

Not every Pauli operator acting on the physical qubits produces a detectable error.

To characterize such operators, one introduces the normalizer of the stabilizer group.

Definition 2.3 *The normalizer of a stabilizer group S within the n-qubit Pauli group $\mathcal{P}_n$ is defined as*

$$N(S) = \{E \in \mathcal{P}_n \mid ES = SE \quad \forall S \in S\}. \tag{2.76}$$

Equivalently, $N(S)$ consists of all Pauli operators that commute with every stabilizer element.

Since every stabilizer element commutes with every other stabilizer element, it follows immediately that

$$S \subseteq N(S). \tag{2.77}$$

The normalizer plays a fundamental role in the characterization of logical operators and code distance.

Consider a codeword

$$|\psi\rangle \in \mathcal{C}. \tag{2.78}$$

If

$$E \in N(S), \tag{2.79}$$

then for any stabilizer element $s \in S$,

$$s(E|\psi\rangle) = Es|\psi\rangle = E|\psi\rangle. \tag{2.80}$$

Hence $E|\psi\rangle$ remains within the code space.

Operators belonging to the normalizer therefore preserve the code space. Some of these operators correspond merely to stabilizer elements and act trivially on the encoded information, while others act nontrivially on the logical qubits. This distinction will be explored in the discussion of logical operators.

Error Detection in Stabilizer Codes

The stabilizer formalism provides a simple criterion for determining whether an error is detectable. Let S denote the stabilizer group associated with a code space $\mathcal{C}$, and let

$$E \in \mathcal{P}_n \tag{2.81}$$

be an error operator belonging to the n-qubit Pauli group.

The detectability of E depends on its commutation relations with the stabilizer generators.

Errors in the Normalizer

Consider an error operator

$$E \in N(S), \tag{2.82}$$

where $N(S)$ denotes the normalizer of the stabilizer group.

For any codeword

$$|\psi\rangle \in \mathcal{C}, \tag{2.83}$$

and for every stabilizer element $s \in S$,

$$s(E|\psi\rangle) = Es|\psi\rangle = E|\psi\rangle. \tag{2.84}$$

Thus $E|\psi\rangle$ remains within the code space.

Consequently, errors belonging to the normalizer cannot be detected through stabilizer measurements, since they do not change any syndrome outcomes.

Two possibilities arise:

1. $E \in S$,

2. $E \in N(S) \setminus S$.

If $E \in S$, then

$$E|\psi\rangle = |\psi\rangle \tag{2.85}$$

for every codeword, and therefore E acts trivially on the encoded information.

In contrast, if

$$E \in N(S) \setminus S, \tag{2.86}$$

then E maps the code space onto itself while acting nontrivially on the logical degrees of freedom. Such operators correspond to logical operations and constitute undetectable logical errors.

Errors Outside the Normalizer Now consider an error operator

$$E \notin N(S). \tag{2.87}$$

By definition, there exists at least one stabilizer generator $g \in S$ such that

$$\{E, g\} = 0. \tag{2.88}$$

For a codeword $|\psi\rangle \in \mathcal{C}$,

$$g(E|\psi\rangle) = -gE|\psi\rangle \tag{2.89}$$
$$= -Eg|\psi\rangle \tag{2.90}$$
$$= -E|\psi\rangle. \tag{2.91}$$

Therefore $E|\psi\rangle$ is a -1 eigenstate of g.

Since the code space consists exclusively of states that are $+1$ eigenstates of every stabilizer generator, the state $E|\psi\rangle$ is orthogonal to the code space.

Hence any Pauli error that anticommutes with at least one stabilizer generator produces a nontrivial syndrome and is therefore detectable.

These observations lead to the following characterization:

- Errors in S act trivially.
- Errors in $N(S) \setminus S$ are undetectable logical operators.
- Errors in $\mathcal{P}_n \setminus N(S)$ are detectable.

Distance of a Stabilizer Code

The distance of a stabilizer code is determined by the minimum weight of an undetectable logical operator.

Before defining code distance, it is useful to introduce the notion of operator weight.

Definition 2.4 *The weight of a Pauli operator*

$$E = P_1 \otimes P_2 \otimes \cdots \otimes P_n \tag{2.92}$$

is the number of tensor factors that are different from the identity operator.

For example,

$$X_1 Z_4 \tag{2.93}$$

has weight two, while

$$X_1 X_2 X_3 \tag{2.94}$$

has weight three.

The distance of a stabilizer code is then defined as follows.

Definition 2.5 *The distance d of a stabilizer code is*

$$d = \min_{E \in N(S) \setminus S} \mathrm{wt}(E), \tag{2.95}$$

where $\mathrm{wt}(E)$ *denotes the weight of the Pauli operator E.*

Thus the distance is the minimum weight of a logical operator that preserves the code space while acting nontrivially on the encoded information.

Since such operators are undetectable, they represent the smallest physical error capable of producing a logical failure.

A stabilizer code with distance d can detect up to

$$d - 1 \tag{2.96}$$

physical qubit errors and can correct

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor \tag{2.97}$$

arbitrary errors.

Quantum Error-Correction Condition

The stabilizer formalism provides a simple criterion for determining whether a set of errors is correctable.

Let

$$\mathcal{E} = \{E_i\} \tag{2.98}$$

denote a collection of Pauli errors.

A sufficient condition for correctability is that for every pair of errors E_i and E_j,

$$E_i^\dagger E_j \notin N(S) \setminus S. \tag{2.99}$$

Equivalently,

$$E_i^\dagger E_j \in S \tag{2.100}$$

or

$$E_i^\dagger E_j \in \mathcal{P}_n \setminus N(S). \tag{2.101}$$

To see this, let

$$P = \prod_{m=1}^{n-k} \frac{I + g_m}{2} \tag{2.102}$$

denote the projector onto the code space.

If

$$E_i^\dagger E_j \in \mathcal{P}_n \setminus N(S), \tag{2.103}$$

then $E_i^\dagger E_j$ anticommutes with at least one stabilizer generator.

Using the projector representation, one finds

$$P E_i^\dagger E_j P = 0. \tag{2.104}$$

Thus distinct correctable errors map the code space into mutually orthogonal syndrome subspaces and can therefore be distinguished by syndrome measurements.

This result constitutes the stabilizer version of the Knill–Laflamme error-correction condition and forms the mathematical foundation of stabilizer-code decoding. [2]

Logical Operators

A stabilizer code encodes quantum information into the code space

$$\mathcal{C} = \{|\psi\rangle \mid s|\psi\rangle = |\psi\rangle, \quad \forall s \in S\}. \tag{2.105}$$

For an $[[n, k, d]]$ stabilizer code, the dimension of the code space is

$$\dim(\mathcal{C}) = 2^k. \tag{2.106}$$

Consequently, the code space may be viewed as the Hilbert space of k logical qubits.

To manipulate the encoded information, one requires operators that act non-trivially within the code space while preserving the stabilizer constraints. Such operators are known as logical operators.

Definition 2.6 *A logical operator is an operator that preserves the code space while acting nontrivially on the encoded logical qubits.*

If L is a logical operator, then for every codeword

$$|\psi\rangle \in \mathcal{C}, \tag{2.107}$$

one must have

$$L|\psi\rangle \in \mathcal{C}. \tag{2.108}$$

Therefore L must commute with every stabilizer element.

$$[L, s] = 0, \qquad \forall s \in S. \tag{2.109}$$

It follows that every logical operator belongs to the normalizer,

$$L \in N(S). \tag{2.110}$$

However, not every element of the normalizer corresponds to a nontrivial logical operation.

Indeed, every stabilizer element satisfies

$$s|\psi\rangle = |\psi\rangle, \qquad \forall |\psi\rangle \in \mathcal{C}, \tag{2.111}$$

and therefore acts trivially on the encoded information.

Consequently, logical operators are identified with equivalence classes

$$N(S)/S. \tag{2.112}$$

Two operators that differ by a stabilizer element produce the same action on all encoded states and therefore represent the same logical operator.

Logical Pauli Operators: Since the code space encodes k logical qubits, one may define k logical Pauli-X operators and k logical Pauli-Z operators,

$$\overline{X}_1, \ldots, \overline{X}_k, \tag{2.113}$$

and

$$\overline{Z}_1, \ldots, \overline{Z}_k. \tag{2.114}$$

These operators satisfy the same algebra as ordinary Pauli operators.

For each logical qubit i,

$$\overline{X}_i^2 = \overline{Z}_i^2 = I, \tag{2.115}$$

and

$$\overline{X}_i \overline{Z}_i = -\overline{Z}_i \overline{X}_i. \tag{2.116}$$

For different logical qubits,

$$[\overline{X}_i, \overline{Z}_j] = 0, \qquad i \neq j. \tag{2.117}$$

Thus the logical operators reproduce the Pauli algebra on the encoded qubits.

Example:
Three-Qubit Repetition Code:
Consider,

$$|0_L\rangle = |000\rangle, \tag{2.118}$$

$$|1_L\rangle = |111\rangle. \tag{2.119}$$

The stabilizer group is generated by

$$S = \langle Z_1 Z_2, Z_2 Z_3 \rangle. \tag{2.120}$$

Any logical operator must commute with both stabilizer generators.

A logical bit-flip operation is implemented by

$$\overline{X} = X_1 X_2 X_3. \tag{2.121}$$

Indeed,

$$\overline{X}|0_L\rangle = |1_L\rangle, \tag{2.122}$$

and

$$\overline{X}|1_L\rangle = |0_L\rangle. \tag{2.123}$$

Similarly, a logical phase operator may be chosen as

$$\overline{Z} = Z_1. \tag{2.124}$$

Since

$$\overline{Z}|0_L\rangle = |0_L\rangle, \tag{2.125}$$

and

$$\overline{Z}|1_L\rangle = -|1_L\rangle, \tag{2.126}$$

the operator acts as the logical Pauli-Z on the encoded qubit.

These logical operators belong to the normalizer of the stabilizer group but are not themselves stabilizer elements. Consequently, they represent nontrivial encoded operations.

The characterization of logical operators through the quotient group $N(S)/S$ is one of the most important results of the stabilizer formalism. In topological quantum codes, logical operators acquire a geometric interpretation as nontrivial topological cycles on the underlying lattice, a perspective that will be central to the discussion of toric and surface codes in subsequent sections.

2.2.8 CSS Codes

Calderbank–Shor–Steane (CSS) codes constitute an important subclass of stabilizer codes in which the stabilizer generators can be separated into purely X-type and purely Z-type operators [9] [10]. This structure considerably simplifies both the analysis and decoding of the code and forms the foundation of many topological quantum error-correcting codes, including surface codes and toric codes.

The CSS construction establishes a direct connection between classical linear error-correcting codes and quantum stabilizer codes. As a result, many concepts from classical coding theory naturally reappear in the study of quantum error correction.

For completeness, we begin by recalling the definition of a classical binary linear code.

Definition 2.7 *An $[n, k, d]$ linear code $\mathcal{C}$ over $\mathbb{F}_{2^m}$ is a k-dimensional subspace of $\mathbb{F}_{2^m}^n$ with minimum distance d. The members of the code $\mathcal{C}$ are known as* codewords.

We now introduce the dual of a classical linear code, which plays a fundamental role in the construction of quantum CSS codes.

Definition 2.8 *Let $\mathcal{C}$ be an $[n, k, d]$ linear code over $\mathbb{F}_{2^m}$. The* dual code *of $\mathcal{C}$, denoted by $\mathcal{C}^\perp$, is defined as*

$$\mathcal{C}^\perp = \left\{ \mathbf{v} \in \mathbb{F}_{2^m}^n : \mathbf{v} \cdot \mathbf{c} = 0 \text{ for all } \mathbf{c} \in \mathcal{C} \right\}, \tag{2.127}$$

where $\mathbf{v} \cdot \mathbf{c}$ denotes the Euclidean inner product defined as follows:

$$\mathbf{v} \cdot \mathbf{c} = \sum_{i=1}^{n} v_i c_i. \tag{2.128}$$

The code $\mathcal{C}^\perp$ is an $[n, n-k, d^\perp]$ linear code over $\mathbb{F}_{2^m}$, where $d^\perp$ denotes the minimum distance of $\mathcal{C}^\perp$.

Based on the preceding definitions, we are now ready to define the CSS code.

Definition 2.9 *[10] Let C_1 and C_2 be binary linear codes with parameters $[n, k_1, d_1]$ and $[n, k_2, d_2]$, respectively, satisfying $C_2^\perp \subseteq C_1$. Then there exists a quantum stabilizer code with parameters $[[n, k_1 + k_2 - n, d]]$, where*

$$d = \min \left\{ \mathrm{wt}(c) \;\middle|\; c \in (C_1 \setminus C_2^\perp) \cup (C_2 \setminus C_1^\perp) \right\}. \tag{2.129}$$

The resulting code is referred to as a Calderbank-Shor-Steane (CSS) code.

The stabilizer generator matrix of the constructed CSS code is defined as follows:

$$H_{CSS} = \left[\begin{array}{c|c} H_1 & \mathbf{0} \\ \hline \mathbf{0} & H_2 \end{array}\right], \tag{2.130}$$

where H_1 and H_2 are the parity-check matrices of the classical codes C_1 and C_2, respectively. The structure of the stabilizer matrix in Equation (2.130) reflects the CSS construction, which separates the correction of bit-flip and phase-flip errors. In particular, the rows corresponding to H_1 generate stabilizer elements containing only Pauli-(X) operators, while the rows corresponding to H_2 generate stabilizer elements containing only Pauli-(Z) operators. Consequently, the stabilizer generators are either pure (X)-type or pure (Z)-type operators, with no mixed (X)- and (Z)-components. This separation significantly simplifies both the encoding and decoding procedures and enables independent treatment of bit-flip and phase-flip errors using the underlying classical codes.

Logical Operators

The CSS structure naturally separates the correction of bit-flip and phase-flip errors.

The Z-type stabilizers detect X-errors, while the X-type stabilizers detect Z-errors.

Similarly, logical operators can be divided into X-type and Z-type logical operators.

Logical X-operators belong to

$$\ker(H_Z) \setminus \mathrm{row}(H_X), \tag{2.131}$$

while logical Z-operators belong to

$$\ker(H_X) \setminus \mathrm{row}(H_Z). \tag{2.132}$$

These operators commute with all stabilizers but are not themselves stabilizer elements, thereby representing nontrivial logical operations on the encoded qubits.

Connection to Topological Codes

The CSS construction provides a natural bridge between classical coding theory and topological quantum error correction.

In topological codes, the parity-check matrices H_X and H_Z acquire a geometric interpretation. Their rows correspond to local constraints associated with geometric objects such as vertices, edges, and faces of a lattice.

For example, in the toric code and surface code, physical qubits are associated with edges of a lattice. The parity-check matrices encode incidence relations between edges and vertices or between edges and plaquettes. Consequently, the condition

$$H_X H_Z^T = 0 \tag{2.133}$$

becomes a statement about the topology of the underlying lattice.

The algebraic structure of CSS codes admits a natural interpretation in terms of chain complexes and homology. In this framework, the parity-check matrices can be identified with boundary operators acting on geometric objects of different dimensions. This perspective provides a powerful connection between quantum error correction and algebraic topology and forms the foundation of topological quantum codes. We discuss this connection in the next subsection.

2.2.9 Homological Perspective of CSS Codes

The CSS construction described in the previous section admits a natural interpretation in terms of algebraic topology. This connection provides the mathematical foundation for a large family of topological quantum error-correcting codes, including surface codes and toric codes. In particular, the parity-check matrices

defining a CSS code can be viewed as boundary operators acting on geometric objects of different dimensions. This perspective establishes a deep relationship between quantum error correction and homology theory [11, 4, 8].

Cell Complexes

A topological space may be decomposed into elementary geometric objects known as cells. For the purposes of quantum error correction, it is sufficient to consider three types of cells:

- 0-cells (vertices),
- 1-cells (edges),
- 2-cells (faces or plaquettes).

A collection of cells assembled according to suitable gluing rules forms a cell complex. Figure references may later be added to illustrate a two-dimensional lattice consisting of vertices, edges, and plaquettes.

Throughout this thesis, topological quantum codes will be constructed from such cell complexes.

Chain Groups

Given a cell complex, one may define vector spaces generated by cells of a fixed dimension. Working over the field $\mathbb{F}_2$, the chain groups are defined as

$$C_0 = \mathrm{span}_{\mathbb{F}_2}\{\text{vertices}\}, \tag{2.134}$$

$$C_1 = \mathrm{span}_{\mathbb{F}_2}\{\text{edges}\}, \tag{2.135}$$

and

$$C_2 = \mathrm{span}_{\mathbb{F}_2}\{\text{faces}\}. \tag{2.136}$$

Elements of these vector spaces are called chains.

For example, an element

$$c = e_1 + e_3 + e_5 \in C_1 \tag{2.137}$$

represents a collection of edges. Since arithmetic is performed modulo two, adding the same edge twice yields zero.

The dimensions of the chain groups are equal to the number of cells of each type,

$$\dim(C_0) = N_V, \tag{2.138}$$

$$\dim(C_1) = N_E, \tag{2.139}$$

$$\dim(C_2) = N_F, \tag{2.140}$$

where N_V, N_E, and N_F denote the numbers of vertices, edges, and faces, respectively.

Boundary Operators

The geometric structure of a cell complex is encoded by boundary operators,

$$\partial_1 : C_1 \to C_0, \tag{2.141}$$

and

$$\partial_2 : C_2 \to C_1. \tag{2.142}$$

The operator ∂_1 maps an edge to its boundary vertices, while ∂_2 maps a face to the collection of edges forming its boundary.

For an edge e connecting vertices v_1 and v_2,

$$\partial_1(e) = v_1 + v_2. \tag{2.143}$$

Similarly, if a face f is bounded by edges e_1, e_2, e_3, e_4, then

$$\partial_2(f) = e_1 + e_2 + e_3 + e_4. \tag{2.144}$$

These operators may be represented by binary matrices. In this representation, the rows and columns encode incidence relations between cells of different dimensions.

Chain Complexes

The chain groups and boundary operators form a sequence

$$C_2 \xrightarrow{\partial_2} C_1 \xrightarrow{\partial_1} C_0. \tag{2.145}$$

A fundamental property of boundary operators is that the boundary of a boundary is empty. Algebraically, this statement is expressed as

$$\partial_1 \partial_2 = 0. \tag{2.146}$$

This relation is the topological origin of the CSS commutation condition

$$H_X H_Z^T = 0. \tag{2.147}$$

As will be seen later, the matrices H_X and H_Z can be identified with suitable boundary operators of the underlying cell complex.

Cycles and Boundaries

The boundary operators introduced in the previous subsection allow us to distinguish between two important classes of chains: cycles and boundaries. These objects play a central role in the homological description of topological quantum codes.

Cycles A cycle is a chain whose boundary vanishes.

More precisely, a 1-chain

$$c \in C_1 \tag{2.148}$$

is called a *cycle* if

$$\partial_1 c = 0. \tag{2.149}$$

The set of all cycles forms a vector space known as the cycle space,

$$Z_1 = \ker(\partial_1). \tag{2.150}$$

Geometrically, a 1-cycle corresponds to a collection of edges that forms a closed loop. Since every vertex in the loop is incident to an even number of edges, the boundary of the chain vanishes.

For example, the boundary of a closed square loop is

$$\partial_1(e_1 + e_2 + e_3 + e_4) = 0, \tag{2.151}$$

and therefore the loop belongs to the cycle space Z_1.

Boundaries A boundary is a chain that arises as the boundary of a higher-dimensional object.

More precisely, a 1-chain

$$b \in C_1 \tag{2.152}$$

is called a *boundary* if there exists a 2-chain

$$f \in C_2 \tag{2.153}$$

such that

$$b = \partial_2 f. \tag{2.154}$$

The collection of all boundaries forms the boundary space

$$B_1 = \mathrm{im}(\partial_2). \tag{2.155}$$

Geometrically, a boundary corresponds to the perimeter of a collection of faces.

For instance, if a plaquette p is bounded by edges

$$e_1, e_2, e_3, e_4, \tag{2.156}$$

then

$$\partial_2 p = e_1 + e_2 + e_3 + e_4 \tag{2.157}$$

defines a boundary cycle.

Every Boundary is a Cycle A fundamental consequence of the relation

$$\partial_1 \partial_2 = 0 \tag{2.158}$$

is that every boundary is automatically a cycle.

Indeed, if

$$b = \partial_2 f, \tag{2.159}$$

then

$$\partial_1 b = \partial_1(\partial_2 f) = (\partial_1 \partial_2) f = 0. \tag{2.160}$$

Therefore,

$$B_1 \subseteq Z_1. \tag{2.161}$$

Geometrically, this result expresses the intuitive statement that the boundary of a surface always forms a closed loop.

However, not every cycle is necessarily a boundary. On topologically nontrivial surfaces there may exist closed loops that cannot be expressed as the boundary of any collection of faces. Such cycles are referred to as nontrivial cycles and will later correspond to logical operators of topological quantum codes.

The distinction between cycles and boundaries is quantified by the homology groups introduced in the next subsection.

Homology Groups

The previous subsection established that every boundary is a cycle,

$$B_1 = \mathrm{im}(\partial_2) \subseteq \ker(\partial_1) = Z_1. \tag{2.162}$$

However, the converse is not necessarily true. On a topologically nontrivial surface there may exist closed loops that cannot be expressed as the boundary of any collection of faces. Such loops represent global topological features of the underlying space.

The purpose of homology theory is to characterize precisely these nontrivial cycles.

Equivalence Classes of Cycles: Two cycles

$$z, z' \in Z_1 \tag{2.163}$$

are said to be homologous if they differ by a boundary. That is,

$$z - z' = \partial_2 c \tag{2.164}$$

for some

$$c \in C_2. \tag{2.165}$$

Since arithmetic is performed over $\mathbb{F}_2$, subtraction and addition are identical, and the condition may be written as

$$z + z' = \partial_2 c. \tag{2.166}$$

Intuitively, two cycles are considered equivalent if one can be continuously deformed into the other through the addition or removal of boundaries.

Consequently, all cycles that differ only by boundaries are grouped into the same equivalence class.

The First Homology Group: The collection of equivalence classes of non-trivial 1-cycles is described by the first homology group,

$$H_1 = \frac{Z_1}{B_1} = \frac{\ker(\partial_1)}{\mathrm{im}(\partial_2)}. \tag{2.167}$$

Elements of H_1 are therefore equivalence classes of cycles modulo boundaries.

A cycle belonging to the trivial homology class can be written as the boundary of a collection of faces,

$$z = \partial_2 c. \tag{2.168}$$

Such cycles are regarded as topologically trivial.

In contrast, cycles that cannot be expressed as boundaries belong to nontrivial homology classes and represent genuine topological features of the underlying surface.

Example: The Torus: The distinction between trivial and nontrivial cycles is most easily understood through the example of a torus.

Consider a loop that surrounds a single plaquette. Since it forms the boundary of that plaquette, it belongs to the trivial homology class.

By contrast, a loop that winds around the torus cannot be expressed as the boundary of any collection of plaquettes. Such a loop therefore represents a nontrivial homology class.

On a torus there exist two independent classes of non-contractible loops:

1. loops winding around the meridional direction,

2. loops winding around the longitudinal direction.

These two independent classes generate the first homology group of the torus.

Dimension of the Homology Group: Since H_1 is a vector space over $\mathbb{F}_2$, its dimension provides a measure of the number of independent nontrivial cycles.

The quantity

$$\beta_1 = \dim(H_1) \tag{2.169}$$

is known as the first Betti number.

For a closed orientable surface of genus g,

$$\beta_1 = 2g. \tag{2.170}$$

Consequently,

$$\dim(H_1) = 2g. \tag{2.171}$$

This result will play a central role in the construction of topological quantum codes. In particular, the number of encoded logical qubits of a homological CSS code is directly related to the dimension of the first homology group.

Connection to Quantum Error Correction: The homological viewpoint provides a natural interpretation of the logical operators of a CSS code.

Boundaries correspond to stabilizer generators and therefore act trivially on the encoded quantum information. Cycles correspond to operators that commute with the stabilizer group. Two cycles differing by a boundary represent the same logical action since they differ only by stabilizer operations.

As a result, nontrivial logical operators are naturally associated with nontrivial homology classes. The number of encoded logical qubits is given by

$$k = \dim(H_1). \tag{2.172}$$

This observation establishes the fundamental connection between algebraic topology and quantum error correction. In topological quantum codes, logical qubits are encoded in global topological degrees of freedom represented by nontrivial homology classes of the underlying surface.

CSS Codes from Chain Complexes

The homological framework introduced in the previous subsections provides a natural method for constructing CSS quantum codes. In this formulation, the parity-check matrices defining the code arise directly from the boundary operators of a chain complex.

Consider the chain complex

$$C_2 \xrightarrow{\partial_2} C_1 \xrightarrow{\partial_1} C_0, \tag{2.173}$$

where C_0, C_1, and C_2 denote the vector spaces generated by vertices, edges, and faces of a cell complex, respectively.

The physical qubits of the quantum code are associated with the 1-cells (edges) of the complex. Consequently, the number of physical qubits is

$$n = \dim(C_1). \tag{2.174}$$

The boundary operators can be represented as binary matrices over $\mathbb{F}_2$. These matrices naturally define the parity-check matrices of a CSS code.

Specifically, the X-type parity-check matrix is identified with the vertex-edge incidence matrix,

$$H_X = [\partial_1]^T, \tag{2.175}$$

while the Z-type parity-check matrix is identified with the face-edge incidence matrix,

$$H_Z = [\partial_2]. \tag{2.176}$$

The rows of H_X correspond to vertices of the complex and generate X-type stabilizers, whereas the rows of H_Z correspond to faces and generate Z-type stabilizers.

The CSS commutation condition follows immediately from the chain-complex structure. Since

$$\partial_1 \partial_2 = 0, \tag{2.177}$$

we obtain

$$H_Z H_X^T = [\partial_2][\partial_1] = 0. \tag{2.178}$$

Taking the transpose yields

$$H_X H_Z^T = 0. \tag{2.179}$$

Thus the requirement that all X-type and Z-type stabilizers commute is a direct consequence of the topological identity that the boundary of a boundary is empty.

Stabilizers as Boundaries Within the homological description, stabilizer generators acquire a simple geometric interpretation.

Each row of H_Z corresponds to the boundary of a face. Therefore, an Z-type stabilizer acts on all edges surrounding a plaquette.

Similarly, each row of H_X corresponds to the set of edges incident on a vertex. Consequently, a X-type stabilizer acts on all edges adjacent to a vertex.

The stabilizer group is therefore generated by local geometric constraints associated with the underlying cell complex.

Logical Operators and Homology The homological interpretation also provides a natural description of logical operators.

Recall that X-type logical operators correspond to elements of

$$\ker(H_Z) \setminus \mathrm{row}(H_X), \tag{2.180}$$

while Z-type logical operators correspond to

$$\ker(H_X) \setminus \mathrm{row}(H_Z). \tag{2.181}$$

Using the identifications

$$H_Z = \partial_2, \qquad H_X = \partial_1^T, \tag{2.182}$$

the X-type logical operators are represented by cycles that are not boundaries,

$$\ker(\partial_1) \setminus \mathrm{im}(\partial_2). \tag{2.183}$$

Consequently, logical operators are naturally associated with nontrivial homology classes of the cell complex.

Two cycles differing by a boundary correspond to operators that differ only by stabilizer generators and therefore implement the same logical action on the encoded quantum information.

Number of Encoded Qubits The number of encoded logical qubits is determined by the dimension of the first homology group,

$$k = \dim(H_1), \tag{2.184}$$

where

$$H_1 = \frac{\ker(\partial_1)}{\mathrm{im}(\partial_2)}. \tag{2.185}$$

Using the rank-nullity theorem, this quantity may be expressed as

$$k = \dim(\ker(\partial_1)) - \dim(\mathrm{im}(\partial_2)). \tag{2.186}$$

Thus the number of encoded qubits is a topological invariant of the underlying cell complex.

For a closed orientable surface of genus g, the first homology group has dimension

$$\dim(H_1) = 2g. \tag{2.187}$$

Therefore, a homological CSS code defined on such a surface encodes

$$k = 2g \tag{2.188}$$

logical qubits.

This remarkable connection between topology and quantum information lies at the heart of topological quantum error correction. In the following section, these ideas will be realized explicitly through the construction of surface codes and toric codes.

2.3 Topological Quantum Codes

Topological quantum codes constitute an important class of quantum error-correcting codes in which encoded quantum information is associated with global topological features of a lattice rather than local physical degrees of freedom [4, 8, 5]. Unlike conventional stabilizer codes, whose construction is often described purely algebraically, topological codes admit a natural geometric interpretation through the language of homology introduced in the previous section.

The central idea underlying topological quantum error correction is to embed physical qubits into a cell complex and define stabilizer generators through local geometric constraints. The resulting codes inherit remarkable robustness against local errors because logical information is stored nonlocally in topological degrees of freedom. As a consequence, local perturbations cannot easily alter the encoded quantum state.

Among the various topological quantum codes proposed in the literature, the most prominent examples are the surface code and the toric code. Both codes are CSS stabilizer codes defined on two-dimensional lattices and possess local stabilizer measurements, high fault-tolerance thresholds, and natural homological interpretations. These properties have made them leading candidates for fault-tolerant quantum computation.

In the following subsections, we discuss the construction of surface codes and toric codes in detail and establish the homological framework that will ultimately be generalized to multigenus topological codes.

2.3.1 Surface Codes

Surface codes are among the most extensively studied quantum error-correcting codes due to their high fault-tolerance thresholds, local stabilizer measurements, and compatibility with two-dimensional nearest-neighbor architectures [12, 5]. They constitute a family of CSS stabilizer codes defined on a two-dimensional lattice and may be viewed as planar variants of Kitaev's toric code [4].

From the perspective of the previous section, surface codes provide a concrete realization of homological CSS codes. The code is constructed from a two-dimensional cell complex whose vertices, edges, and plaquettes determine the stabilizer structure and logical operators.

Lattice Construction

Consider a square lattice embedded on a planar surface. Physical qubits are placed on the edges of the lattice. Let,

$$V, \qquad E, \qquad F \tag{2.189}$$

denote the sets of vertices, edges, and plaquettes, respectively.

The number of physical qubits is therefore

$$n = |E|. \tag{2.190}$$

Unlike the toric code, which employs periodic boundary conditions, the surface code is defined on a finite planar lattice possessing boundaries. These boundaries play an important role in determining the number of encoded logical qubits.

A typical surface-code lattice consists of two types of boundaries:

- Rough boundaries,
- Smooth boundaries.

These boundaries correspond to different terminations of the stabilizer structure and allow the planar lattice to encode logical qubits despite having trivial global topology.
This has code parameter:

$$[[d^2 + (d-1)^2, 1, d]]_2$$

Stabilizer Generators

The surface code is a CSS code whose stabilizer generators are associated with vertices and plaquettes of the lattice.

For each vertex $v \in V$, one defines a vertex (or star) operator

$$A_v = \prod_{e \in \mathrm{star}(v)} X_e, \tag{2.191}$$

where the product extends over all edges incident on the vertex v.

Similarly, for each plaquette $p \in F$, one defines a plaquette operator

$$B_p = \prod_{e \in \partial p} Z_e, \tag{2.192}$$

where the product runs over the edges forming the boundary of the plaquette.

The stabilizer group is generated by all star and plaquette operators,

$$S = \langle A_v,\, B_p \rangle\,. \tag{2.193}$$

A codeword $|\psi\rangle$ belongs to the surface-code space if

$$A_v|\psi\rangle = |\psi\rangle, \qquad \forall v \in V, \tag{2.194}$$

and

$$B_p|\psi\rangle = |\psi\rangle, \qquad \forall p \in F. \tag{2.195}$$

Thus the code space is the common $+1$ eigenspace of all stabilizer generators.

Commutation Relations

The CSS structure guarantees that all stabilizer generators commute.

Two star operators commute because they consist exclusively of Pauli-X operators,

$$[A_v, A_{v'}] = 0. \tag{2.196}$$

Similarly, any two plaquette operators commute,

$$[B_p, B_{p'}] = 0. \tag{2.197}$$

The nontrivial case involves a star operator and a plaquette operator.

A vertex and a plaquette may share either zero or two common edges. Since each shared edge contributes one anticommutation between X and Z, the total number of anticommutations is always even. Consequently,

$$[A_v, B_p] = 0. \tag{2.198}$$

This geometric argument is the lattice manifestation of the CSS commutation condition

$$H_X H_Z^T = 0. \tag{2.199}$$

Equivalently, it reflects the homological identity

$$\partial_1 \partial_2 = 0. \tag{2.200}$$

Therefore, the stabilizer generators define a valid CSS quantum code.

Error Chains and Syndromes

One of the most appealing features of surface codes is the geometric interpretation of errors and syndromes. Physical errors can be represented as chains on the underlying lattice, while syndrome measurements reveal only the endpoints of these chains. This picture provides the foundation for efficient decoding algorithms and highlights the topological nature of error correction.

Consider a surface code defined on a lattice with physical qubits located on edges. A Pauli error acting on a collection of qubits may be represented by a chain of edges.

For example, an X-type error acting on a set of edges γ may be written as

$$E_X(\gamma) = \prod_{e \in \gamma} X_e, \tag{2.201}$$

where γ denotes a subset of lattice edges.

Similarly, a Z-type error chain is represented by

$$E_Z(\gamma) = \prod_{e \in \gamma} Z_e. \tag{2.202}$$

Since the surface code is a CSS code, bit-flip and phase-flip errors can be analyzed independently.

Syndromes of X-Type Errors An X-error commutes with all vertex stabilizers A_v, since both are composed entirely of Pauli-X operators.

However, an X-error may anticommute with plaquette stabilizers B_p. In particular, a plaquette stabilizer changes sign whenever it shares an odd number of edges with the error chain.

Consequently, the syndrome associated with an X-error is obtained by measuring the plaquette stabilizers,

$$B_p = \pm 1. \tag{2.203}$$

Plaquettes for which

$$B_p = -1 \tag{2.204}$$

are referred to as syndrome defects.

For a connected error chain, syndrome defects appear only at the endpoints of the chain. Intermediate portions of the chain affect neighboring plaquettes in pairs, causing the corresponding anticommutations to cancel.

Thus syndrome measurements reveal the boundary of the error chain rather than the chain itself.

Syndromes of Z-Type Errors The situation is completely analogous for Z-type errors.

A Z-error chain commutes with all plaquette stabilizers but may anticommute with vertex stabilizers. The corresponding syndrome is therefore obtained from measurements of the vertex operators,

$$A_v = \pm 1. \tag{2.205}$$

Vertices with outcome

$$A_v = -1 \tag{2.206}$$

indicate the presence of syndrome defects.

As before, defects occur only at the endpoints of the error chain.

Error Chains and Equivalence Classes An important feature of the surface code is that syndrome measurements do not uniquely determine the underlying error chain.

Suppose two error chains γ and γ' possess the same endpoints. Then

$$\partial\gamma = \partial\gamma'. \tag{2.207}$$

The difference between the two chains is a cycle,

$$\gamma + \gamma' \in \ker(\partial). \tag{2.208}$$

Consequently, many distinct error configurations may produce exactly the same syndrome pattern.

From the perspective of error correction, the objective of the decoder is therefore not necessarily to identify the exact physical error. Instead, it is sufficient to determine an equivalent correction chain that returns the system to the original logical state.

Decoding Problem Let E denote the actual physical error and let R denote the recovery operation inferred from the syndrome.

The combined operator

$$RE \tag{2.209}$$

determines whether the recovery is successful.

If RE corresponds to a stabilizer element, then the logical state is restored successfully. However, if RE differs from a stabilizer by a nontrivial logical operator, a logical error occurs.

Therefore, decoding may be viewed as the problem of determining the homology class of the error chain using only information about its endpoints.

This geometric interpretation forms the basis of minimum-weight perfect matching decoders and many other decoding algorithms used for surface and toric codes [8, 13, 14].

Logical Operators

As discussed in the previous subsection, syndrome measurements reveal only the endpoints of error chains and therefore do not uniquely determine the underlying error configuration. The reason for this ambiguity is the existence of nontrivial cycles on the lattice, which give rise to logical operators acting on the encoded quantum information.

Recall that the stabilizer generators of the surface code are associated with vertices and plaquettes. A product of stabilizer generators corresponds to a boundary and therefore acts trivially on the encoded state. Logical operators arise from chains that commute with all stabilizers but cannot be expressed as products of stabilizer generators.

Logical Z-Operators Consider a connected chain of edges γ_Z joining two rough boundaries of the lattice. The corresponding operator is

$$\overline{Z} = \prod_{e \in \gamma_Z} Z_e. \tag{2.210}$$

Since every vertex stabilizer intersects the chain on either zero or two edges, the operator commutes with all vertex stabilizers. Furthermore, it commutes with all plaquette stabilizers because they are composed entirely of Pauli-Z operators.

Thus,

$$[\overline{Z}, A_v] = 0, \qquad [\overline{Z}, B_p] = 0, \tag{2.211}$$

for all vertices v and plaquettes p. The operator therefore preserves the code space and acts as a logical operator.

Any deformation of the path γ_Z by multiplication with plaquette stabilizers produces an equivalent logical operator. Consequently, logical operators are associated with equivalence classes of paths rather than individual chains.

Logical X-Operators Similarly, consider a chain γ_X connecting two smooth boundaries. The corresponding operator is

$$\overline{X} = \prod_{e \in \gamma_X} X_e. \tag{2.212}$$

This operator commutes with all stabilizer generators and therefore preserves the code space.

Different representatives of the same logical operator may be obtained by multiplying by vertex stabilizers. Thus, logical X-operators are likewise defined only up to stabilizer equivalence.

Anticommutation Relation The logical operators must reproduce the Pauli algebra of an encoded qubit.

Let $\overline{X}$ and $\overline{Z}$ intersect on a single edge. On that edge, the corresponding Pauli operators satisfy

$$XZ = -ZX. \tag{2.213}$$

Since all remaining factors commute, one obtains

$$\overline{X}\,\overline{Z} = -\overline{Z}\,\overline{X}. \tag{2.214}$$

Thus the logical operators satisfy the same anticommutation relation as ordinary Pauli operators,

$$\{\overline{X}, \overline{Z}\} = 0. \tag{2.215}$$

This relation demonstrates that the surface code encodes a logical qubit whose computational degrees of freedom are represented by extended topological objects rather than individual physical qubits.

Homological Interpretation The homological framework developed earlier provides a natural interpretation of logical operators.

A product of plaquette stabilizers corresponds to a boundary and therefore acts trivially on the encoded state. Consequently, two Z-type chains differing by a boundary represent the same logical operator.

Similarly, products of vertex stabilizers generate trivial deformations of X-type chains.

Logical operators are therefore associated with equivalence classes of cycles modulo boundaries, that is, with elements of the first homology group

$$H_1 = \frac{\ker(\partial_1)}{\mathrm{im}(\partial_2)}. \tag{2.216}$$

In the planar surface code, the presence of boundaries allows nontrivial logical operators to be realized as strings connecting opposite boundaries. In contrast, for closed surfaces such as the torus, logical operators correspond to non-contractible loops. This distinction will become important in the discussion of toric codes and higher-genus topological codes.

Logical Failure A logical error occurs when the combination of the physical error chain and the recovery chain forms a nontrivial logical operator.

Let E denote the actual error and R the recovery operation inferred by the decoder. If

$$RE \in S, \tag{2.217}$$

where S is the stabilizer group, then the recovery is successful.

However, if

$$RE \in N(S) \setminus S, \tag{2.218}$$

then the resulting operator acts nontrivially on the encoded qubit and a logical failure occurs.

Therefore, successful decoding requires determining not the exact error chain but its homology class. This observation underlies the operation of modern decoding algorithms such as minimum-weight perfect matching and is central to the fault-tolerance properties of topological quantum codes.

Code Distance

The performance of a quantum error-correcting code is characterized by its distance, which determines the number of physical errors that can be reliably detected and corrected. For a stabilizer code with stabilizer group S and normalizer $N(S)$, the code distance is

$$d = \min_{L \in N(S) \setminus S} \mathrm{wt}(L), \tag{2.219}$$

where $\mathrm{wt}(L)$ denotes the number of physical qubits on which the operator L acts nontrivially [2, 1].

In the surface code, logical operators are represented by extended strings on the lattice. Consequently, the distance admits a simple geometric interpretation: it is the minimum length of a nontrivial logical string.

For a planar surface code of distance d, the shortest logical X- and Z-operators connect opposite boundaries of the lattice and contain d physical qubits. Therefore,

$$\mathrm{wt}(\overline{X}) = \mathrm{wt}(\overline{Z}) = d. \tag{2.220}$$

The code distance thus corresponds to the length of the shortest path capable of implementing a logical operation on the encoded qubit.

Distance Scaling An important advantage of topological quantum codes is that the code distance can be increased simply by enlarging the underlying lattice.

As the distance increases, the minimum weight of a logical operator also increases, making logical failures exponentially less likely when the physical error rate remains below the threshold value [8, 5].

This property forms the basis of fault-tolerant quantum computation. By increasing the lattice size, one can suppress logical error rates while preserving the locality of stabilizer measurements.

2.3.2 Rotated Surface Codes

The rotated surface code is a resource-efficient variant of the planar surface code that preserves the essential topological properties of the code while significantly reducing the number of physical qubits required to achieve a given code distance [5]. Owing to its lower qubit overhead and local stabilizer structure, the rotated surface code has become one of the most widely studied architectures for fault-tolerant quantum computation and serves as the basis for many modern surface-code implementations.

From a homological perspective, the rotated surface code encodes quantum information using the same topological principles as the standard surface code. The primary difference lies in the geometry of the lattice and the arrangement of stabilizer generators.

Motivation

For a given code distance d, the standard planar surface code requires a relatively large number of physical qubits. Many of these qubits arise from the rectangular arrangement of edges and plaquettes used in the lattice construction.

The rotated surface code exploits a more compact geometry obtained by rotating the square lattice by 45° and appropriately redefining the boundaries. This construction preserves the code distance while reducing the total number of physical qubits required.

As a result, the rotated surface code provides the same level of protection against logical errors with lower hardware overhead.

Lattice Construction

The rotated surface code is defined on a square lattice whose stabilizers are arranged in a checkerboard pattern.

Unlike the standard surface code, where qubits are often associated with lattice edges, the rotated formulation is commonly represented with physical qubits located

at lattice vertices. For a distance-d rotated surface code, the lattice contains

$$n = d^2 \tag{2.221}$$

physical qubits.

The stabilizers are arranged in alternating X- and Z-type plaquettes. Interior stabilizers typically act on four neighboring qubits, while boundary stabilizers may have reduced weight due to the finite extent of the lattice.

The boundaries are chosen such that one pair corresponds to rough boundaries and the other pair corresponds to smooth boundaries, thereby allowing the code to encode a single logical qubit.

Stabilizer Generators

The rotated surface code remains a CSS stabilizer code and therefore possesses separate X-type and Z-type stabilizers.

For each X-type plaquette p_X, one defines

$$S_X(p_X) = \prod_{i \in p_X} X_i, \tag{2.222}$$

while each Z-type plaquette p_Z defines

$$S_Z(p_Z) = \prod_{i \in p_Z} Z_i. \tag{2.223}$$

The stabilizer group is generated by all such plaquette operators,

$$S = \langle S_X(p_X), S_Z(p_Z) \rangle . \tag{2.224}$$

Because neighboring X- and Z-plaquettes share either zero or two qubits, all stabilizers commute,

$$[S_X, S_Z] = 0. \tag{2.225}$$

Thus the rotated surface code defines a valid CSS quantum code.

Logical Operators

The rotated surface code encodes a single logical qubit.

Logical operators are represented by strings of Pauli operators extending across the lattice.

A logical Z-operator may be chosen as a string of Z operators connecting the two rough boundaries,

$$\overline{Z} = \prod_{i \in \gamma_Z} Z_i, \tag{2.226}$$

while a logical X-operator is represented by a string of X operators connecting the two smooth boundaries,

$$\overline{X} = \prod_{i \in \gamma_X} X_i. \tag{2.227}$$

These operators commute with all stabilizers but are not themselves stabilizer elements. Furthermore, they intersect an odd number of times, implying

$$\overline{X}\,\overline{Z} = -\overline{Z}\,\overline{X}. \tag{2.228}$$

Consequently, they implement the Pauli algebra of a single encoded qubit.

Code Parameters

For a rotated surface code of distance d, the number of physical qubits is

$$n = d^2, \tag{2.229}$$

and the number of encoded qubits is

$$k = 1. \tag{2.230}$$

The code parameters may therefore be written as

$$[[d^2, 1, d]]. \tag{2.231}$$

The minimum weight of a nontrivial logical operator is equal to the code distance,

$$\mathrm{wt}(\overline{X}) = \mathrm{wt}(\overline{Z}) = d. \tag{2.232}$$

Advantages over Standard Surface Codes

The principal advantage of the rotated surface code is its improved encoding efficiency.

For a fixed code distance, the rotated geometry requires substantially fewer physical qubits than the standard planar construction while preserving the same logical error-correction capability. This reduction in hardware overhead has made the rotated surface code particularly attractive for both theoretical studies and experimental implementations.

Moreover, the local stabilizer structure is retained, ensuring compatibility with nearest-neighbor architectures and efficient decoding algorithms.

Because of these advantages, the rotated surface code is frequently employed as the outer code in concatenated bosonic-topological constructions, including the Surface-GKP code. In such schemes, each data qubit of the rotated surface code is replaced by a GKP-encoded oscillator, allowing continuous-variable error correction at the physical layer and topological error correction at the logical layer [7] [15].

2.3.3 Toric Codes

The toric code, introduced by Kitaev in 1997, is one of the earliest and most influential examples of a topological quantum error-correcting code [4]. It is defined on a $L \times L$ square lattice with periodic boundary conditions. The data qubits are placed on the edges of the lattice. The total number of data qubits is $n = 2L^2$. This code encodes 2 logical qubits. The minimum weight of the logical operator is L, so the distance of the code is $d = L$. The code parameters may therefore be written as

$$[[2d^2, 2, d]]. \tag{2.233}$$

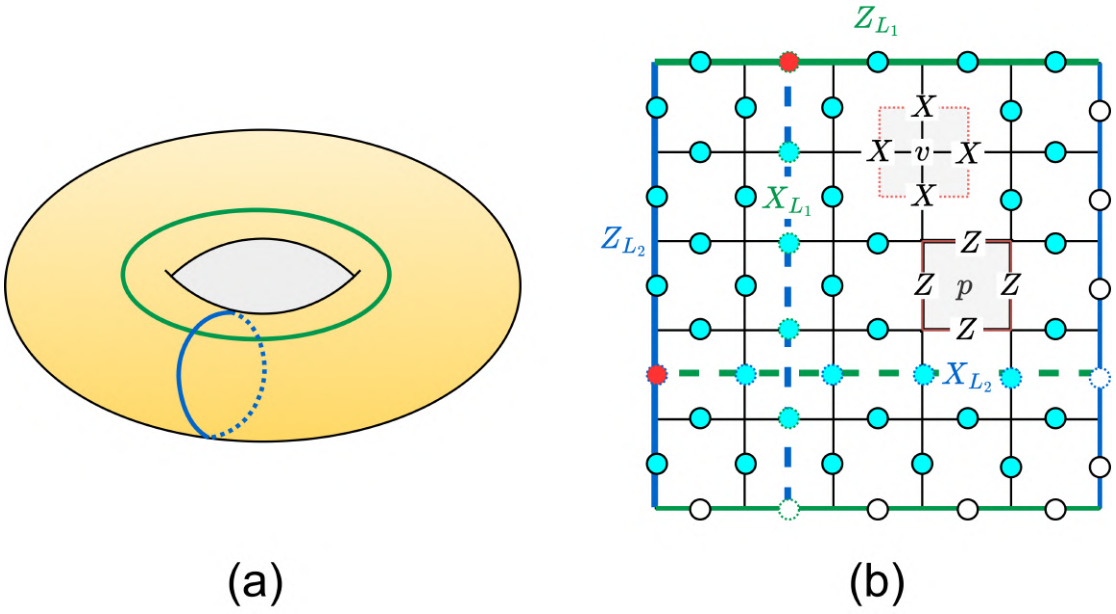


Figure 2.1: (a) The two non-trivial loops of the torus are shown as the green and the blue line. These two loops act as the logical operators for the toric code. (b) Plaquettes represent the Z stabilizers, and the vertices of the lattice represents the X stabilizers. One of those are shown in the figure. The green and the blue solid lines are the logical Z operators, whereas the the dotted lines are the loops in the dual lattice that represent the logical X operators. Note that the pair Z_{L_1} and X_{L_1} intersects at only one qubits hence they anti-commute, but they intersect with stabilizers only at even number of data qubits; hence, they commute with all the other stabilizers. The other logical pair Z_{L_2}, X_{L_2} and their intersecting point is also shown in the figure.

Stabilizers

There are two types of stabilizer operators.

- **Star operators:** It is defined on the four edges connected to the vertex v.

$$A_v = \prod_{i \in \delta(v)} X_i,$$

  where $\delta(v)$ is the co-boundary of the vertex v

- **Plaquette operators:**It is defined on the four edges on the boundary of the plaquette p.

$$B_p = \prod_{i \in \sigma(p)} Z_i,$$

  where $\sigma(p)$ is the boundary of the plaquette p.

Logical operators

The logical operators correspond to the non-trivial loops of the torus. From the algebraic topology perspective, we can find that the first homology group of the torus is

$$H_1(\mathbb{T}^2) \cong \mathbb{Z}_2 \oplus \mathbb{Z}_2$$

The two non-trivial loops are shown in the Figure 2.2(a). These non-trivial cycles on the primal lattice are the logical Z operators, and the logical X operators are defined by the non-trivial cycles on the dual lattice.

2.3.4 Rotated Toric Codes

As the similar discussion provided earlier, the rotated Toric code[16] is a resource-efficient variant of the Toric code that significantly reduces the number of physical qubits.

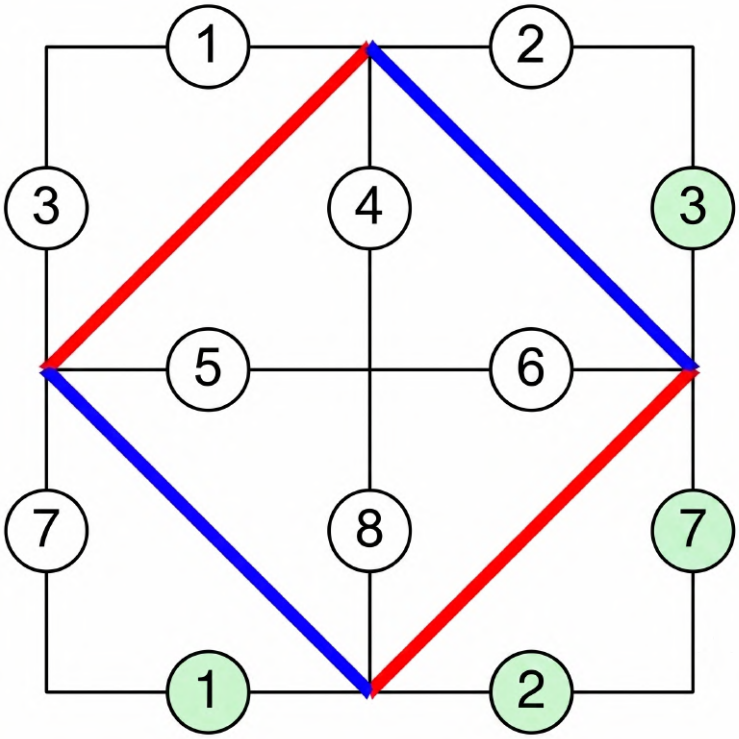


Figure 2.2: Example: After rotating the $[[8, 2, 2]]$ Toric code to 45 degrees, we can get $[[4, 2, 2]]$ code. Both have distance two, but the latter uses only half the number of qubits.

Now, the code parameters can be written as

$$[[d^2, 2, d]]. \tag{2.234}$$

2.4 Continuous-Variable (CV) Quantum Error Correction

The quantum error-correcting codes discussed thus far encode information into finite-dimensional quantum systems, typically qubits. Within this discrete-variable framework, quantum information is protected by distributing logical states across multiple physical qubits and employing stabilizer measurements to detect and correct errors. Topological codes such as surface codes and toric codes provide particularly powerful realizations of this approach, offering high error thresholds, local stabilizer measurements, and protection based on global topological properties of the underlying lattice.

Despite their remarkable properties, qubit-based encodings are not the only route toward fault-tolerant quantum information processing. Many experimentally relevant quantum systems are naturally described by continuous degrees of freedom rather than two-level systems. Examples include electromagnetic cavity modes, optical fields, and mechanical resonators, all of which are well approximated by quantum harmonic oscillators. Such systems possess infinite-dimensional Hilbert spaces and therefore provide fundamentally different opportunities for encoding and protecting quantum information.

The dominant noise processes affecting continuous-variable systems also differ from those encountered in qubit-based architectures. Whereas discrete-variable quantum error correction typically focuses on Pauli bit-flip and phase-flip errors, continuous-variable systems are more naturally subject to small displacement errors in phase space. Consequently, new error-correction strategies are required to protect quantum information against these forms of noise.

Continuous-variable quantum error correction addresses this challenge by encoding logical information into carefully engineered states of harmonic oscillators. Among the various continuous-variable codes proposed in the literature, the Gottesman–Kitaev–Preskill (GKP) code occupies a particularly important position. The GKP code encodes a qubit into an oscillator through highly structured lattice states in phase space, thereby converting small displacement errors into correctable logical errors.

An especially attractive feature of the GKP encoding is its compatibility with concatenated error-correction architectures. In such schemes, a bosonic code serves as the first layer of protection against continuous-variable noise, while a higher-level stabilizer code corrects the residual logical errors. Recent studies have demonstrated that concatenating GKP codes with topological codes can substantially improve logical performance and fault-tolerance thresholds under experimentally relevant noise models.

Since the primary focus of this thesis is the study of GKP-concatenated topological codes, it is necessary to first develop the theoretical framework of continuous-variable quantum information and GKP error correction. The remainder of this chapter therefore introduces the quantum harmonic oscillator, phase-space methods,

displacement operators, and the Gottesman–Kitaev–Preskill code, which together form the foundation of the concatenated coding schemes investigated in later chapters.

2.4.1 Quantum Harmonic Oscillator and Phase Space

Continuous-variable quantum information is most naturally described using the quantum harmonic oscillator, which serves as the fundamental model for a wide variety of physical systems, including optical modes, microwave cavities, mechanical resonators, and collective excitations of quantum fields [1, 6, 17]. Unlike a qubit, whose state space is two-dimensional, a quantum harmonic oscillator possesses an infinite-dimensional Hilbert space. This enlarged state space provides additional flexibility for encoding and protecting quantum information.

The Hamiltonian of a quantum harmonic oscillator is given by [17]

$$\hat{H} = \frac{\hat{p}^2}{2m} + \frac{1}{2}m\omega^2\hat{q}^2, \tag{2.235}$$

where m denotes the mass of the oscillator, ω is its angular frequency, and $\hat{q}$ and $\hat{p}$ are the position and momentum operators, respectively.

The position and momentum operators satisfy the canonical commutation relation

$$[\hat{q}, \hat{p}] = i\hbar. \tag{2.236}$$

This noncommutativity leads directly to the Heisenberg uncertainty principle,

$$\Delta q \, \Delta p \geq \frac{\hbar}{2}. \tag{2.237}$$

For quantum information applications it is convenient to introduce dimensionless quadrature operators. Setting $\hbar = 1$, one defines

$$\hat{q} = \frac{1}{\sqrt{2}}\left(\hat{a} + \hat{a}^\dagger\right), \tag{2.238}$$

and

$$\hat{p} = \frac{-i}{\sqrt{2}}\left(\hat{a} - \hat{a}^\dagger\right), \tag{2.239}$$

where $\hat{a}$ and $\hat{a}^\dagger$ denote the annihilation and creation operators, respectively.

The ladder operators eg. annihilation or creation operators, satisfy the bosonic commutation relation

$$[\hat{a}, \hat{a}^\dagger] = 1. \tag{2.240}$$

In terms of these operators, the Hamiltonian can be expressed as

$$\hat{H} = \omega\left(\hat{a}^\dagger\hat{a} + \frac{1}{2}\right). \tag{2.241}$$

The number operator is defined by

$$\hat{n} = \hat{a}^\dagger \hat{a}, \tag{2.242}$$

and its eigenstates, known as Fock states, satisfy

$$\hat{n}|n\rangle = n|n\rangle, \qquad n = 0, 1, 2, \ldots. \tag{2.243}$$

The corresponding energy eigenvalues are

$$E_n = \omega \left(n + \frac{1}{2} \right). \tag{2.244}$$

The set of Fock states

$$\{|0\rangle, |1\rangle, |2\rangle, \ldots\} \tag{2.245}$$

forms a complete orthonormal basis for the oscillator Hilbert space.

The ladder operators act on Fock states according to

$$\hat{a}|n\rangle = \sqrt{n}|n-1\rangle, \tag{2.246}$$

$$\hat{a}^\dagger|n\rangle = \sqrt{n+1}|n+1\rangle. \tag{2.247}$$

The vacuum state is defined by

$$\hat{a}|0\rangle = 0. \tag{2.248}$$

Repeated application of the creation operator generates higher number states:

$$|n\rangle = \frac{(\hat{a}^\dagger)^n}{\sqrt{n!}}|0\rangle. \tag{2.249}$$

While the Fock basis provides a natural description of energy eigenstates, continuous-variable quantum information is more naturally understood in terms of phase space. In this representation, a quantum state is characterized by its position and momentum quadratures, and noise processes correspond to displacements in phase space. Since Gottesman–Kitaev–Preskill (GKP) codes are fundamentally lattice codes defined in phase space, the phase-space description of the quantum harmonic oscillator plays a central role in continuous-variable quantum error correction.

The phase-space formulation and the associated displacement operators will be introduced in the next subsection.

Quadrature Operators

The fundamental observables of a continuous-variable quantum system are the position and momentum quadratures, represented by the operators $\hat{q}$ and $\hat{p}$. These operators play a role analogous to the Pauli operators in discrete-variable quantum systems and provide a complete description of the state of a quantum harmonic oscillator.

The quadrature operators satisfy the canonical commutation relation

$$[\hat{q}, \hat{p}] = i, \tag{2.250}$$

where we have adopted units with $\hbar = 1$.

Unlike classical observables, the noncommutativity of $\hat{q}$ and $\hat{p}$ implies that they cannot be simultaneously measured with arbitrary precision. This property leads directly to the Heisenberg uncertainty principle,

$$\Delta q \, \Delta p \geq \frac{1}{2}. \tag{2.251}$$

The expectation values

$$\langle \hat{q} \rangle, \qquad \langle \hat{p} \rangle, \tag{2.252}$$

determine the center of a quantum state in phase space, while the corresponding variances

$$(\Delta q)^2 = \langle \hat{q}^2 \rangle - \langle \hat{q} \rangle^2, \tag{2.253}$$

$$(\Delta p)^2 = \langle \hat{p}^2 \rangle - \langle \hat{p} \rangle^2, \tag{2.254}$$

characterize its spread in the two quadrature directions.

For the vacuum state $|0\rangle$, one finds

$$\langle \hat{q} \rangle = \langle \hat{p} \rangle = 0, \tag{2.255}$$

and

$$\Delta q = \Delta p = \frac{1}{\sqrt{2}}, \tag{2.256}$$

which saturates the uncertainty relation,

$$\Delta q \, \Delta p = \frac{1}{2}. \tag{2.257}$$

States satisfying this minimum uncertainty condition are known as minimum-uncertainty states and play a central role in continuous-variable quantum information.

The quadrature operators provide a natural coordinate system for describing oscillator states in phase space. In particular, displacement errors, which constitute the dominant noise mechanism considered in GKP error correction, correspond to small shifts of the quantum state along the q- and p-quadrature directions. Consequently, understanding the behavior of quantum states under quadrature displacements is essential for the development of continuous-variable quantum error-correcting codes.

To describe such displacements formally, it is convenient to introduce the phase-space representation of continuous-variable states and the associated displacement operators, which will be discussed in the next subsection.

2.4.2 Displacement Operator

The displacement operator generates translations in phase space and plays a central role in constructing GKP codewords. It is defined as

$$D(\alpha) = e^{\alpha\hat{a}^\dagger - \alpha^*\hat{a}}. \tag{2.258}$$

Using the Baker-Campbell-Hausdorff relation, it may be expressed as

$$D(\alpha) = e^{-|\alpha|^2/2} e^{\alpha\hat{a}^\dagger} e^{-\alpha^*\hat{a}}. \tag{2.259}$$

Substituting

$$\hat{a} = \frac{1}{\sqrt{2}}(\hat{q} + i\hat{p}), \tag{2.260}$$

and writing

$$\alpha = \frac{u + iv}{\sqrt{2}}, \tag{2.261}$$

gives

$$D(\alpha) = e^{i(v\hat{q} - u\hat{p})}. \tag{2.262}$$

Hence displacement operators correspond to translations in phase space.

In position representation, the momentum operator is

$$\hat{p} = -i\frac{d}{dq}. \tag{2.263}$$

Therefore,

$$e^{-i\hat{p}x} f(q) = e^{-x\frac{d}{dq}} f(q). \tag{2.264}$$

Expanding the exponential operator in Taylor series yields

$$e^{-x\frac{d}{dq}} f(q) = f(q) - xf'(q) + \frac{x^2}{2!} f''(q) - \cdots \tag{2.265}$$

$$= f(q - x), \tag{2.266}$$

which demonstrates that the displacement operator with a momentum-direction component acts as a translation operator in position space.

2.4.3 Coherent States

A coherent state is an infinite superposition of Fock states and resembles the most classical state of the electromagnetic field. It may be expanded in the Fock basis as

$$|\alpha\rangle = e^{-|\alpha|^2/2} \sum_{n=0}^{\infty} \frac{\alpha^n}{\sqrt{n!}} |n\rangle. \tag{2.267}$$

The probability amplitude of measuring n photons is

$$\langle n|\alpha\rangle = e^{-|\alpha|^2/2}\frac{\alpha^n}{\sqrt{n!}}. \tag{2.268}$$

Hence, the photon number probability distribution becomes

$$P(n) = |\langle n|\alpha\rangle|^2 = e^{-|\alpha|^2}\frac{|\alpha|^{2n}}{n!}. \tag{2.269}$$

Thus, coherent states exhibit a Poisson photon number distribution. The average photon number is

$$\langle \hat{n}\rangle = \sum_n nP(n) = |\alpha|^2. \tag{2.270}$$

2.4.4 Generation of Coherent States

Coherent states may be generated by applying the displacement operator to the vacuum state:

$$|\beta\rangle = D(\beta)|0\rangle. \tag{2.271}$$

Using the disentangled form of the displacement operator,

$$|\beta\rangle = e^{-|\beta|^2/2}e^{\beta\hat{a}^\dagger}e^{-\beta^*\hat{a}}|0\rangle. \tag{2.272}$$

Since

$$\hat{a}|0\rangle = 0, \tag{2.273}$$

we obtain

$$|\beta\rangle = e^{-|\beta|^2/2}e^{\beta\hat{a}^\dagger}|0\rangle. \tag{2.274}$$

Expanding the exponential:

$$|\beta\rangle = e^{-|\beta|^2/2}\sum_{n=0}^{\infty}\frac{(\beta\hat{a}^\dagger)^n}{n!}|0\rangle. \tag{2.275}$$

Using

$$(\hat{a}^\dagger)^n|0\rangle = \sqrt{n!}|n\rangle, \tag{2.276}$$

we recover

$$|\beta\rangle = e^{-|\beta|^2/2}\sum_{n=0}^{\infty}\frac{\beta^n}{\sqrt{n!}}|n\rangle. \tag{2.277}$$

Thus coherent states can be interpreted as displaced vacuum states and serve as fundamental ingredients in phase-space quantum information processing.

2.5 Construction of Approximate GKP Codewords

Having introduced displacement operators and coherent states, we now construct the logical codewords of the GKP encoding [6]. The key idea is to create a periodic lattice structure in phase space by superposing translated wavepackets.

2.5.1 Vacuum State in Position Representation

The vacuum state of a harmonic oscillator corresponds to a Gaussian wavepacket in position space. Denoting the vacuum by

$$|\psi_0\rangle \equiv |0\rangle, \tag{2.278}$$

its position-space wavefunction is

$$\psi_0(q) = \langle q|\psi_0\rangle = \frac{1}{(\pi\sigma^2)^{1/4}} \exp\left(-\frac{q^2}{2\sigma^2}\right), \tag{2.279}$$

where σ denotes the width of the Gaussian.

Using completeness of position eigenstates,

$$I = \int_{-\infty}^{\infty} |q\rangle\langle q| dq, \tag{2.280}$$

the vacuum state may be written as

$$|\psi_0\rangle = \int_{-\infty}^{\infty} dq\, |q\rangle\langle q|\psi_0\rangle. \tag{2.281}$$

Substituting the Gaussian wavefunction gives

$$|\psi_0\rangle = \int_{-\infty}^{\infty} \frac{dq}{(\pi\sigma^2)^{1/4}} e^{-q^2/(2\sigma^2)} |q\rangle. \tag{2.282}$$

Similarly, in momentum representation,

$$|\psi_0\rangle = \int_{-\infty}^{\infty} \frac{dp}{(\pi/\sigma^2)^{1/4}} e^{-\sigma^2 p^2/2} |p\rangle. \tag{2.283}$$

Thus the vacuum state is Gaussian in both quadratures.

2.5.2 Translation of Wavepackets

As discussed previously, the displacement operator acts as a translation operator in position space:

$$T(x) = e^{-ix\hat{p}}. \tag{2.284}$$

Its action on a wavefunction is

$$T(x)f(q) = f(q - x). \tag{2.285}$$

Therefore, repeated applications of translation operators generate shifted copies of the original Gaussian wavepacket. Superpositions of such translated states create periodic structures in phase space.

2.5.3 Approximate GKP Logical Codewords

The logical GKP basis states are constructed by placing Gaussian peaks periodically along the position quadrature.

The logical zero state is written as

$$|\tilde{0}\rangle = N_0 \sum_{s=-\infty}^{\infty} e^{-\frac{1}{2}\kappa^2(2s\alpha)^2} T(2s\alpha)|\psi_0\rangle, \tag{2.286}$$

while the logical one state is

$$|\tilde{1}\rangle = N_1 \sum_{s=-\infty}^{\infty} e^{-\frac{1}{2}\kappa^2[(2s+1)\alpha]^2} T\big((2s+1)\alpha\big)|\psi_0\rangle. \tag{2.287}$$

Here:

- α determines the lattice spacing,
- κ introduces a Gaussian envelope,
- N_0 and N_1 are normalization constants.

For symmetric approximate codewords, we may take

$$N_0 \approx N_1. \tag{2.288}$$

Unlike ideal GKP states consisting of infinitely sharp Dirac combs, these states possess finite energy and are therefore experimentally realizable.

2.5.4 Logical Plus State

The logical plus state is defined as

$$|\tilde{+}\rangle = \frac{|\tilde{0}\rangle + |\tilde{1}\rangle}{\sqrt{2}}. \tag{2.289}$$

Combining the two codewords yields

$$|\tilde{+}\rangle \propto \sum_{s=-\infty}^{\infty} e^{-\frac{1}{2}\kappa^2(s\alpha)^2} T(s\alpha)|\psi_0\rangle. \tag{2.290}$$

Thus the logical plus state contains peaks separated by α, forming a periodic grid.

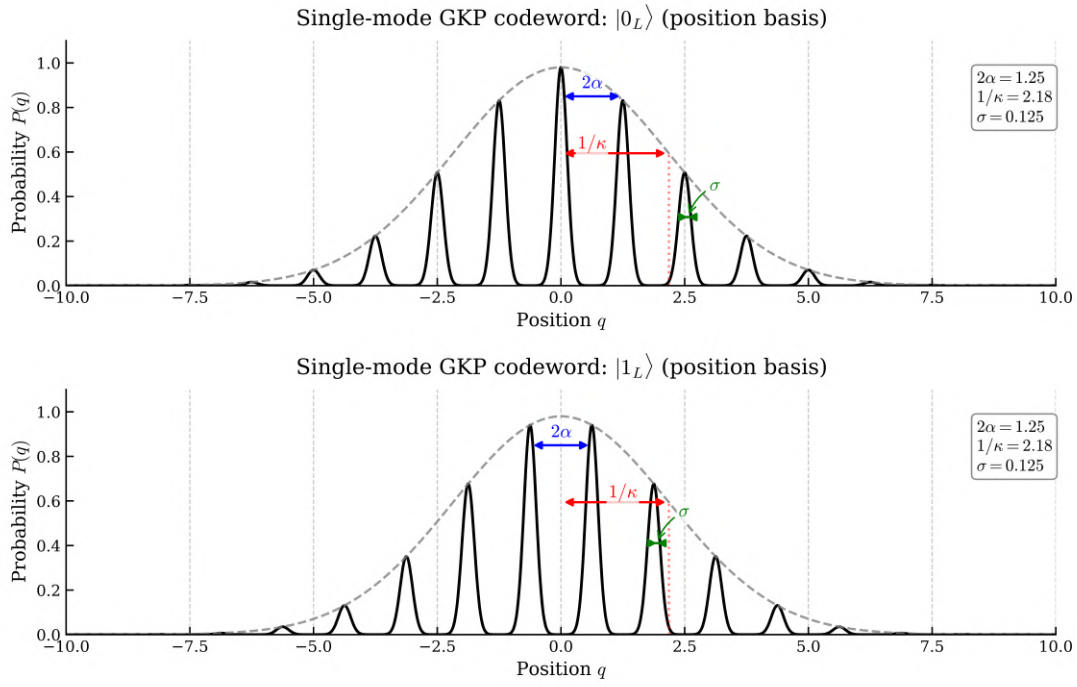


Figure 2.3: Approximate codeword probabilities, e.g., $|\langle q|\tilde{0}\rangle|^2$ and $|\langle q|\tilde{1}\rangle|^2$, plotted in position space.

2.5.5 Momentum-Space Interpretation

Applying Fourier transformation to the position-space grid produces another periodic structure in momentum space. The logical states therefore form lattices in both quadratures.

In the momentum representation, the peaks are separated by

$$\Delta p = \frac{2\pi}{\alpha}. \tag{2.291}$$

Hence GKP states simultaneously possess periodic structures in position and momentum spaces. This lattice structure forms the foundation of GKP stabilizers and enables protection against small displacement errors.

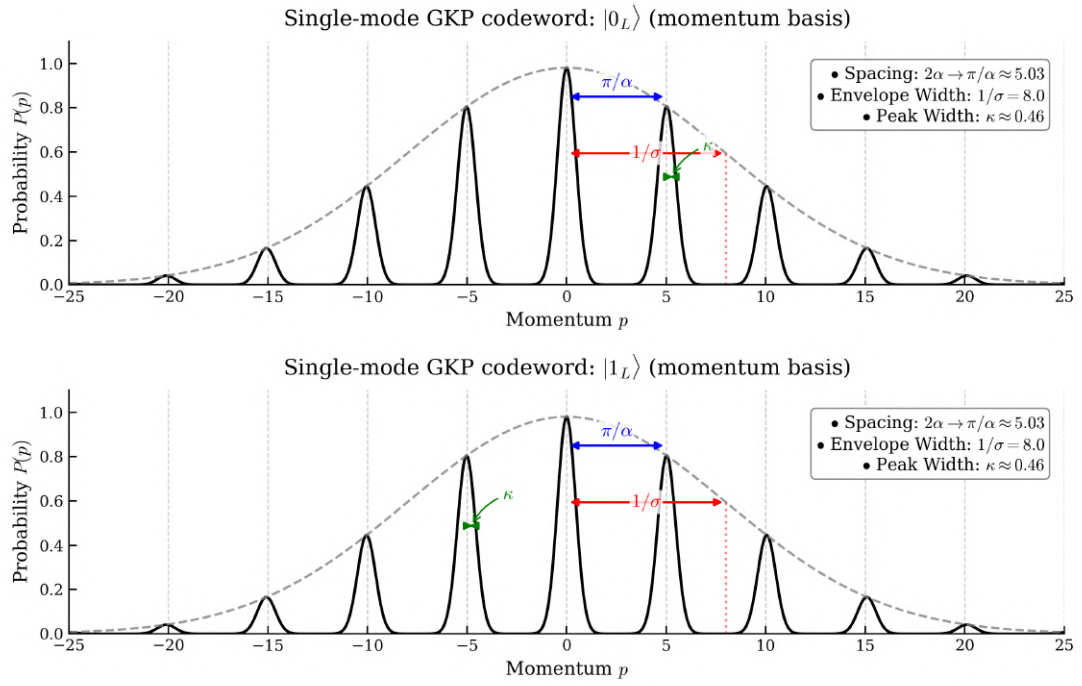


Figure 2.4: Approximate codeword probabilities, e.g., $|\langle p|\tilde{0}\rangle|^2$ and $|\langle p|\tilde{1}\rangle|^2$, plotted in momentum space.

2.6 Ideal GKP Codewords and their relation with Approximate GKP codewords

The approximate GKP states introduced in the previous section consist of periodically displaced Gaussian wavepackets weighted by a Gaussian envelope. These states possess finite energy and are experimentally realizable. The ideal GKP codewords arise as a limiting case in which both the individual peaks become infinitely sharp and the global envelope becomes infinitely broad [6].

2.6.1 From Approximate to Ideal GKP States

Recall the approximate logical codewords

$$|\tilde{0}\rangle = N_0 \sum_{s=-\infty}^{\infty} e^{-\frac{1}{2}\kappa^2(2s\alpha)^2} T(2s\alpha)|\psi_0\rangle, \tag{2.292}$$

$$|\tilde{1}\rangle = N_1 \sum_{s=-\infty}^{\infty} e^{-\frac{1}{2}\kappa^2[(2s+1)\alpha]^2} T\big((2s+1)\alpha\big)|\psi_0\rangle. \tag{2.293}$$

The translated vacuum state is a Gaussian of width σ:

$$\psi_0(q) = \frac{1}{(\pi\sigma^2)^{1/4}} e^{-q^2/(2\sigma^2)}. \tag{2.294}$$

To obtain ideal codewords, we take two limits:

1. $\sigma \to 0$: each Gaussian peak becomes infinitely narrow,

$$\frac{1}{(\pi\sigma^2)^{1/4}} e^{-q^2/(2\sigma^2)} \longrightarrow \delta(q). \tag{2.295}$$

2. $\kappa \to 0$: the broad Gaussian envelope becomes flat,

$$e^{-\frac{1}{2}\kappa^2(2s\alpha)^2} \to 1. \tag{2.296}$$

Applying these limits converts the finite-energy wavepackets into infinitely sharp periodic combs.

Hence the ideal logical zero state becomes

$$|0_L\rangle \propto \sum_{s=-\infty}^{\infty} |q = 2s\alpha\rangle, \tag{2.297}$$

and similarly

$$|1_L\rangle \propto \sum_{s=-\infty}^{\infty} |q = (2s+1)\alpha\rangle. \tag{2.298}$$

Thus the logical codewords consist of periodic Dirac-comb structures in position space.

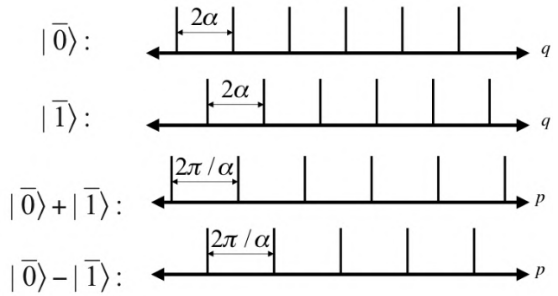


Figure 2.5: Ideal GKP codewords in the infinite-squeezing limit.

2.6.2 Approximate GKP States as Smeared Ideal Codewords

Ideal GKP states consist of infinitely sharp Dirac combs and therefore possess infinite energy. Consequently, they are non-normalizable and cannot be physically

realized. Practical implementations instead employ approximate GKP states obtained by introducing finite squeezing.

An approximate GKP state may be viewed as an ideal codeword subjected to a distribution of small displacement errors in phase space:

$$|\Psi\rangle = \iint du\, dv\ \eta(u,v) D(u,v)|\xi\rangle, \tag{2.299}$$

where $|\xi\rangle$ denotes an ideal GKP codeword and

$$D(u,v) = e^{i(v\hat{q} - u\hat{p})} \tag{2.300}$$

is the phase-space displacement operator.

The function $\eta(u,v)$ characterizes the finite squeezing and represents a sharply peaked distribution centered around the origin. Physically, it determines the amount of smearing introduced in phase space.

Since ideal GKP states are non-normalizable, the approximate state may be interpreted as a regularized version of the ideal codeword together with a small distribution of displacement errors.

For successful error correction, the displacement amplitudes should remain within the correctable region

$$|u| < \frac{\alpha}{2}, \tag{2.301}$$

$$|v| < \frac{\pi}{2\alpha}. \tag{2.302}$$

As long as the probability distribution $\eta(u,v)$ remains concentrated within this region, the state can be recovered successfully.

2.6.3 Choice of Smearing Function

A convenient choice for the error distribution is a Gaussian:

$$\eta(u,v) = A \exp\left[-\frac{1}{2}\left(\frac{u^2}{\Delta^2} + \frac{v^2}{\kappa^2}\right)\right], \tag{2.303}$$

where Δ and κ characterize the widths in position and momentum directions respectively.

Normalization requires

$$\iint du\, dv |\eta(u,v)|^2 = 1, \tag{2.304}$$

yielding

$$A = \frac{1}{\sqrt{\pi\kappa\Delta}}. \tag{2.305}$$

Hence

$$\eta(u,v) = \frac{1}{\sqrt{\pi\kappa\Delta}} \exp\left[-\frac{1}{2}\left(\frac{u^2}{\Delta^2} + \frac{v^2}{\kappa^2}\right)\right]. \tag{2.306}$$

Finite squeezing therefore replaces ideal lattice points by localized Gaussian regions in phase space.

2.6.4 Orthogonality of Ideal GKP Codewords

Consider two approximate codewords

$$|\Psi_1\rangle = \iint du\, dv\ \eta_1(u,v) D(u,v)|\xi_1\rangle, \tag{2.307}$$

$$|\Psi_2\rangle = \iint du' dv' \eta_2(u',v') D(u',v')|\xi_2\rangle. \tag{2.308}$$

Their overlap is

$$\langle\Psi_1|\Psi_2\rangle = \iint dudv \iint du' dv' \eta_1^*(u,v)\eta_2(u',v') \times \langle\xi_1|D^\dagger(u,v)D(u',v')|\xi_2\rangle. \tag{2.309}$$

Using Weyl-Heisenberg algebra,

$$D^\dagger(u,v)D(u',v') = D(\bar{u},\bar{v})e^{i\phi}, \tag{2.310}$$

where

$$\bar{u} = u' - u, \tag{2.311}$$

$$\bar{v} = v' - v, \tag{2.312}$$

and

$$\phi = \frac{1}{2}(uv' - u'v). \tag{2.313}$$

The phase factor contributes only a global phase and does not affect measurable quantities.

For ideal codewords one finds

$$\langle\xi|D(\bar{u},\bar{v})|\xi\rangle \approx \delta(\bar{u})\delta(\bar{v}), \tag{2.314}$$

provided the displacement lies inside the correctable region.

Similarly, for two distinct logical codewords,

$$\langle\xi_1|D(\bar{u},\bar{v})|\xi_2\rangle \approx \langle\xi_1|\xi_2\rangle\delta(\bar{u})\delta(\bar{v}). \tag{2.315}$$

Thus ideal logical states satisfy

$$\langle 0_L|1_L\rangle = 0, \tag{2.316}$$

and therefore form perfectly orthogonal codewords.

2.6.5 Non-Orthogonality of Approximate Codewords

Substituting the orthogonality relation into the overlap expression yields

$$\langle \Psi_1 | \Psi_2 \rangle = \langle \xi_1 | \xi_2 \rangle \iint du\, dv\; \eta_1^*(u,v)\eta_2(u,v). \tag{2.317}$$

Unlike ideal states, approximate GKP states possess finite widths in phase space. Neighboring Gaussian peaks overlap weakly, producing a small but non-zero overlap between logical states.

Hence

$$\langle \tilde{0} | \tilde{1} \rangle \neq 0, \tag{2.318}$$

although the overlap decreases exponentially as squeezing increases.

In the limit

$$\Delta \to 0, \qquad \kappa \to 0, \tag{2.319}$$

the smearing vanishes and the approximate states recover the ideal orthogonal codewords:

$$|\tilde{0}\rangle \to |0_L\rangle, \tag{2.320}$$

$$|\tilde{1}\rangle \to |1_L\rangle. \tag{2.321}$$

2.6.6 Choice of Spacing

The parameter α determines the spacing between neighboring peaks of the GKP lattice in position space. For the logical zero state,

$$|0_L\rangle \propto \sum_{s=-\infty}^{\infty} |q = 2s\alpha\rangle, \tag{2.322}$$

the separation between adjacent peaks is

$$d_q = 2\alpha. \tag{2.323}$$

Similarly, the logical plus state forms a periodic structure in momentum space. Since the Fourier transform of a Dirac comb is another Dirac comb, the momentum-space spacing becomes

$$d_p = \frac{2\pi}{\alpha}. \tag{2.324}$$

Thus the position and momentum spacings are reciprocal to each other:

$$d_q d_p = (2\alpha)\left(\frac{2\pi}{\alpha}\right) = 4\pi. \tag{2.325}$$

Consequently, increasing the spacing in one quadrature compresses the other in the conjugate quadrature.

For the standard GKP encoding one chooses

$$\alpha = \sqrt{\pi}. \tag{2.326}$$

This gives

$$d_q = 2\sqrt{\pi}, \tag{2.327}$$

and

$$d_p = \frac{2\pi}{\sqrt{\pi}} = 2\sqrt{\pi}. \tag{2.328}$$

Hence the spacing becomes identical in both quadratures:

$$d_q = d_p. \tag{2.329}$$

Therefore the phase-space becomes symmetric, producing a square geometry. This choice treats position and momentum errors equally and simplifies the construction of stabilizers and logical operators.

2.6.7 Alternative Geometries

Different choices of α produce different geometries in phase space.

Case 1: $\alpha = \sqrt{\pi}$

$$d_q = d_p = 2\sqrt{\pi} \tag{2.330}$$

This corresponds to the conventional square GKP lattice.

Case 2: $\alpha > \sqrt{\pi}$ For example, choosing

$$\alpha = 2\sqrt{\pi} \tag{2.331}$$

gives

$$d_q = 4\sqrt{\pi}, \tag{2.332}$$

$$d_p = \sqrt{\pi}. \tag{2.333}$$

Then it becomes elongated along the position quadrature and compressed in momentum space, producing a rectangular lattice.

Case 3: $\alpha < \sqrt{\pi}$ Choosing

$$\alpha = \frac{\sqrt{\pi}}{2} \tag{2.334}$$

yields

$$d_q = \sqrt{\pi}, \tag{2.335}$$

$$d_p = 4\sqrt{\pi}. \tag{2.336}$$

In this case it is compressed in position space and stretched in momentum space.

Hence the parameter α controls the aspect ratio in phase-space. Although the square-GKP is the standard choice in GKP encoding, alternative lattices can be advantageous when the physical noise model is asymmetric between the two quadratures.

2.7 Square GKP and their stabilizers and Logicals

Setting $\alpha = \sqrt{\pi}$ yields,

2.7.1 Position Space Representation

$$|0_L\rangle \propto \sum_{s=-\infty}^{\infty} |q = 2s\sqrt{\pi}\rangle, \tag{2.337}$$

$$|1_L\rangle \propto \sum_{s=-\infty}^{\infty} |q = (2s+1)\sqrt{\pi}\rangle. \tag{2.338}$$

Hence, $|0_L\rangle$ and $|1_L\rangle$ are separated by $\sqrt{\pi}$, while the periodicity of each logical state is $2\sqrt{\pi}$.

The corresponding wavefunctions are

$$\psi_{0_L}(q) \propto \sum_{s=-\infty}^{\infty} \delta(q - 2s\sqrt{\pi}), \tag{2.339}$$

$$\psi_{1_L}(q) \propto \sum_{s=-\infty}^{\infty} \delta(q - (2s+1)\sqrt{\pi}). \tag{2.340}$$

2.7.2 Momentum Space Representation

Applying Fourier transformation to a Dirac comb produces another Dirac comb. Consequently the GKP states possess periodic structures in momentum space as well:

$$|0_L\rangle \propto \sum_{s=-\infty}^{\infty} |p = s\sqrt{\pi}\rangle, \tag{2.341}$$

$$|1_L\rangle \propto \sum_{s=-\infty}^{\infty} (-1)^s |p = s\sqrt{\pi}\rangle. \tag{2.342}$$

2.7.3 Phase-Space Translations and Lattice Structure

The displacement operator introduced earlier generates translations in phase space. For a phase-space vector

$$\mathbf{u} = \begin{pmatrix} u_q \\ u_p \end{pmatrix}, \tag{2.343}$$

the associated displacement operator is

$$D(\mathbf{u}) = e^{i(u_p\hat{q} - u_q\hat{p})}. \tag{2.344}$$

Thus every displacement operator may be uniquely associated with a vector in phase space:

$$\mathbf{u} \quad \Longleftrightarrow \quad D(\mathbf{u}). \tag{2.345}$$

Consequently, a collection of displacement vectors can be represented geometrically as points on a lattice. The algebra of displacement operators therefore admits a direct geometric interpretation.

For the square GKP code, choose two primitive vectors

$$\mathbf{g}_1 = 2\sqrt{\pi}\begin{pmatrix}1\\0\end{pmatrix}, \qquad \mathbf{g}_2 = 2\sqrt{\pi}\begin{pmatrix}0\\1\end{pmatrix}. \tag{2.346}$$

Any lattice vector can then be generated through integer linear combinations:

$$\mathbf{v} = m\mathbf{g}_1 + n\mathbf{g}_2, \qquad m, n \in \mathbb{Z}. \tag{2.347}$$

Hence the entire lattice is specified by two generators.

The corresponding displacement operators become

$$D(\mathbf{g}_1) = e^{-i2\sqrt{\pi}\hat{p}}, \tag{2.348}$$

$$D(\mathbf{g}_2) = e^{i2\sqrt{\pi}\hat{q}}. \tag{2.349}$$

Thus there exists a one-to-one correspondence between lattice generator vectors and generator displacement operators.

2.7.4 Stabilizer Operators

The GKP codewords are invariant under translations generated by these primitive lattice vectors. Therefore the corresponding displacement operators define the stabilizers:

$$S_p = D(\mathbf{g}_1) = e^{-i2\sqrt{\pi}\hat{p}}, \tag{2.350}$$

$$S_q = D(\mathbf{g}_2) = e^{i2\sqrt{\pi}\hat{q}}. \tag{2.351}$$

Any stabilizer element can be obtained from integer powers of these generators:

$$S(m, n) = S_p^m S_q^n. \tag{2.352}$$

Equivalently,

$$S(m, n) = D(m\mathbf{g}_1 + n\mathbf{g}_2). \tag{2.353}$$

Hence the stabilizer group is generated directly by the underlying phase-space lattice.

2.7.5 Logical Operators

Logical operators correspond to displacement vectors that are not elements of the stabilizer lattice but preserve the overall code space.

The smallest such vectors are

$$\mathbf{l}_X = \frac{\mathbf{g}_1}{2}, \qquad \mathbf{l}_Z = \frac{\mathbf{g}_2}{2}. \tag{2.354}$$

The corresponding operators become

$$\bar{X} = D(\mathbf{l}_X) = e^{-i\sqrt{\pi}\hat{p}}, \tag{2.355}$$

$$\bar{Z} = D(\mathbf{l}_Z) = e^{i\sqrt{\pi}\hat{q}}. \tag{2.356}$$

Since these vectors are half of stabilizer generators, they do not belong to the stabilizer lattice. Instead they connect different logical sectors:

$$\bar{X}|0_L\rangle = |1_L\rangle. \tag{2.357}$$

Thus, the distinction between stabilizer and logical operators arises entirely from the geometry of their associated phase-space displacement vectors.

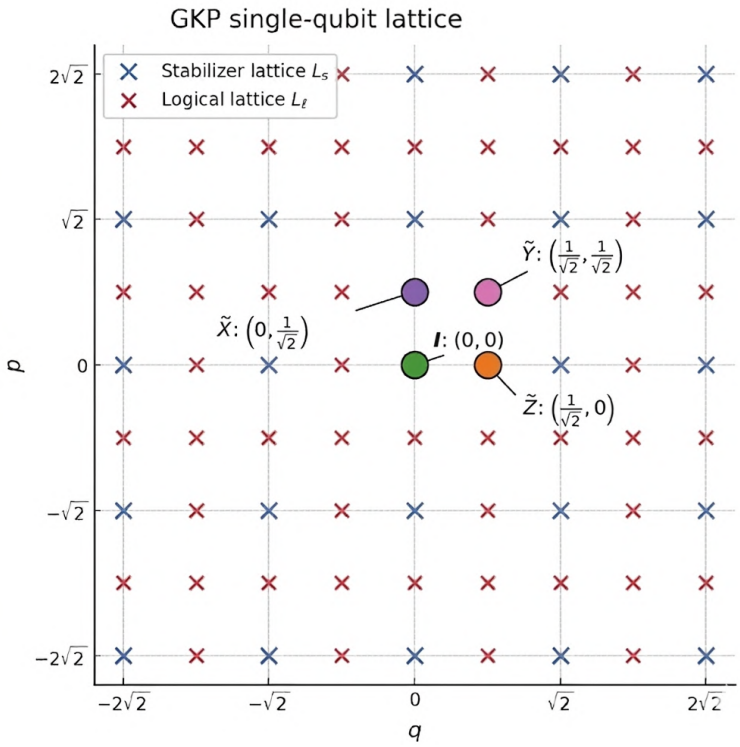


Figure 2.6: Phase space Normalizer Lattice

2.8 Multimode Bosonic Formalism

To extend GKP encoding to multiple oscillators, it is convenient to adopt a compact phase-space description using vector notation. This framework naturally describes multimode displacement operators, Gaussian operations and symplectic transformations [15].

2.8.1 Multimode Quadrature Operators

Consider a system of N bosonic modes with annihilation and creation operators satisfying

$$[\hat{a}_j, \hat{a}_k^\dagger] = \delta_{jk}. \tag{2.358}$$

For each mode, define the quadrature operators

$$\hat{q}_j = \frac{\hat{a}_j + \hat{a}_j^\dagger}{\sqrt{2}}, \tag{2.359}$$

$$\hat{p}_j = \frac{-i(\hat{a}_j - \hat{a}_j^\dagger)}{\sqrt{2}}. \tag{2.360}$$

Rather than treating quadratures individually, we collect them into a phase-space vector. Using the *qpqp* ordering convention,

$$\hat{\mathbf{x}} = (\hat{q}_1, \hat{p}_1, \hat{q}_2, \hat{p}_2, \cdots, \hat{q}_N, \hat{p}_N)^T. \tag{2.361}$$

The quadratures satisfy

$$[\hat{x}_j, \hat{x}_k] = i\Omega_{jk}, \tag{2.362}$$

where Ω is the symplectic form.

2.8.2 Symplectic Matrix

The symplectic matrix for a single mode is

$$\omega = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \tag{2.363}$$

and for N modes

$$\Omega = I_N \otimes \omega. \tag{2.364}$$

Explicitly,

$$\Omega = \begin{pmatrix} \omega & & \\ & \omega & \\ & & \ddots \end{pmatrix}. \tag{2.365}$$

The matrix possesses the properties

$$\Omega^T = -\Omega, \tag{2.366}$$

$$\Omega^{-1} = -\Omega. \tag{2.367}$$

The symplectic form encodes the canonical commutation relations of the multimode system.

2.8.3 Multimode Displacement Operators

Let

$$\mathbf{u} \in \mathbb{R}^{2N} \tag{2.368}$$

be a real phase-space displacement vector:

$$\mathbf{u} = (u_{q_1}, u_{p_1}, \ldots, u_{q_N}, u_{p_N})^T. \tag{2.369}$$

The multimode displacement operator is defined as

$$D(\mathbf{u}) = \exp\left[-i\mathbf{u}^T\Omega\hat{\mathbf{x}}\right]. \tag{2.370}$$

For a single mode this reduces to

$$D(u_q, u_p) = e^{i(u_p\hat{q} - u_q\hat{p})}, \tag{2.371}$$

which matches the displacement operator introduced previously.

The displacement operator shifts quadrature operators according to

$$D^\dagger(\mathbf{u})\hat{\mathbf{x}}D(\mathbf{u}) = \hat{\mathbf{x}} + \mathbf{u}. \tag{2.372}$$

Thus phase-space displacements act as translations in the quadrature coordinates.

2.8.4 Weyl Commutation Relation

The composition rule for multimode displacement operators follows from the Weyl-Heisenberg algebra:

$$D(\mathbf{u})D(\mathbf{v}) = e^{-i\mathbf{u}^T\Omega\mathbf{v}}D(\mathbf{v})D(\mathbf{u}). \tag{2.373}$$

Unlike ordinary translations, phase-space displacements generally do not commute. Their non-commutativity is determined by the symplectic product

$$\mathbf{u}^T\Omega\mathbf{v}. \tag{2.374}$$

This quantity plays a central role in the stabilizer description of multimode GKP codes.

2.8.5 Gaussian Unitary Operations

Gaussian unitary operations correspond to linear transformations of quadratures:

$$U_S^\dagger \hat{\mathbf{x}} U_S = S\hat{\mathbf{x}}, \tag{2.375}$$

where S is a real matrix.

To preserve canonical commutation relations, S must satisfy

$$S\Omega S^T = \Omega. \tag{2.376}$$

Matrices satisfying this relation are called symplectic matrices:

$$S \in Sp(2N, \mathbb{R}). \tag{2.377}$$

Examples include beam splitters, phase shifters and squeezing operations. Such transformations preserve the phase-space structure and play an important role in constructing multimode GKP lattices.

2.9 Multimode GKP Stabilizers and Lattice Structure

The multimode formulation naturally extends the geometric picture developed for the single-mode GKP code. Rather than describing stabilizers as isolated displacement operators, it is useful to associate them with vectors in a phase-space lattice [15].

2.9.1 Stabilizer Generator Vectors

Consider an N-mode bosonic system with phase-space coordinates in $\mathbb{R}^{2N}$. Let

$$\mathbf{v}_j \in \mathbb{R}^{2N}, \qquad j = 1, \ldots, 2N \tag{2.378}$$

denote a set of independent generator vectors.

To each vector we associate a displacement operator

$$\hat{S}_j = \hat{D}(\sqrt{2\pi}\mathbf{v}_j). \tag{2.379}$$

Using the multimode displacement operator definition,

$$\hat{D}(\mathbf{u}) = \exp\left[i\mathbf{u}^T\Omega^{-1}\hat{\mathbf{x}}\right], \tag{2.380}$$

the stabilizer generators become

$$\hat{S}_j = \exp\left[i\sqrt{2\pi}\mathbf{v}_j^T\Omega^{-1}\hat{\mathbf{x}}\right]. \tag{2.381}$$

Defining

$$\hat{g}_j = \sqrt{2\pi}\mathbf{v}_j^T\Omega^{-1}\hat{\mathbf{x}}, \tag{2.382}$$

one may write

$$\hat{S}_j = e^{i\hat{g}_j}. \tag{2.383}$$

Hence $\hat{g}_j$ acts as the Hermitian generator of the jth stabilizer.

2.9.2 Generator Matrix Representation

Collecting all generator vectors into a matrix,

$$M = \begin{pmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_{2N}^T \end{pmatrix}, \tag{2.384}$$

the stabilizer generators can be written compactly as

$$\hat{\mathbf{g}} = \sqrt{2\pi} M \Omega^{-1} \hat{\mathbf{x}}. \tag{2.385}$$

The matrix M therefore specifies the entire stabilizer structure of the multimode code.

2.9.3 Stabilizer Group and Phase-Space Lattice

All stabilizer elements are generated from integer powers of the independent generators:

$$\mathcal{S} = \left\{ \hat{S}_1^{a_1} \hat{S}_2^{a_2} \cdots \hat{S}_{2N}^{a_{2N}} \;\middle|\; \mathbf{a} \in \mathbb{Z}^{2N} \right\}. \tag{2.386}$$

Since all stabilizer generators commute, a generic stabilizer element becomes

$$\hat{S} = e^{i\mathbf{a}^T \hat{\mathbf{g}}}. \tag{2.387}$$

Substituting the generator expression yields

$$\hat{S} = \exp\left[i\sqrt{2\pi} (\mathbf{a}^T M) \Omega^{-1} \hat{\mathbf{x}} \right]. \tag{2.388}$$

Using the displacement operator definition,

$$\hat{S} = \hat{D}(\sqrt{2\pi} M^T \mathbf{a}). \tag{2.389}$$

Thus every stabilizer corresponds to a phase-space vector

$$\mathbf{u} = \sqrt{2\pi} M^T \mathbf{a}. \tag{2.390}$$

Consequently the set of all stabilizers can be identified with the lattice

$$\Lambda(M) = \{ \sqrt{2\pi} M^T \mathbf{a} \mid \mathbf{a} \in \mathbb{Z}^{2N} \}. \tag{2.391}$$

Hence there exists an isomorphism

$$\mathcal{S} \cong \Lambda(M), \tag{2.392}$$

establishing a one-to-one correspondence between stabilizer operators and phase-space lattice points.

2.9.4 Commutation Requirement and Symplectic Integral Lattices

Not every collection of phase-space vectors defines a valid multimode GKP code. Since stabilizers must form an Abelian group, all stabilizer generators are required to commute.

For multimode displacement operators, the Weyl relation is

$$\hat{D}(\mathbf{u})\hat{D}(\mathbf{v}) = e^{-i\mathbf{u}^T\Omega\mathbf{v}}\hat{D}(\mathbf{v})\hat{D}(\mathbf{u}). \tag{2.393}$$

Thus two displacement operators commute only when

$$\mathbf{u}^T\Omega\mathbf{v} = 2\pi k, \qquad k \in \mathbb{Z}. \tag{2.394}$$

Now consider two stabilizer lattice vectors

$$\mathbf{u} = \sqrt{2\pi}M^T\mathbf{a}, \tag{2.395}$$

$$\mathbf{v} = \sqrt{2\pi}M^T\mathbf{b}, \tag{2.396}$$

where

$$\mathbf{a}, \mathbf{b} \in \mathbb{Z}^{2N}. \tag{2.397}$$

Substituting into the commutation condition gives

$$\mathbf{u}^T\Omega\mathbf{v} = 2\pi\mathbf{a}^T M\Omega M^T\mathbf{b}. \tag{2.398}$$

Define

$$A = M\Omega M^T. \tag{2.399}$$

Then

$$\mathbf{u}^T\Omega\mathbf{v} = 2\pi\mathbf{a}^T A\mathbf{b}. \tag{2.400}$$

For arbitrary integer vectors $\mathbf{a}$ and $\mathbf{b}$, the commutation requirement becomes

$$\mathbf{a}^T A\mathbf{b} \in \mathbb{Z}. \tag{2.401}$$

Therefore all entries of A must be integers:

$$A_{ij} \in \mathbb{Z}. \tag{2.402}$$

Since Ω is antisymmetric,

$$A^T = -A, \tag{2.403}$$

so A is an integer antisymmetric matrix.

Hence a valid multimode GKP code requires

$$M\Omega M^T = A, \qquad A_{ij} \in \mathbb{Z}. \tag{2.404}$$

Lattices satisfying this condition are known as *symplectic integral lattices*.

Thus the requirement that stabilizers commute translates directly into a geometric constraint on the underlying phase-space lattice.

2.9.5 Canonical Form of the Commutation Matrix

The generator matrix is not unique. Different generator matrices may describe the same lattice.

Let

$$M' = RM, \tag{2.405}$$

where R is an integer unimodular matrix satisfying

$$\det(R) = \pm 1. \tag{2.406}$$

Under this transformation,

$$A' = M'\Omega M'^T \tag{2.407}$$

becomes

$$A' = RAR^T. \tag{2.408}$$

A theorem for integer antisymmetric matrices guarantees the existence of a unimodular transformation such that

$$RAR^T = \mathrm{diag}(d_1, d_2, \ldots, d_N) \otimes \omega, \tag{2.409}$$

where

$$\omega = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}. \tag{2.410}$$

The integers

$$d_1, d_2, \ldots, d_N \tag{2.411}$$

are known as the elementary divisors of the lattice. And physically they represents dimension encoded in each mode.

Hence every multimode GKP stabilizer lattice can be brought into a canonical symplectic form.

2.9.6 Normalizer Lattice and Logical Operators

Logical operators correspond to phase-space translations that preserve the stabilizer code space. Equivalently, they are displacement operators that commute with every stabilizer element.

Let

$$\hat{L} = \hat{D}(\sqrt{2\pi}\mathbf{u}) \tag{2.412}$$

be a candidate logical operator, and let

$$\hat{S} = \hat{D}(\sqrt{2\pi}\mathbf{v}) \tag{2.413}$$

be a stabilizer element with

$$\mathbf{v} \in \Lambda(M). \tag{2.414}$$

Using the Weyl commutation relation,

$$\hat{D}(\mathbf{u})\hat{D}(\mathbf{v}) = e^{-i\mathbf{u}^T\Omega\mathbf{v}}\hat{D}(\mathbf{v})\hat{D}(\mathbf{u}), \tag{2.415}$$

commutation requires

$$\mathbf{u}^T\Omega\mathbf{v} \in \mathbb{Z} \qquad \forall\mathbf{v} \in \Lambda(M). \tag{2.416}$$

Therefore the set of all vectors commuting with every stabilizer forms the dual lattice

$$\Lambda^\perp = \{\mathbf{u} \mid \mathbf{u}^T\Omega\mathbf{v} \in \mathbb{Z},\ \forall\mathbf{v} \in \Lambda\}. \tag{2.417}$$

Hence all logical operators arise from vectors belonging to the dual lattice.

2.9.7 Generator Matrix of the Dual Lattice

Let

$$\mathbf{v} = M^T\mathbf{a}, \qquad \mathbf{a} \in \mathbb{Z}^{2N}. \tag{2.418}$$

The dual-lattice condition requires

$$\mathbf{u}^T\Omega M^T\mathbf{a} \in \mathbb{Z} \tag{2.419}$$

for all integer vectors $\mathbf{a}$.

This implies

$$\mathbf{u}^T\Omega M^T = \mathbf{b}^T, \tag{2.420}$$

where

$$\mathbf{b} \in \mathbb{Z}^{2N}. \tag{2.421}$$

Solving yields a generator matrix for the dual lattice:

$$M^\perp = \Omega A^{-1} M, \tag{2.422}$$

where

$$A = M\Omega M^T. \tag{2.423}$$

Hence vectors in the normalizer lattice may be expressed as

$$\mathbf{u} = (M^\perp)^T\mathbf{b}. \tag{2.424}$$

Thus the dual lattice determines all possible logical translations.

2.9.8 Logical Operators and Cosets

Not all vectors in the dual lattice correspond to distinct logical operators.

Suppose two logical operators differ by a stabilizer:

$$\hat{L}' = \hat{L}\hat{S}(\mathbf{v}), \tag{2.425}$$

with

$$\mathbf{v} \in \Lambda. \tag{2.426}$$

For any code state,

$$\hat{L}'|\psi\rangle = \hat{L}\hat{S}(\mathbf{v})|\psi\rangle \tag{2.427}$$

$$= \hat{L}|\psi\rangle, \tag{2.428}$$

since stabilizers act trivially on the code space.

Hence two logical operators differing by a stabilizer are physically indistinguishable.

Consequently vectors satisfy the equivalence relation

$$\mathbf{u} \sim \mathbf{u} + \mathbf{v}, \qquad \mathbf{v} \in \Lambda. \tag{2.429}$$

Distinct logical operators are therefore classified by cosets:

$$\Lambda^{\perp}/\Lambda. \tag{2.430}$$

Thus logical operators correspond to equivalence classes of dual-lattice vectors modulo stabilizer vectors.

2.9.9 Encoded Dimension

The number of distinct logical operators equals the number of cosets:

$$|\Lambda^{\perp}/\Lambda|. \tag{2.431}$$

Using lattice volume relations,

$$|\Lambda^{\perp}/\Lambda| = \left(\frac{\det(M)}{\det(M^{\perp})}\right)^2. \tag{2.432}$$

Since

$$\det\big(M^{\perp}\big) = \frac{1}{\det(M)}, \tag{2.433}$$

one obtains

$$|\Lambda^{\perp}/\Lambda| = [\det(M)]^2. \tag{2.434}$$

If the code encodes a d-dimensional logical system, then there are d^2 logical operators:

$$d^2 = [\det(M)]^2. \tag{2.435}$$

Hence the encoded dimension is

$$d = \det(M). \tag{2.436}$$

Therefore the determinant of the generator matrix directly determines the logical dimension encoded by the multimode GKP lattice.

2.9.10 Code Distance in Multimode GKP Codes

For GKP codes, stabilizers and logical operators are represented by phase-space displacement vectors. As discussed previously, stabilizer vectors form a lattice Λ, while logical operators belong to the dual lattice $\Lambda^{\perp}$.

Since logical operators differing by stabilizers act identically on the code space, nontrivial logical operators correspond to vectors in

$$\Lambda^{\perp}/\Lambda. \tag{2.437}$$

The distance of a GKP code is defined as the smallest nontrivial displacement capable of implementing a logical operation. Geometrically, it corresponds to the minimum distance between distinct cosets.

Therefore the code distance is

$$d = \min_{\mathbf{u}\in\Lambda^{\perp},\mathbf{u}\notin\Lambda} \sqrt{2\pi}\|\mathbf{u}\|, \tag{2.438}$$

where

$$\|\mathbf{u}\| = \sqrt{u_1^2 + u_2^2 + \cdots + u_{2N}^2} \tag{2.439}$$

denotes the Euclidean norm of the displacement vector.

Thus the distance corresponds to the shortest logical vector that does not belong to the stabilizer lattice [15].

This definition closely parallels the notion of distance in qubit stabilizer codes, where the distance corresponds to the minimum weight of a nontrivial logical operator. However, in GKP codes the role of Hamming weight is replaced by Euclidean length in phase space.

Logical Operator Distances

Suppose the lattice has been transformed into canonical form

$$A = \mathrm{diag}(d_1, d_2, \ldots, d_N) \otimes \omega. \tag{2.440}$$

Consider the special case

$$d_1 = 2, \qquad d_2 = d_3 = \cdots = d_N = 1. \tag{2.441}$$

This corresponds to a code encoding one logical qubit.

The quotient space

$$\Lambda^{\perp}/\Lambda \tag{2.442}$$

is generated by two vectors

$$\mathbf{w}_1, \qquad \mathbf{w}_2, \tag{2.443}$$

which correspond respectively to logical $\bar{X}$ and $\bar{Z}$.

Since vectors differing by stabilizers are equivalent,

$$\bar{X} \sim \{\mathbf{w}_1 + \mathbf{u} \mid \mathbf{u} \in \Lambda\}, \tag{2.444}$$

$$\bar{Z} \sim \{\mathbf{w}_2 + \mathbf{u} \mid \mathbf{u} \in \Lambda\}, \tag{2.445}$$

and similarly

$$\bar{Y} \sim \{\mathbf{w}_1 + \mathbf{w}_2 + \mathbf{u} \mid \mathbf{u} \in \Lambda\}. \tag{2.446}$$

Hence the minimum lengths associated with the logical operators become

$$d_X = \min_{\mathbf{b} \in \mathbb{Z}^{2N}} \sqrt{2\pi} \|\mathbf{w}_1 - M^T \mathbf{b}\|, \tag{2.447}$$

$$d_Z = \min_{\mathbf{b} \in \mathbb{Z}^{2N}} \sqrt{2\pi} \|\mathbf{w}_2 - M^T \mathbf{b}\|, \tag{2.448}$$

$$d_Y = \min_{\mathbf{b} \in \mathbb{Z}^{2N}} \sqrt{2\pi} \|\mathbf{w}_1 + \mathbf{w}_2 - M^T \mathbf{b}\|. \tag{2.449}$$

Therefore the code distance becomes

$$d = \min(d_X, d_Y, d_Z). \tag{2.450}$$

2.10 Gaussian Transformations Between GKP Codes

Different GKP codes may be related through Gaussian unitary operations. Since Gaussian unitaries correspond to symplectic transformations in phase space, they provide a natural way to map one stabilizer lattice into another. [15]

Consider a stabilizer element

$$\hat{S} = \hat{D}(\sqrt{2\pi} M^T \mathbf{a}), \tag{2.451}$$

where

$$\mathbf{a} \in \mathbb{Z}^{2N}. \tag{2.452}$$

Suppose a Gaussian unitary $\hat{U}$ acts on the system. The transformed stabilizer becomes

$$\hat{S}' = \hat{U} \hat{S} \hat{U}^\dagger. \tag{2.453}$$

Using the displacement representation,

$$\hat{S}' = \hat{U} \exp\left[i\sqrt{2\pi} \mathbf{a}^T M \Omega^{-1} \hat{\mathbf{x}} \right] \hat{U}^\dagger. \tag{2.454}$$

For Gaussian unitaries, quadrature operators transform linearly:

$$\hat{U}^\dagger \hat{\mathbf{x}} \hat{U} = S\hat{\mathbf{x}} + \mathbf{u}, \tag{2.455}$$

where S is a symplectic matrix and $\mathbf{u}$ is a phase-space displacement vector.

Ignoring the displacement term, which contributes only an overall shift, the transformed quadratures satisfy

$$\hat{U}\hat{\mathbf{x}}\hat{U}^{\dagger} = S^{-1}\hat{\mathbf{x}}. \tag{2.456}$$

Using the symplectic relation

$$S\Omega S^{T} = \Omega, \tag{2.457}$$

one obtains

$$S^{-1} = \Omega S^{T}\Omega^{-1}. \tag{2.458}$$

Substituting into the transformed stabilizer gives

$$\hat{S}' = \exp\left[i\sqrt{2\pi}\mathbf{a}^{T}MS^{T}\Omega^{-1}\hat{\mathbf{x}}\right]. \tag{2.459}$$

Recognizing the displacement form,

$$\hat{S}' = \hat{D}(\sqrt{2\pi}(SM^{T})\mathbf{a}). \tag{2.460}$$

Hence the transformed generator matrix becomes

$$M' = MS^{T}. \tag{2.461}$$

Therefore applying a Gaussian unitary transforms the underlying stabilizer lattice through a symplectic map.

This establishes an important correspondence:

$$\text{Gaussian unitary} \Longleftrightarrow \text{Symplectic transformation} \Longleftrightarrow \text{Lattice transformation}$$

Consequently, different multimode GKP codes related by symplectic transformations belong to the same equivalence class.

2.10.1 Decomposition into Square Lattice GKP Codes

Previously we showed that every commutation matrix may be brought into canonical form

$$A = \mathrm{diag}(d_1, d_2, \ldots, d_N) \otimes \omega. \tag{2.462}$$

Correspondingly, define a square-lattice generator matrix

$$M_{\mathrm{sq}}(d) = \mathrm{diag}(\sqrt{d_1}, \sqrt{d_1}, \ldots, \sqrt{d_N}, \sqrt{d_N}). \tag{2.463}$$

Then one may always decompose the generator matrix as

$$M = M_{\mathrm{sq}}(d)S^{T}. \tag{2.464}$$

Thus any general GKP lattice can be expressed as a symplectic transformation acting on a tensor product of independent square GKP lattices.

Therefore an arbitrary multimode GKP code may be interpreted as

$$\text{General GKP code} \Longleftrightarrow \text{Gaussian unitary} + \text{Square GKP code}. \tag{2.465}$$

This result provides a constructive method for realizing general GKP codes experimentally: prepare independent square lattice GKP states and apply the corresponding Gaussian unitary transformation.

2.11 Decoding of Multimode GKP Codes

The stabilizer formalism developed previously specifies the structure of multimode GKP codes and their logical operators. To protect encoded information against physical noise, one must additionally determine how errors are identified and corrected. For GKP codes this problem becomes geometric: given noisy phase-space displacements, one must infer the most likely lattice point corresponding to the true error. [15]

2.11.1 Physical Noise Model

Consider an N-mode bosonic system with quadrature vector

$$\hat{x} = (\hat{q}_1, \hat{p}_1, \ldots, \hat{q}_N, \hat{p}_N)^T. \tag{2.466}$$

Assume an additive Gaussian shift-noise model:

$$\hat{x} \to \hat{x}' = \hat{x} + \xi, \tag{2.467}$$

where

$$\xi = (\xi_q^{(1)}, \xi_p^{(1)}, \ldots, \xi_q^{(N)}, \xi_p^{(N)})^T \tag{2.468}$$

is a random displacement vector whose components satisfy

$$\xi_j \sim \mathcal{N}(0, \sigma^2). \tag{2.469}$$

Thus each quadrature undergoes a small random displacement drawn independently from a Gaussian distribution.

2.11.2 Displacement Representation of Errors

Phase-space displacements are represented by multimode displacement operators

$$\hat{D}(\xi) = \exp\left(i\xi^T \Omega^{-1} \hat{x}\right). \tag{2.470}$$

Hence a physical error corresponds to applying a displacement operator to the encoded GKP state.

The goal of decoding is to determine a recovery displacement

$$\hat{D}(-\xi^*) \tag{2.471}$$

that best compensates the unknown error.

2.11.3 Syndrome Extraction

Recall that stabilizer generators take the form

$$\hat{S}_j = \exp\left[i\sqrt{2\pi}\, v_j^T \Omega^{-1} \hat{x}\right]. \tag{2.472}$$

Writing

$$\theta_j = \sqrt{2\pi} v_j^T \Omega^{-1} \hat{x}, \tag{2.473}$$

a displacement error shifts the stabilizer phase according to

$$\theta_j \to \theta_j + \sqrt{2\pi} v_j^T \Omega^{-1} \xi. \tag{2.474}$$

Hence syndrome measurements produce

$$s_j = \sqrt{2\pi} v_j^T \Omega^{-1} \xi \pmod{2\pi}. \tag{2.475}$$

Collecting all components gives

$$s = \sqrt{2\pi} M \Omega^{-1} \xi \pmod{2\pi}. \tag{2.476}$$

2.11.4 Syndrome Cosets and Lattice Ambiguity

The syndrome equation implies

$$\sqrt{2\pi} M \Omega^{-1} \xi = s + 2\pi a, \tag{2.477}$$

where

$$a \in \mathbb{Z}^{2N}. \tag{2.478}$$

Solving for the error gives

$$\xi = \frac{1}{\sqrt{2\pi}} \Omega M^{-1} (s + 2\pi a). \tag{2.479}$$

Using

$$M^{\perp} = \Omega A^{-1} M, \tag{2.480}$$

one obtains

$$\xi = \eta(s) - \sqrt{2\pi} (M^{\perp})^T b, \tag{2.481}$$

where

$$\eta(s) = -\frac{1}{\sqrt{2\pi}} (\Omega M^{\perp})^T s \tag{2.482}$$

depends continuously on syndrome information and

$$b \in \mathbb{Z}^{2N} \tag{2.483}$$

labels integer lattice shifts.

For a fixed measured syndrome, the physical error does not correspond to a unique point. Rather,

$$\xi \in \eta(s) + \Lambda^{\perp}, \tag{2.484}$$

where

$$\Lambda^{\perp} = \left\{ \sqrt{2\pi}(M^{\perp})^{T} b : b \in \mathbb{Z}^{2N} \right\}. \tag{2.485}$$

Therefore syndrome measurements determine an entire coset of possible errors rather than a unique displacement. Decoding thus reduces to identifying the most likely member of this coset.

2.11.5 Maximum-Likelihood Decoding and Closest Vector Problem

Under the additive Gaussian noise model,

$$\xi \sim \mathcal{N}(0, \sigma^{2} I), \tag{2.486}$$

the probability density of observing an error vector is

$$P(\xi) = \frac{1}{(2\pi\sigma^{2})^{N}} \exp\left(-\frac{\|\xi\|^{2}}{2\sigma^{2}} \right). \tag{2.487}$$

For a fixed syndrome s, the set of compatible errors is

$$\xi \in \eta(s) + \Lambda^{\perp}. \tag{2.488}$$

The decoder must choose the most likely element from this coset.

Maximum-likelihood decoding therefore requires

$$b^{*} = \arg\max_{b} P(\xi_{b}). \tag{2.489}$$

Substituting the Gaussian distribution gives

$$b^{*} = \arg\max_{b} \exp\left[-\frac{\|\eta(s) - \sqrt{2\pi}(M^{\perp})^{T} b\|^{2}}{2\sigma^{2}} \right]. \tag{2.490}$$

Since the exponential is monotonic, this becomes

$$b^{*} = \arg\min_{b} \left\| \eta(s) - \sqrt{2\pi}(M^{\perp})^{T} b \right\|. \tag{2.491}$$

Thus decoding reduces to finding the nearest lattice point to $\eta(s)$.
This is precisely the Closest Vector Problem (CVP):

$$\text{Find} \quad \lambda^* \in \Lambda^\perp \tag{2.492}$$

such that

$$\|\eta(s) - \lambda^*\| \tag{2.493}$$

is minimized.

Hence maximum-likelihood decoding of GKP codes becomes a geometric nearest-neighbor search in phase space.

2.11.6 Recovery Operation and Decoding Outcome

After solving the closest vector problem, the estimated displacement is

$$\xi^* = \eta(s) - \sqrt{2\pi}(M^\perp)^T b^*. \tag{2.494}$$

The recovery operation applies the inverse displacement

$$D(-\xi^*). \tag{2.495}$$

Acting on the erroneous state gives

$$D(-\xi^*)D(\xi) = D(\xi - \xi^*)e^{i\alpha}, \tag{2.496}$$

where α is an irrelevant global phase.
Substituting the true and estimated errors:

$$\xi = \eta(s) - \sqrt{2\pi}(M^\perp)^T b_{\text{true}}, \tag{2.497}$$

gives

$$\xi - \xi^* = \sqrt{2\pi}(M^\perp)^T (b^* - b_{\text{true}}). \tag{2.498}$$

Define

$$c = b^* - b_{\text{true}}. \tag{2.499}$$

Then

$$\xi - \xi^* = \sqrt{2\pi}(M^\perp)^T c. \tag{2.500}$$

The recovery outcome depends on the residual displacement.
If

$$(M^{\perp})^T c \in \Lambda(M), \tag{2.501}$$

the residual operator belongs to the stabilizer lattice and acts trivially on the code space. Therefore correction succeeds.

Conversely, if

$$(M^{\perp})^T c \notin \Lambda(M), \tag{2.502}$$

the residual displacement belongs to a different coset of the normalizer lattice and implements a nontrivial logical operation.

Hence decoding failure corresponds not to an incorrect estimate itself, but to choosing a different logical coset.

2.11.7 Minimum-Energy Interpretation

For Gaussian noise,

$$P(\xi) \propto e^{-\|\xi\|^2/(2\sigma^2)}. \tag{2.503}$$

Therefore maximizing probability is equivalent to minimizing

$$\|\xi\|^2. \tag{2.504}$$

Thus closest-point decoding may also be interpreted as a minimum-energy decoding strategy:

$$\text{MED} \Longleftrightarrow \text{choose smallest-energy displacement.} \tag{2.505}$$

Hence GKP decoding under Gaussian noise reduces naturally to an energy minimization problem over lattice points.

2.11.8 Decoding Complexity and the Closest Vector Problem

The previous discussion showed that maximum-likelihood decoding of multimode GKP codes reduces to a Closest Vector Problem (CVP) over the dual lattice. Although this formulation is geometrically intuitive, solving CVP exactly becomes computationally challenging as the number of modes [15].

Closest Vector Problem

Given a lattice

$$\Lambda = \{M^T a \mid a \in \mathbb{Z}^{2N}\}, \tag{2.506}$$

and a target vector

$$y \in \mathbb{R}^{2N}, \tag{2.507}$$

the closest vector problem asks for a lattice point

$$\lambda^* \in \Lambda \tag{2.508}$$

such that

$$\| y - \lambda^* \| \tag{2.509}$$

is minimized.

For GKP decoding, the target vector corresponds to the syndrome-dependent quantity

$$y = \eta(s), \tag{2.510}$$

while lattice points correspond to candidate error vectors.

Thus decoding requires solving

$$\lambda^* = \arg\min_{\lambda \in \Lambda^\perp} \|\eta(s) - \lambda\|. \tag{2.511}$$

2.11.9 Computational Complexity of CVP

Unlike decoding for simple one-dimensional square GKP codes, multimode lattices generally possess highly correlated geometries. Consequently the number of candidate lattice points grows rapidly with dimension.

For an N-mode system, decoding occurs in a $2N$-dimensional phase space. If one naively searches over all lattice points within a radius R, the number of candidates scales approximately as

$$N_{\text{cand}} \sim e^{cN}, \tag{2.512}$$

for some constant c.

Therefore brute-force maximum-likelihood decoding exhibits exponential complexity.

More fundamentally, the closest vector problem is known to be NP-hard under randomized reductions. Consequently, an efficient polynomial-time exact decoder is not expected to exist for arbitrary lattices.

Thus although maximum-likelihood decoding is optimal, its computational cost rapidly becomes prohibitive as the number of modes increases.

2.11.10 Structured Lattices for Efficient Decoding

The complexity of decoding depends strongly on the geometry of the underlying lattice.

Certain highly structured lattices possess symmetries that enable more efficient decoding procedures.

Examples include:

- Checkerboard (D_n) lattices,
- Tesseract lattices,

These lattices often admit specialized decoding algorithms with significantly lower complexity than generic CVP solvers.

Consequently lattice design itself becomes an important aspect of GKP code construction.

2.12 Examples of Multimode GKP Lattice Constructions

To better understand how lattice geometry influences code properties and decoding performance, we now discuss several important examples of multimode GKP lattices [15] [18].

These examples illustrate how modifying the generator matrix changes encoded dimension, logical operators and code distance.

2.12.1 Hypercubic ($\mathbb{Z}_{2N}$) Lattice

Consider the simplest possible generator matrix

$$M_{\mathbb{Z}_{2N}} = I_{2N}. \tag{2.513}$$

The commutation matrix becomes

$$\begin{aligned} A &= M\Omega M^T && (2.514)\\ &= \Omega && (2.515)\\ &= I_N \otimes \omega. && (2.516) \end{aligned}$$

Thus all elementary divisors satisfy

$$d_1 = d_2 = \cdots = d_N = 1. \tag{2.517}$$

Using the determinant relation derived previously,

$$d_{\text{enc}} = |\det(M)|, \tag{2.518}$$

one obtains

$$d_{\text{enc}} = 1. \tag{2.519}$$

Therefore no logical information is encoded.

The corresponding state is simply a tensor product of stabilizer states with no logical degrees of freedom.

Hence the hypercubic lattice provides a useful reference geometry but does not itself realize a nontrivial GKP code.

2.12.2 Rectangular GKP Lattice

The standard square GKP code treats position and momentum symmetrically. However, realistic physical systems often experience asymmetric noise in the two quadratures.

To accommodate such asymmetry one may deform the square lattice into a rectangular lattice.

For a single-mode code define

$$M_{\text{rect}} = \begin{pmatrix} \sqrt{2}\eta & 0 \\ 0 & \sqrt{2}/\eta \end{pmatrix}, \tag{2.520}$$

where

$$\eta > 0 \tag{2.521}$$

controls the aspect ratio of the lattice.

The determinant becomes

$$\det(M_{\text{rect}}) = 2, \tag{2.522}$$

so the code still encodes one logical qubit.

Thus changing η modifies only the geometry while preserving encoded dimension.

2.12.3 Relation to Square GKP via Squeezing

Recall that any GKP lattice may be generated from a square lattice through a symplectic transformation.

For the square GKP qubit,

$$M_{\text{sq}} = \sqrt{2} I_2. \tag{2.523}$$

One immediately observes

$$M_{\text{rect}} = M_{\text{sq}} S^T, \tag{2.524}$$

with

$$S^T = \begin{pmatrix} \eta & 0 \\ 0 & \eta^{-1} \end{pmatrix}. \tag{2.525}$$

Since

$$S\Omega S^T = \Omega, \tag{2.526}$$

the transformation is symplectic.

Physically this corresponds to a squeezing operation acting on the square GKP state.

Hence rectangular GKP lattices may be interpreted simply as squeezed versions of square GKP codes.

2.12.4 Logical Distances of Rectangular GKP Codes

Using the dual lattice construction, the logical generators become

$$w_1 = \begin{pmatrix} \eta/\sqrt{2} \\ 0 \end{pmatrix}, \qquad w_2 = \begin{pmatrix} 0 \\ 1/(\sqrt{2}\eta) \end{pmatrix}. \tag{2.527}$$

The corresponding logical operators are

$$\bar{X} = \exp\left(-i\sqrt{\pi}\eta\hat{p}\right), \tag{2.528}$$

$$\bar{Z} = \exp\left(i\frac{\sqrt{\pi}}{\eta}\hat{q}\right). \tag{2.529}$$

Hence the logical distances become

$$d_X = \eta\sqrt{\pi}, \tag{2.530}$$

$$d_Z = \eta^{-1}\sqrt{\pi}, \tag{2.531}$$

and

$$d_Y = \sqrt{\pi(\eta^2 + \eta^{-2})}. \tag{2.532}$$

For

$$\eta = 1, \tag{2.533}$$

one recovers the square GKP code:

$$d_X = d_Z. \tag{2.534}$$

Thus the parameter η redistributes protection between position and momentum quadratures.

Rectangular GKP codes therefore become advantageous when noise strengths differ between the two quadratures.

2.12.5 Dense Lattice Packing and Decoding Performance

The previous examples illustrate that different generator matrices produce different phase-space geometries. Beyond encoded dimension, lattice geometry strongly affects error-correction performance.

Under Gaussian displacement noise, decoding reduces to a closest-vector problem. Consequently, the probability of successful recovery depends on the size of the Voronoi cell surrounding each lattice point.

Lattices with larger packing radii possess larger correctable regions and therefore exhibit improved decoding performance.

From a geometric perspective, the problem becomes one of sphere packing: among all lattices with fixed density, which arrangement maximizes the minimum distance between neighboring lattice points?

Dense sphere packings therefore provide natural candidates for improved GKP constructions.

Chamberland *et al.* emphasized that highly symmetric dense lattices can significantly improve bosonic code performance by enlarging correctable displacement regions.

2.12.6 Checkerboard Lattice D_n

An important family of dense lattices is the checkerboard lattice D_n, defined by

$$D_n = \left\{ x \in \mathbb{Z}^n \;\middle|\; \sum_{i=1}^{n} x_i \equiv 0 \pmod 2 \right\}. \tag{2.535}$$

Thus only integer vectors with even coordinate parity are retained.

Compared with the cubic lattice $\mathbb{Z}^n$, the checkerboard construction removes half of the lattice points while increasing packing efficiency.

A generator matrix for D_n may be written as

$$M_{D_n} = \begin{pmatrix} 1 & -1 & 0 & \cdots & 0 \\ 0 & 1 & -1 & \cdots & 0 \\ \vdots & & & \ddots & \\ 0 & 0 & 0 & 1 & -1 \\ 1 & 1 & 1 & \cdots & 1 \end{pmatrix}. \tag{2.536}$$

The determinant satisfies

$$\det(M_{D_n}) = 2, \tag{2.537}$$

indicating a denser packing structure relative to the hypercubic lattice.

2.12.7 Tesseract Lattice GKP Code

The tesseract lattice is a four-dimensional analogue of a cube and provides another example of a multimode GKP construction.

A generator matrix for the tesseract lattice may be written as :

$$M_{\text{tess}} = 2^{1/4} \begin{pmatrix} 1/\sqrt{2} & 0 & 1/\sqrt{2} & 0 \\ 0 & 1 & 0 & 0 \\ 1/\sqrt{2} & 0 & -1/\sqrt{2} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{2.538}$$

Interestingly, this lattice naturally decomposes into two sublattices:

- a D_2 checkerboard structure in the position quadratures,
- a $\mathbb{Z}_2$ structure in the momentum quadratures.

Thus the tesseract lattice combines distinct lattice geometries across different quadratures.

2.12.8 Concatenation Interpretation

A particularly important observation is that the tesseract code admits a simple concatenated interpretation.

Consider two rectangular GKP codes with squeezing parameter

$$\eta = 2^{1/4}, \tag{2.539}$$

concatenated with a two-qubit repetition code having stabilizer

$$XX. \tag{2.540}$$

Following Construction-A, one obtains a generator matrix

$$M'_{\text{tess}} = RM_{\text{tess}}, \tag{2.541}$$

where R is a unimodular matrix satisfying

$$\det(R) = \pm 1. \tag{2.542}$$

Since unimodular transformations preserve lattice volume, M'_{tess} and M_{tess} represent different bases of the same lattice.

Therefore the tesseract code can be interpreted as a rectangular GKP code concatenated with a repetition code.

2.12.9 Logical Distances of the Tesseract Code

Because the repetition-code stabilizer XX does not detect logical $\bar{X}$ errors, the logical X-distance remains unchanged from the rectangular code:

$$d_X^{\text{tess}} = 2^{1/4}\sqrt{\pi}. \tag{2.543}$$

The remaining logical operators inherit additional protection from the repetition structure, leading to

$$d_Z^{\text{tess}} = 2^{1/4}\sqrt{\pi}, \tag{2.544}$$

and

$$d_Y^{\text{tess}} = 2^{1/4}\sqrt{2\pi}. \tag{2.545}$$

Thus

$$d_{\text{tess}} = 2^{1/4}\sqrt{\pi}. \tag{2.546}$$

Compared with the square GKP code, the tesseract lattice provides an enlarged logical distance through geometric and code concatenation effects.

2.12.10 Generalized Repetition-Rectangular Codes

The tesseract code may be generalized to higher dimensions.

Since the tesseract construction corresponds to a two-qubit repetition code, one may instead concatenate an N-qubit repetition code with rectangular GKP states.

Choosing

$$\eta = N^{1/4}, \tag{2.547}$$

produces a family of $2N$-dimensional lattices, denoted by

$$\text{rep-rec}_N. \tag{2.548}$$

The corresponding logical distances become

$$d_X^{\text{rep}-\text{rec}} = N^{1/4}\sqrt{\pi}, \tag{2.549}$$

$$d_Z^{\text{rep}-\text{rec}} = N^{1/4}\sqrt{\pi}, \tag{2.550}$$

$$d_Y^{\text{rep}-\text{rec}} = N^{1/4}\sqrt{2\pi}. \tag{2.551}$$

Therefore

$$d_{\text{rep}-\text{rec}} = N^{1/4}\sqrt{\pi}. \tag{2.552}$$

For sufficiently large N, these generalized codes achieve larger distances than the D_{2N} family.

2.13 Lattice Structure as the Computational Backbone of GKP Decoding

Although exact closest-vector decoding is NP-hard for a general lattice, practical GKP codes rarely employ arbitrary lattice geometries. Instead, physically relevant lattices are carefully designed so that their dual generator matrices possess special algebraic structures [15] [18].

Recall that decoding is performed over the dual lattice

$$\Lambda^{\perp}(M) = \left\{ (M^{\perp})^{T} b : b \in \mathbb{Z}^{2N} \right\}. \tag{2.553}$$

Hence the complexity of decoding is determined directly by the structure of $M^{\perp}$.

The central idea is that if the dual generator matrix admits special decompositions, then a difficult global closest-vector problem can be transformed into several smaller tractable subproblems.

Thus complexity reduction does not arise from eliminating CVP itself; rather it arises by transforming one difficult high-dimensional search into many simpler decoding tasks.

2.13.1 Direct Sum Structure

Suppose the dual generator matrix decomposes as

$$M^{\perp} = \bigoplus_{i=1}^{k} M_i^{\perp}. \tag{2.554}$$

Then the dual lattice factorizes as

$$\Lambda^{\perp}(M) = \bigoplus_{i=1}^{k} \Lambda_i. \tag{2.555}$$

Consequently the closest-point problem separates into independent subproblems:

$$\chi_t(\Lambda^{\perp}) = \bigoplus_{i=1}^{k} \chi_{\pi_i(t)}(\Lambda_i), \tag{2.556}$$

where π_i denotes projection onto the corresponding subspace.

Instead of solving one large $2N$-dimensional problem, the decoder solves several smaller CVPs independently and combines the results.

If each component decoder has complexity

$$C_i, \tag{2.557}$$

the total complexity becomes

$$C_{\text{total}} \approx \sum_i C_i, \tag{2.558}$$

rather than exponential scaling with the total dimension.

Hence direct-sum structure converts a difficult global problem into parallel local decoding tasks.

2.13.2 Union of Cosets

Consider a union of shifted cosets

$$\Sigma = \bigcup_{i=0}^{l-1} (r_i + \Lambda). \tag{2.559}$$

The closest-point operation satisfies [15]

$$\chi_t(\Sigma) = \chi_t\Big(\{r_i + \chi_{t-r_i}(\Lambda)\}\Big). \tag{2.560}$$

Therefore decoding proceeds through the following steps:

1. shift the target vector by each coset vector,
2. decode each shifted lattice independently,
3. compare candidate distances,
4. choose the nearest solution.

Thus complexity scales approximately as

$$C_\Sigma \sim l\, C_\Lambda, \tag{2.561}$$

where l denotes the number of cosets.

Therefore when the number of cosets remains small, decoding introduces only polynomial overhead.

2.13.3 Glue Lattices

Many concatenated GKP constructions naturally produce glue lattices [15] of the form

$$\Lambda = \bigcup_{i=0}^{l-1} \left(r_i + \bigoplus_{j=1}^{m} \Lambda_j \right). \tag{2.562}$$

Glue lattices combine direct-sum decomposition and coset-union structure simultaneously.

Therefore decoding becomes hierarchical:

1. decode individual component lattices,
2. combine component outputs,
3. evaluate a small set of gluing vectors,
4. select the nearest candidate.

Thus concatenated constructions inherit efficient decoding algorithms from their constituent lattices.

2.13.4 From Exponential Search to Practical Decoders

This structure becomes particularly important for concatenated GKP codes such as surface-GKP codes.

A naive decoder may require searching over exponentially many cosets:

$$N_{\text{cosets}} = 2^{N+1}, \tag{2.563}$$

which rapidly becomes intractable.

However, exploiting decomposition structure allows one to combine lattice decoding with additional algorithms such as minimum-weight perfect matching.

As a result, the global exponential search can be replaced by polynomial-time decoding procedures.

Furthermore, certain structured lattices such as $\mathbb{Z}_{2N}$ repetition-rectangular and D_n-based constructions admit near-linear decoding algorithms [15].

Hence lattice design serves a dual purpose:

$$\text{improved code distance} + \text{reduced decoding complexity}. \tag{2.564}$$

Therefore lattice engineering is not merely a geometric optimization problem; it determines whether large-scale GKP decoding becomes computationally feasible.

Chapter 3

Higher genus Toric code

A torus of genus one can be constructed by identifying opposite edges of a square, which results in a closed surface with a single handle. To obtain a torus of genus two, an analogous construction can be carried out using an octagon with appropriate edge identifications.[1]

3.1 Stabilizers

Here, we have used the same convention as the toric code. The data qubits are placed on the edges. Plaquettes are the Z-stabilizers, and the vertices are the X-stabilizers. Note that each vertex and plaquette has an even number of common edges. So all the stabilizers commute.

- **Plaquette operators:** It is defined on the four edges on the boundary of the plaquette p except for the octagon in the center, which has 8 edges in its boundary.

$$B_p = \prod_{i \in \sigma(p)} Z_i,$$

  where $\sigma(p)$ is the boundary of the plaquette p.

- **Star operators:** It is defined on the four edges connected to the vertex v with the exception of the outer corner vertex. As explained earlier, all the outer 8 vertices are identified as a single vertex. For this vertex, we can see that it is connected to 8 data qubits.

$$A_v = \prod_{i \in \delta(v)} X_i,$$

  where $\delta(v)$ is the co-boundary of the vertex v

[1]The Multi-Genus Toric code architecture employed in this thesis was developed primarily by Mr. Mrinmoy Karmakar. I gratefully acknowledge his contributions and the numerous discussions that shaped the direction of this work.

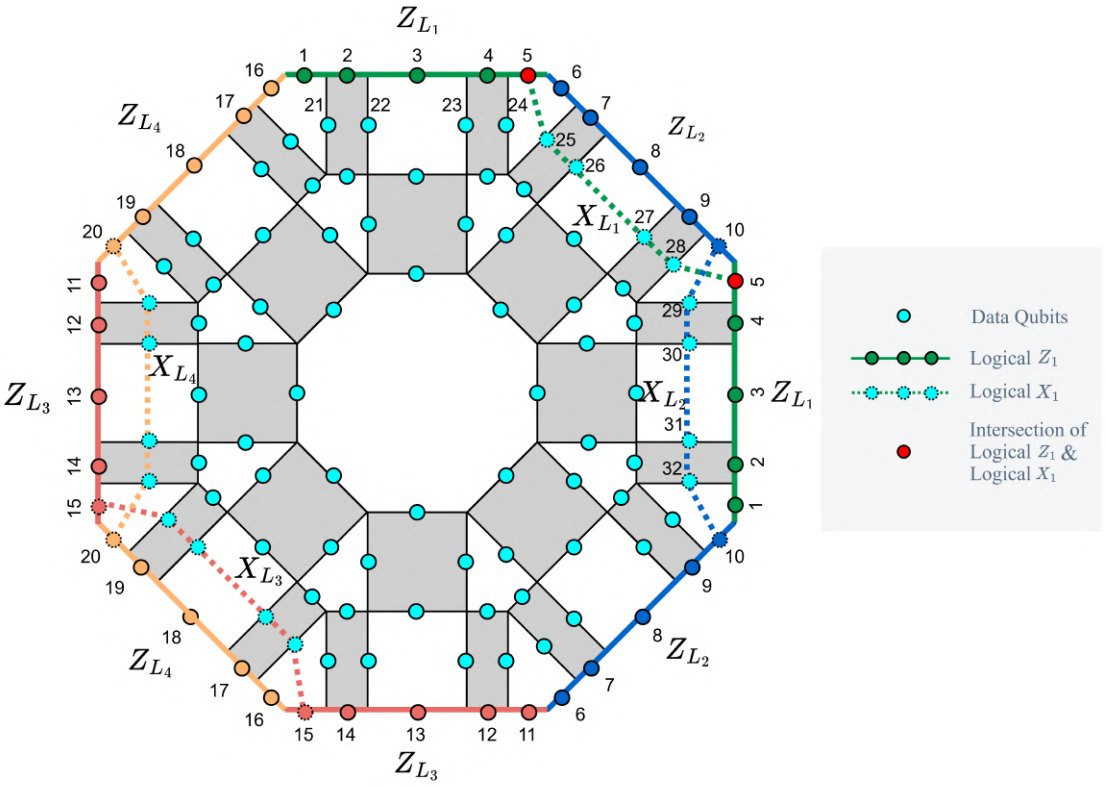


Figure 3.1: Logical operators of the genus two toric code. Here in the diagram, the solid (dotted) green, blue, red, and yellow lines represent the logical Z operators (logical X operators). For example, pauli Z operated on the qubits 1,2,3,4 and 5 is the logical Z_{L_1} operator, and pauli X operated on qubit 5, 24, 25, 26, 27, and 28 is the logical X_{L_1}. They intersect at qubit 5, hence the anti-commutation.

3.2 Logical Operator

This code can encode four logical qubits. In Figure 3.1, the green, blue, yellow, and red solid lines indicate those non-trivial loops. Here, the solid colored lines are the logical Z operators and the dotted lines are the logical X operators.

As we can encode 4 logical qubits, we have 4 pairs of logical operators. Similar to the toric code, the Z-logical operators overlap with X-stabilizers on an even number of qubits; hence, they commute with all the X-stabilizers. Similarly, the X-logical operators intersect the plaquettes at an even number of qubits; so, they also commute with all the Z-stabilizers. Note that the logical Z_{L_1} and the logical X_{L_1} intersects at one qubit; so, they anti-commute. The same is true for the other pairs of logical operators. The code parameter is given by $[[4d^2, 4, d]]$, where d is an odd number.

3.3 Generalized Toric code

To construct a torus with multiple handles, we consider a polygon with $n = 4g$ edges, where g denotes the number of handles of the surface. By imposing an appropriate set of edge identifications, all vertices of the n-gon are identified as a single vertex in the resulting surface. This construction produces a torus of genus g, which we denote by T_g. The first homology group of this surface is given by

$$H_1(T_g) \cong Z_2^{2g}.$$

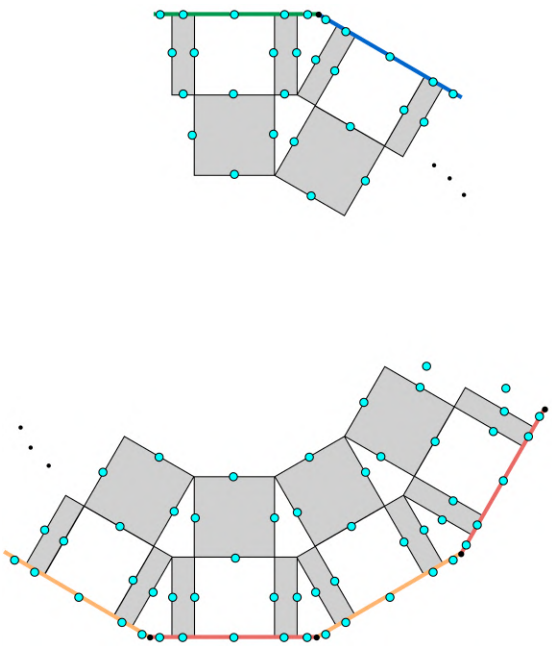


Figure 3.2: Toric code with genus g. Logical operators are marked with the colored solid lines.

Using a similar structure, we can get the generalized version of the Toric code with code parameter $[[2gd^2, 2g, d]]_2$, where d is odd.

3.4 Construction of generalized Rotated Toric code

By exploiting lattice symmetries and reducing the redundancy in the stabilizer placement, the rotated constructions significantly improve the encoding rate. Using similar boundary conditions, we arrive at the following structure for the generalized toric code.

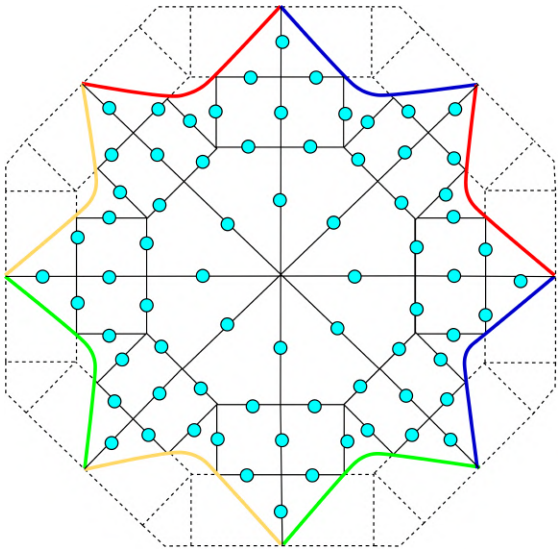


Figure 3.3: Rotated toric code for genus 2 torus. The modified boundary identifications are shown in the figure

The code parameters are $[[gd^2, 2g, d]]_2$ where d is an even number. The code rate is doubled compared to the un-rotated version. The same convention has been used as before. The data qubits are placed on the edges. Plaquettes are the Z-stabilizers, and the vertices are the X-stabilizers. Even after reducing the number of qubits the internal structure remains the same, i.e, each vertex and plaquette has an even number of common edges. So all the stabilizers commute. All the $2g$ pairs of logical operators anti-commute pairwise, but commute with other logical operators.

3.5 Summary

In this chapter, we reviewed the construction of higher-genus toric codes and introduced their rotated counterparts. By exploiting lattice symmetries and modified boundary identifications, the generalized rotated toric code achieves an improved encoding rate while preserving the local stabilizer structure of conventional toric codes. The resulting code family encodes $2g$ logical qubits on a genus-g surface and serves as the topological foundation for the Multi-Genus Rotated Toric-GKP codes considered in this thesis.

In the following chapters, these topological codes will be concatenated with the square GKP code and their performance under Gaussian displacement noise will be investigated using the closest-point decoding framework. Particular emphasis will be placed on threshold estimation, logical-fidelity analysis, and comparisons with the Rotated Surface-GKP code.

Chapter 4

Concatenation with GKP Codes

Building upon the discrete topological error-correcting frameworks established in the previous chapters, this chapter transitions into the physical implementation of these codes within continuous-variable (CV) systems. Specifically, we explore the concatenation of multi-qubit stabilizer codes—such as surface and multi-genus toric codes—with Gottesman-Kitaev-Preskill (GKP) codes. By framing multimode GKP codes through a lattice perspective, we can seamlessly map discrete binary parity-check matrices into continuous phase-space lattices. This chapter details the construction methodologies for these concatenated lattices, the unique topological challenges introduced by higher-genus homologies, and the implementation of efficient Minimum Weight Perfect Matching (MWPM) decoders tailored for CV noise.

4.1 Surface-GKP Construction

To physically encode logical quantum information into a continuous-variable system, a standard approach is to concatenate a qubit-based stabilizer code with single-mode GKP codes. Consider a generic $[[N, k]]$ stabilizer code concatenated with N square GKP codes to encode k logical qubits [15].

From a lattice perspective, the unconcatenated base state is a separable lattice generated by N independent copies of the square GKP code[cite: 692]. In the canonical phase-space basis, the generator matrix for this base lattice is given by:

$$M^{(\mathrm{sq})} = \sqrt{2} I_{2N} \tag{4.1}$$

where I_{2N} is the $2N \times 2N$ identity matrix[15]. The prefactor of $\sqrt{2}$ arises because each base GKP code encodes a single qubit (two states) per mode [18].

To embed the parity-check constraints of the $[[N, k]]$ stabilizer code into this CV phase space, we utilize a procedure known in classical coding theory as Construction-A [18]. Let $\{g_j \mid j = 1, \ldots, N - k\}$ denote the set of binary vectors corresponding to the $N - k$ generators of the stabilizer group [15]. The concatenated GKP lattice

is formed by replacing exactly $N-k$ rows of the base matrix $M^{(\mathrm{sq})}$ with the scaled binary vectors of these stabilizers [15]:

$$\left\{ \frac{1}{\sqrt{2}} g_j^T \quad \text{for} \quad j = 1, \ldots, N-k \right\} \tag{4.2}$$

A critical requirement of this construction is that the resulting concatenated generator matrix, denoted $M_{\mathrm{conc}}^{(\mathrm{sq})}$, must remain full-rank. For each stabilizer generator g_j, the algorithm identifies the index l of its first non-zero element ($g_{jl} \neq 0$). If the l-th row of $M_{\mathrm{conc}}^{(\mathrm{sq})}$ has not yet been replaced by a previous stabilizer, it is overwritten by $g_j^T/\sqrt{2}$. If the row has already been claimed, the algorithm proceeds to the next non-zero element in g_j until a valid replacement is made.

The pseudocode for this standard construction is detailed in Algorithm 1.

Algorithm 1 Standard Concatenated GKP Lattice Construction [15]

Require: The set of binary vectors $\{g_j \mid j = 1, \ldots, N-k\}$ representing the stabilizer generators.
Ensure: The full-rank concatenated generator matrix $M_{\mathrm{conc}}^{(\mathrm{sq})}$.

```
M_conc^(sq) ← √2 I_2N                          ▷ Initialize the base square lattice
for j = 1 to N − k do
    for l = 1 to 2N do
        if g_jl ≠ 0 then
            if the l-th row of M_conc^(sq) has not been replaced then
                M_conc^(sq)[l, :] ← (1/√2) g_j
                break                           ▷ Move to the next stabilizer generator
            end if
        end if
    end for
end for
```

This greedy row-replacement strategy works exceptionally well for planar surface codes with tree-like generator supports. However, as we will demonstrate in the following section, relying on this first-available-index scan introduces catastrophic failures when applied to the complex homological cycles inherent in multi-genus toric codes.

4.1.1 General Standard Form Construction

Because the aforementioned algorithm will not necessarily give a full-rank matrix for such complex topologies, a more robust, general approach must be employed. This approach works directly with the standard form of the stabilizer code [15].

For this construction, we view the set of binary vectors $\{\boldsymbol{g}_j^T\}$ as an $(N-k)\times(2N)$ matrix G with components g_{jl}. Using standard stabilizer formalism techniques [2,

1], via Gaussian elimination and relabelling of the physical qubits if needed, one can bring the parity-check matrix G into the standard form:

$$\begin{bmatrix} I & A_1 & A_2 & B & 0 & C \\ 0 & 0 & 0 & D & I & E \end{bmatrix} \tag{4.3}$$

Here, r is the rank of the left $(N-k) \times N$ submatrix of G, I represents the identity matrix of the appropriate dimension, and A_1, A_2, B, C, D, E are all integer-valued matrices. With the stabilizer generators properly arranged in this standard form, the generator matrix for the concatenated GKP code, ordered in the *qqpp* convention, can be explicitly constructed as:

$$M_{\text{conc}}^{(sq)} = \frac{1}{\sqrt{2}} \begin{bmatrix} I & A_1 & A_2 & B & 0 & C \\ 0 & 0 & 0 & D & I & E \\ 0 & 0 & 2I & 0 & 0 & 0 \\ 0 & 0 & 0 & 2I & 0 & 0 \\ 0 & 2I & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2I \end{bmatrix} \tag{4.4}$$

To rigorously verify that $M_{\text{conc}}^{(sq)}$ generates a valid GKP lattice, we first note that since the set $\{\boldsymbol{g}_j\}$ corresponds to stabilizer generators that mutually commute, they must satisfy the symplectic inner product condition:

$$\text{mod}(\boldsymbol{g}_j^T \Omega^{\text{qqpp}} \boldsymbol{g}_l, 2) = 0 \quad \text{for all } j, l = 1, \ldots, N-k \tag{4.5}$$

where Ω^{qqpp} is the symplectic form in the position-momentum basis. Because of this commutativity, the Gram matrix given by $M_{\text{conc}}^{(sq)} \Omega^{\text{qqpp}} (M_{\text{conc}}^{(sq)})^T$ is strictly integer-valued.

Furthermore, by performing elementary column operations—specifically swapping the two columns in Equation 4.4—it can be shown that the matrix possesses a determinant of 2^k [15]. This geometric volume confirms our fundamental requirement: we conclude that $M_{\text{conc}}^{(sq)}$ forms a valid GKP lattice that encodes exactly k logical qubits.

It is highly important to note a subtlety regarding these column operations. Swapping the columns of $M_{\text{conc}}^{(sq)}$ is mathematically equivalent to multiplying by a non-symplectic orthogonal matrix from the right. In general, this transformation leads to a non-symplectic integral matrix M'. Despite the fact that M' shares the exact same determinant (2^k) as $M_{\text{conc}}^{(sq)}$, it physically cannot be regarded as a valid GKP code. This is consistent with the discrete stabilizer formalism, as swapping the corresponding columns of the binary parity-check matrix G generally breaks the required commutation relations of the physical stabilizers [2].

4.1.2 Topological Scrambling and the RREF Approach

While the General Standard Form Construction guarantees the generation of a valid, full-rank continuous GKP lattice, its direct application to spatially structured

topological codes presents a severe practical hurdle. As established in the previous section, transforming a generic parity-check matrix G into the standard stabilizer form requires Gaussian elimination over $GF(2)$, which frequently necessitates column permutations to isolate the required identity blocks.

In a purely abstract stabilizer code, qubit relabeling is a trivial algebraic operation. However, for topological codes—and specifically for the generalized multi-genus rotated toric code developed in this work—the physical qubit labels strictly encode the geometric and topological structure of the lattice. The indices dictate nearest-neighbor connectivity, boundary conditions, and the complex non-trivial homological cycles characteristic of higher-genus surfaces.

Applying arbitrary $GF(2)$ column swaps to achieve the standard form irreparably scrambles this spatial mapping. If the physical geometry of the qubits is lost, the underlying topological correlations of the noise are scattered randomly across the parity-check matrix. Consequently, topological decoding algorithms that rely on localized metric spaces, such as Minimum Weight Perfect Matching (MWPM), become fundamentally incompatible with the resulting continuous lattice.

To circumvent the destruction of the spatial layout while still overcoming the rank-deficiency of the greedy algorithm (Algorithm 1), we implement a modified construction utilizing the Reduced Row Echelon Form (RREF). By strictly restricting our $GF(2)$ operations to elementary *row* operations (row swapping, row addition, and scalar multiplication), we avoid column permutations entirely.

Let G be the initial binary parity-check matrix of the multi-genus rotated toric code in its natural spatial ordering. We compute the RREF of G over $GF(2)$, denoted as G_{RREF}. Because row operations merely replace stabilizer generators with valid linearly independent combinations of themselves, the generated stabilizer group remains identical. More importantly, the columns—and thus the physical qubit locations on the multi-genus manifold—remain entirely undisturbed.

By extracting the linearly independent rows from G_{RREF}, we obtain a full-rank parity-check matrix that flawlessly preserves the spatial lattice geometry. We can then systematically map these preserved rows into the phase-space generator matrix $M_{\text{conc}}^{(sq)}$ without inducing the topological scrambling inherent to the standard form construction. This guarantees that the subsequent closest lattice point decoding and MWPM algorithms can operate on the continuous variables while fully exploiting the protected topological pathways of the higher-genus surface.

To practicalize this row-restricted RREF method, we can exploit the inherent structure of Calderbank-Shor-Steane (CSS) codes. For a CSS code, the stabilizer parity-check matrix naturally decouples into independent X-type and Z-type blocks, H_X and H_Z. Consequently, the continuous-variable mapping can be perfectly partitioned between the q (position) and p (momentum) quadratures.

By applying the row-restricted RREF algorithm to H_X and H_Z independently, we identify a strict set of linearly independent pivot columns for each quadrature. The procedure for generating the topologically-safe, full-rank CSS-GKP lattice matrix is detailed in Algorithm 2.

Algorithm 2 Topologically-Safe CSS-GKP Lattice Construction via RREF

Require: The binary parity-check matrices H_X of size $m_X \times N$ and H_Z of size $m_Z \times N$.

Ensure: The full-rank, geometrically preserved concatenated generator matrix $M_{\text{conc}}^{(\text{sq})}$.

Compute the RREF of H_X using **only row operations** to obtain $H_{X,\text{RREF}}$ and a list of pivot indices P_X.

Compute the RREF of H_Z using **only row operations** to obtain $H_{Z,\text{RREF}}$ and a list of pivot indices P_Z.

$M_{\text{conc}}^{(\text{sq})} \leftarrow \sqrt{2} I_{2N}$ ▷ Initialize the base separable lattice

for $i = 1$ **to** m_X **do**

    $c_X \leftarrow P_X[i]$ ▷ Get the column index of the i-th X-pivot

    $M_{\text{conc}}^{(\text{sq})}[c_X, 1 \dots N] \leftarrow \frac{1}{\sqrt{2}} H_{X,\text{RREF}}[i, :]$ ▷ Embed into q-quadrature

end for

for $i = 1$ **to** m_Z **do**

    $c_Z \leftarrow P_Z[i]$ ▷ Get the column index of the i-th Z-pivot

    $M_{\text{conc}}^{(\text{sq})}[N + c_Z, (N+1) \dots 2N] \leftarrow \frac{1}{\sqrt{2}} H_{Z,\text{RREF}}[i, :]$ ▷ Embed into p-quadrature

end for

A paramount advantage of this partitioned RREF construction is the strict preservation of both the symplectic geometry and the CSS transversality. Because the algorithm strictly avoids column swaps, it entirely bypasses the implicit non-symplectic orthogonal transformations that spoil the commutation relations of the physical stabilizers. Consequently, the symplectic form Ω^{qqpp} remains completely uncorrupted.

Furthermore, by explicitly mapping the X-checks entirely within the first N phase-space coordinates and the Z-checks within the latter N coordinates, the algorithm guarantees that X and Z errors cannot leak into each other's lattice phase spaces. This strict decoupling ensures that the dual syndrome graphs required for MWPM remain perfectly independent, allowing the decoder to accurately process the continuous-variable analog syndrome information without introducing artificial cross-talk between the quadratures.

Chapter 5

Structure of the decoder

5.1 Polynomial-Time Closest-Point Decoder for the Surface-GKP Code

To decode the continuous-variable errors intrinsic to the surface-GKP and Multi Genus Toric-GKP code, we employ the exact polynomial-time closest point decoder introduced by Lin, Chamberland, and Noh [15]. For an arbitrary unconcatenated multimode GKP lattice, finding the closest lattice point to a noisy measurement constitutes the Closest Vector Problem (CVP), which is generally NP-hard and requires exponential-time algorithms such as sphere decoding. However, as demonstrated in Section IX of [15], the topological structure of Calderbank-Shor-Steane (CSS) codes, such as the surface code, allows us to decouple the quadratures and exactly map the continuous CVP onto a two-dimensional graph matching problem. This mapping reduces the decoding complexity to polynomial time, $\mathcal{O}(N^3)$, utilizing the Minimum Weight Perfect Matching (MWPM) algorithm.

We consider a Surface-GKP code obtained by concatenating a distance-d rotated surface code with $N = d^2$ square GKP modes. The dual lattice associated with the Surface-GKP code can be written as

$$\Lambda(M_{\text{surf}}^{\perp}) = \bigcup_{j=0}^{2^{N+1}-1} \left[\frac{1}{\sqrt{2}} g_j + \Lambda(\sqrt{2}\, I_{2N}) \right], \tag{5.1}$$

where the vectors $\{g_j\}$ correspond to generators of the normalizer of the underlying surface code. For convenience, we choose

$$g_0 = 0, \tag{5.2}$$

and label the remaining generators such that

$$g_1, \ldots, g_{N-1} \tag{5.3}$$

correspond to independent stabilizer generators, while

$$g_N,\ g_{N+1} \tag{5.4}$$

represent the logical X and logical Z operators of the encoded qubit.

The decoding problem consists of finding the lattice point nearest to a measured vector

$$t \in \mathbb{R}^{2N}. \tag{5.5}$$

Formally, the closest-point decoder seeks

$$\chi_t\Big(\sqrt{2\pi}M_{\text{surf}}^{\perp}\Big) = \arg \min_{v\in\Lambda(\sqrt{\pi}M_{\text{surf}}^{\perp})} \|t - v\|. \tag{5.6}$$

Directly solving Eq. (5.6) is difficult because the lattice consists of exponentially many cosets. To simplify the analysis, it is convenient to rescale the lattice by a factor of $\sqrt{1/\pi}$, yielding

$$\Lambda' = \sqrt{2}\Lambda(M_{\text{surf}}^{\perp}) = \bigcup_{j=0}^{2^{N+1}-1} \left[g_j + \Lambda(2\mathbb{Z}^{2N})\right]. \tag{5.7}$$

Similarly, we define the scaled target vector

$$t' = \sqrt{\frac{1}{\pi}}\, t. \tag{5.8}$$

The decoding task is therefore reduced to finding the lattice point

$$\chi' = \chi_{t'}(\Lambda') = \sqrt{\frac{1}{\pi}}\, \chi_t\big(\sqrt{\pi}M_{\text{surf}}^{\perp}\big)\,. \tag{5.9}$$

Reduction to a Parity-Constrained Integer Optimization

Since the lattice Λ' is an integer lattice, each component of the closest lattice point χ' must be an integer. Furthermore, for each coordinate i, the closest integer to t'_i must be either the nearest integer

$$\lfloor t'_i \rceil \tag{5.10}$$

or the 2nd nearest integer

$$\omega(t'_i), \tag{5.11}$$

Consequently, every component of the closest lattice point χ' must belong to one of these two possibilities. The search space can therefore be restricted to the set

$$S_{\chi'} = \{\chi' \mid \chi'_i \in \{\lfloor t'_i \rceil, \omega(t'_i)\},\ 1 \le i \le 2N\}\,. \tag{5.12}$$

Although $S_{\chi'}$ contains 2^{2N} candidate vectors, it is significantly smaller than the full lattice and contains the true closest lattice point.

To understand the structure of the problem more clearly, consider the vector

$$\chi'' = \big(\lfloor t'_1 \rfloor, \ldots, \lfloor t'_{2N} \rfloor\big), \tag{5.13}$$

obtained by independently rounding every coordinate of t' to its nearest lower integer.

Since χ'' is an integer vector, it may be the closest lattice point to t'. However, in general χ'' does not belong to the lattice Λ'. The reason is that some coordinates may have been rounded in a manner that violates the parity constraints defining the lattice.

To characterize these constraints, recall that

$$\Lambda' = \left\{ v \in \mathbb{Z}^{2N} \;\middle|\; \mathrm{mod}\big(g_i^T \Omega v, 2\big) = 0,\ 1 \le i \le N-1 \right\}, \tag{5.14}$$

Equation (5.14) follows directly from the definition of the scaled lattice Λ' and expresses the requirement that all stabilizer parity checks be satisfied.

Therefore, the closest-point problem can be reformulated as follows: among all vectors in $S_{\chi'}$, find the one that satisfies the parity constraints of Eq. (5.14) while remaining as close as possible to the target vector t'.

The key observation is that violations of Eq. (5.14) can be interpreted as defects associated with stabilizer generators of the underlying surface code. These defects admit a graphical representation, allowing the constrained optimization problem to be mapped onto a minimum-weight perfect matching problem. This construction is developed in the following subsection.

Reduction to a Minimum-Weight Perfect Matching Problem

Although the vector χ' must satisfy the parity constraints

$$\mathrm{mod}\left(g_i^T \Omega \chi', 2\right) = 0, \qquad 1 \le i \le N-1, \tag{5.15}$$

it is convenient to begin with the integer vector χ'' obtained by independently rounding each component of t' to its nearest integer. In general, χ'' does not satisfy the parity constraints and therefore does not necessarily belong to the lattice Λ'.

The decoding problem can then be viewed as determining which coordinates of χ'' should be rounded in the opposite direction. Let

$$\chi'_i \in \{\lfloor t'_i \rfloor, \omega(t'_i)\}, \tag{5.16}$$

Changing the rounding choice of a coordinate modifies the stabilizer parities and may restore the violated constraints. The objective is therefore to identify a set of coordinates that should be rounded differently while minimizing the resulting increase in Euclidean distance from the target vector t'.

To formulate this problem graphically, we construct a weighted graph

$$G = (V, E, W). \tag{5.17}$$

Each vertex

$$v_i \in V \tag{5.18}$$

corresponds to a stabilizer generator g_i.

Two vertices v_i and v_j are connected by an edge whenever the corresponding stabilizer generators share at least one nonzero component. Defining

$$S_{ij} = \{k \mid g_{ik} = g_{jk} = 1\}, \tag{5.19}$$

the edge (v_i, v_j) exists whenever $S_{ij} \neq \varnothing$.

The weight associated with the edge is chosen to represent the minimum increase in Euclidean distance produced by changing the rounding direction of one of the shared coordinates. Explicitly,

$$w_{ij} = \min_{k \in S_{ij}} |\omega(t'_k) - t'_k|^2 - |\lfloor t'_k \rfloor - t'_k|^2 . \tag{5.20}$$

Thus w_{ij} quantifies the additional cost incurred when the k-th coordinate is rounded opposite to its locally optimal choice.

Next, we identify the set of violated parity checks

$$H = \left\{ v_i \in G \;\middle|\; \mathrm{mod}\left(g_i^T \Omega \chi'', 2\right) \neq 0 \right\}. \tag{5.21}$$

The vertices in H correspond to stabilizers whose parity constraints are violated by the independently rounded vector χ''. These vertices play the role of syndrome defects in the decoding graph.

If the number of defect vertices is odd, an additional boundary vertex is introduced so that the total number of highlighted vertices becomes even. A minimum-weight perfect matching is then performed on the weighted graph G.

The resulting matching specifies the minimum-cost set of stabilizer pairs whose parity violations should be corrected. Equivalently, it identifies the set of coordinates that must be rounded opposite to their locally optimal values in order to obtain a vector satisfying all parity constraints.

Once the matching has been determined, the corrected integer vector χ' is reconstructed from the corresponding coordinate flips. The final closest lattice point is then given by

$$\sqrt{\pi}\,\chi', \tag{5.22}$$

which provides the decoded recovery operation for the Surface-GKP code.

By executing the Blossom algorithm [19], the decoder identifies the exact set of edges (qubits) that must be shifted from $\lfloor t'_i \rfloor$ to $\omega(t'_i)$. This methodology entirely bypasses the NP-hard global search space of standard lattices, efficiently finding the exact closest lattice point in polynomial time and maximizing the achievable threshold of the concatenated surface-GKP architecture.

5.2 Decoder for the Multi-Genus Toric-GKP Code

The closest-point decoding framework developed for the Surface-GKP code can be generalized naturally to the Multi-Genus Toric-GKP code considered in this work. As in the Surface-GKP construction, decoding proceeds by first performing GKP error correction on each oscillator and subsequently decoding the residual errors using the outer topological code.

A crucial requirement for the polynomial-time closest-point decoder is that a single residual shift error induces at most two syndrome defects. This property allows the decoding problem to be mapped onto a minimum-weight perfect matching problem.

For the Multi-Genus Toric-GKP code, this condition remains satisfied. The underlying multi-genus toric code is a CSS topological code whose stabilizer generators are associated with vertices and plaquettes of the lattice. Each physical qubit is located on an edge and participates in exactly two neighboring vertex checks and two neighboring plaquette checks. Consequently, a single X-type error produces either zero or two violated Z-type stabilizers, while a single Z-type error produces either zero or two violated X-type stabilizers.

As a result, the syndrome generated by an elementary error consists of pairs of defect vertices connected by an error chain. The decoding problem can therefore be formulated as a pairing problem between syndrome defects, exactly as in the Surface-GKP decoder.

Throughout this work we restrict attention to square GKP states as the inner bosonic code. Under this assumption, the dual lattice of the concatenated Multi-Genus Toric-GKP code can be constructed analogously to the Surface-GKP case by replacing the stabilizer generators of the rotated surface code with those of the multi-genus toric code. The resulting closest-point problem therefore admits the same graph-theoretic interpretation, with the syndrome graph now defined on the topology of the underlying genus-g surface.

Chapter 6

Simulation Methodology

This chapter describes the numerical framework used to evaluate the performance of the concatenated Surface-GKP and Multi-Genus Toric-GKP codes. The simulations are based on a Gaussian displacement noise model acting on the physical GKP oscillators, followed by syndrome extraction, closest-point decoding, and logical recovery. Logical fidelities are estimated using large-scale Monte Carlo simulations over a range of physical noise strengths.

6.1 Noise Model

The physical noise model considered throughout this work is independent Gaussian displacement noise acting on each bosonic mode. Let

$$\hat{\mathbf{x}} = (\hat{q}_1, \ldots, \hat{q}_N, \hat{p}_1, \ldots, \hat{p}_N)^T \tag{6.1}$$

denote the vector of quadrature operators for an N-mode GKP code.
A displacement error is represented by a displacement vector

$$\boldsymbol{\xi} = (\xi_{q,1}, \ldots, \xi_{q,N}, \xi_{p,1}, \ldots, \xi_{p,N})^T \in \mathbb{R}^{2N}, \tag{6.2}$$

which acts on the phase-space coordinates according to

$$\hat{\mathbf{x}} \to \hat{\mathbf{x}} + \boldsymbol{\xi}. \tag{6.3}$$

The displacement vector is sampled from an isotropic Gaussian distribution

$$p(\boldsymbol{\xi}) = \frac{1}{(2\pi\sigma^2)^N} \exp\left(-\frac{\|\boldsymbol{\xi}\|^2}{2\sigma^2}\right), \tag{6.4}$$

where σ characterizes the strength of the physical noise.
For each Monte Carlo trial, a displacement vector

$$\boldsymbol{\xi} \sim \mathcal{N}(0, \sigma^2 I_{2N}) \tag{6.5}$$

is generated independently.

The displacement error may be represented by the Weyl operator

$$\hat{D}(\boldsymbol{\xi}) = \exp\left(i\boldsymbol{\xi}^T \Omega^{-1} \hat{\mathbf{x}}\right), \tag{6.6}$$

where

$$\Omega = \begin{pmatrix} 0 & I_N \\ -I_N & 0 \end{pmatrix} \tag{6.7}$$

is the symplectic form.

The objective of decoding is to infer the displacement vector from the measured syndrome information and apply a recovery operation that returns the state to the original code space.

6.2 Syndrome Extraction

The first stage of decoding consists of extracting syndrome information from the displaced GKP state. Let the concatenated GKP code be specified by the generator matrix M. The stabilizers of the code are given by

$$\hat{S}_j = \exp\left(i\sqrt{2\pi}\, v_j^T \Omega^{-1} \hat{\mathbf{x}}\right), \tag{6.8}$$

where v_j denotes the j-th row of M.

In the absence of noise, an encoded state is a simultaneous $+1$ eigenstate of all stabilizer operators. Suppose now that the state is subjected to a displacement error

$$\hat{D}(\boldsymbol{\xi}). \tag{6.9}$$

Using the Weyl commutation relations, the displacement modifies the phase of each stabilizer according to

$$\theta_j \to \theta_j + \sqrt{2\pi}\, v_j^T \Omega^{-1} \boldsymbol{\xi}. \tag{6.10}$$

The measured syndrome is therefore

$$s_j = \sqrt{2\pi}\, v_j^T \Omega^{-1} \boldsymbol{\xi} \pmod{2\pi}. \tag{6.11}$$

Collecting all syndrome components into a vector

$$\mathbf{s} = (s_1, s_2, \ldots, s_{2N})^T, \tag{6.12}$$

one obtains

$$\mathbf{s} = \sqrt{2\pi} M \Omega^{-1} \boldsymbol{\xi} \pmod{2\pi}. \tag{6.13}$$

Equation (6.13) shows that the measured syndrome contains information about the displacement error only modulo the stabilizer lattice. Consequently, the syndrome does not uniquely determine the physical error.

To remove the modulo ambiguity, one introduces an integer vector

$$\mathbf{a} \in \mathbb{Z}^{2N}, \tag{6.14}$$

such that

$$\sqrt{2\pi} M \Omega^{-1} \boldsymbol{\xi} = \mathbf{s} + 2\pi \mathbf{a}. \tag{6.15}$$

Solving for the displacement vector yields

$$\boldsymbol{\xi} = \eta(\mathbf{s}) - \sqrt{2\pi} (M^{\perp})^{T} \mathbf{a}, \tag{6.16}$$

where

$$\eta(\mathbf{s}) = \frac{-1}{\sqrt{2\pi}} (\Omega M^{\perp})^{T} \mathbf{s} \tag{6.17}$$

is a syndrome-dependent representative and

$$\Lambda^{\perp} = \left\{ \sqrt{2\pi} (M^{\perp})^{T} \mathbf{a} \;\middle|\; \mathbf{a} \in \mathbb{Z}^{2N} \right\} \tag{6.18}$$

is the dual stabilizer lattice.

Therefore, the measured syndrome determines only a coset

$$\boldsymbol{\xi} \in \eta(\mathbf{s}) + \Lambda^{\perp}. \tag{6.19}$$

The decoding problem is thus reduced to identifying which lattice element of the coset most likely generated the observed syndrome.

In the numerical implementation, Eq. (6.13) is evaluated through

$$\mathbf{s} = \mathrm{mod}_{2\pi} \left(A \boldsymbol{\xi} \right), \tag{6.20}$$

where

$$A = \sqrt{2\pi}\, M \Omega^{T}. \tag{6.21}$$

This corresponds directly to the syndrome extraction step used in the simulation algorithm.

6.3 Construction of the Concatenated GKP Code

For each code distance d, the stabilizer generators of the underlying topological code are first constructed. In the case of the Surface or Toric-GKP code, the binary parity-check matrices

$$H_X \qquad \text{and} \qquad H_Z \tag{6.22}$$

of the rotated surface code are generated and converted into the corresponding CSS-GKP lattice representation.

Following the construction described in Chapter 5, the concatenated GKP generator matrix

$$M \tag{6.23}$$

is obtained from the CSS parity-check matrices using the CSS-GKP construction. From M, the matrices

$$M^{-T} \tag{6.24}$$

and

$$M^{\perp} = \Omega M^{-T} \Omega^T \tag{6.25}$$

are computed.

The syndrome extraction matrix

$$A = \sqrt{2\pi} M \Omega^T \tag{6.26}$$

and the closest-point decoding matrix

$$B = -\frac{1}{\sqrt{2\pi}} (\Omega M^{\perp})^T \tag{6.27}$$

are subsequently precomputed and reused throughout the simulation.

This preprocessing stage is performed once for each code distance and significantly reduces the computational cost of the Monte Carlo simulations.

6.4 Monte Carlo Simulation Procedure

The logical performance of the concatenated code is estimated using Monte Carlo sampling over many independent realizations of Gaussian displacement noise.

For a fixed physical noise strength σ, each Monte Carlo trial consists of the following steps:

1. Generate a random displacement vector

$$\boldsymbol{\xi} \sim \mathcal{N}\left(0, \sigma^2 I\right). \tag{6.28}$$

2. Compute the stabilizer syndrome

$$\mathbf{s} = \mathrm{mod}_{2\pi}(A\boldsymbol{\xi}). \tag{6.29}$$

3. Compute the target vector

$$\mathbf{t} = B\mathbf{s}. \tag{6.30}$$

4. Apply the closest-point decoder to obtain

$$\mathbf{c}_{\text{cp}}. \tag{6.31}$$

5. Estimate the displacement error

$$\hat{\boldsymbol{\xi}} = \mathbf{t} - \mathbf{c}_{\text{cp}}. \tag{6.32}$$

6. Compute the residual displacement

$$\mathbf{e_e ff} = \boldsymbol{\xi} - \hat{\boldsymbol{\xi}}. \tag{6.33}$$

7. Determine whether

$$\mathbf{e}_{\text{eff}} \in \Lambda(M), \tag{6.34}$$

where $\Lambda(M)$ denotes the stabilizer lattice.

If this condition is satisfied, the trial is counted as a successful recovery; otherwise a logical failure is recorded.

6.5 Logical Success Criterion

Let N_{succ} denote the number of successful decoding events obtained from

$$N_{\text{trial}}$$

Monte Carlo samples.
The logical fidelity is estimated as

$$F_L = \frac{N_{\text{succ}}}{N_{\text{trial}}}. \tag{6.35}$$

Equivalently, the logical infidelity is

$$P_L = 1 - F_L. \tag{6.36}$$

For each code distance and physical noise strength, 10^6 Monte Carlo trials were performed to obtain statistically reliable estimates of the logical fidelity and thresholds.

Algorithm 3 Monte Carlo Simulation of the Surface and Toric-GKP Code

Require: Code distance d, noise strength σ, number of trials N_{trial}

Construct the rotated surface-code parity-check matrices H_X and H_Z

Build the concatenated GKP generator matrix M

Compute

$$M^{\perp} = \Omega M^{-T} \Omega^T$$

Compute the syndrome and decoding matrices

$$A = \sqrt{2\pi} M \Omega^T, \qquad B = -\frac{1}{\sqrt{2\pi}} (\Omega M^{\perp})^T$$

Initialize $N_{\text{succ}} \leftarrow 0$

for $i = 1, \ldots, N_{\text{trial}}$ **do**

    Sample a Gaussian displacement

$$\boldsymbol{\xi} \sim \mathcal{N}(0, \sigma^2 I)$$

    Compute the syndrome

$$\mathbf{s} = \text{mod}_{2\pi}(A\boldsymbol{\xi})$$

    Compute the target vector

$$\mathbf{t} = B\mathbf{s}$$

    Apply the closest-point decoder

$$\mathbf{c}_{\text{cp}} = \text{Decoder}(\mathbf{t})$$

    Estimate the displacement

$$\hat{\boldsymbol{\xi}} = \mathbf{t} - \mathbf{c}_{\text{cp}}$$

    Compute the residual error

$$\mathbf{e_eff} = \boldsymbol{\xi} - \hat{\boldsymbol{\xi}}$$

    if $\mathbf{e}_{\text{eff}} \in \Lambda(M)$ **then**

        $N_{\text{succ}} \leftarrow N_{\text{succ}} + 1$

    end if

end for

Estimate the logical fidelity:

$$F_L = \frac{N_{\text{succ}}}{N_{\text{trial}}}$$

return F_L

Chapter 7

Results and Discussion

This chapter presents the numerical results obtained from Monte Carlo simulations of the Surface-GKP and Multi-Genus Toric-GKP codes. The primary objective is to investigate the logical fidelity, threshold behavior, and encoding efficiency of the proposed Multi-Genus Toric-GKP construction and compare its performance with the conventional Surface-GKP code.

All simulations were performed using the decoding framework described in Chapter 6. For each physical noise strength, independent Gaussian displacement errors were sampled and decoded using the polynomial-time closest-point decoder. Logical fidelities were estimated from large-scale Monte Carlo simulations consisting of 10^5 independent trials per noise point unless otherwise stated.

The results are organized into three parts. First, the performance of the Surface-GKP code is presented to establish a baseline reference. Next, the performance of the Multi-Genus Toric-GKP code is investigated. Finally, the two code families are compared in terms of logical fidelity, threshold behavior, and encoding efficiency.

7.1 Surface-GKP Code Performance

Before investigating the proposed Multi-Genus Toric-GKP construction, it is necessary to validate the simulation framework and decoder implementation against previously reported results. To this end, the concatenated Surface-GKP code was simulated using the polynomial-time closest-point decoder described in Chapter 5.

The Surface-GKP code serves as an important benchmark because its threshold behavior has been studied extensively in the literature [7]. Reproducing the known logical-fidelity curves and threshold estimates provides confidence in the correctness of the implemented decoding framework before applying it to the Multi-Genus Toric-GKP code.

7.1.1 Logical Fidelity as a Function of Noise Strength

Figure 7.1 shows the logical fidelity of the Surface-GKP code as a function of the physical Gaussian displacement noise strength σ. The simulations were performed for code distances

$$d = 3, 5, 7, 9, \text{ and } 11. \tag{7.1}$$

The noise strength was varied over the interval

$$0.40 \leq \sigma \leq 0.80. \tag{7.2}$$

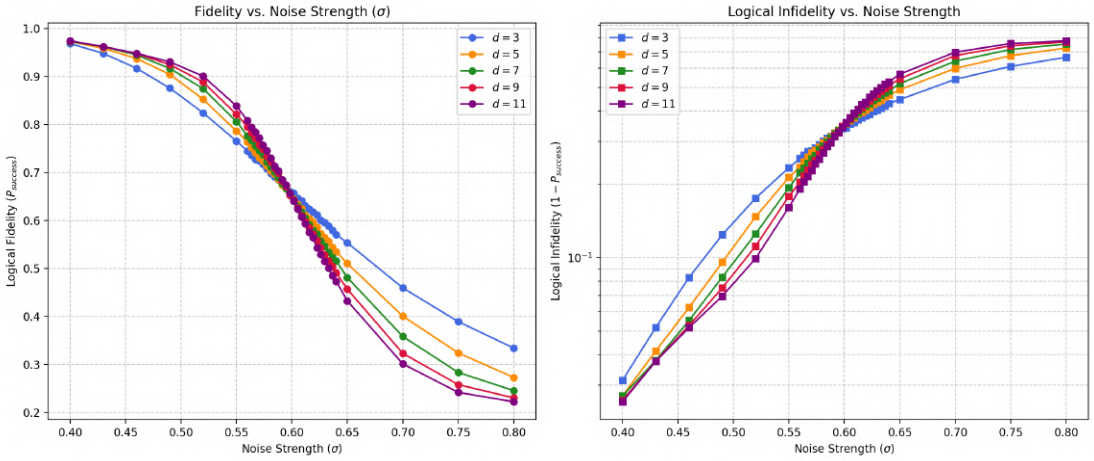


Figure 7.1: Logical fidelity of the Rotated-Surface-GKP code as a function of the physical displacement noise strength σ. Results are shown for code distances $d = 3, 5, 7, 9$, and 11. Each data point was obtained from 10^5 Monte Carlo trials.

As expected, the logical fidelity decreases monotonically with increasing physical noise strength. In the low-noise regime, larger code distances exhibit significantly improved logical performance, indicating the successful suppression of logical errors through increased code redundancy.

As the noise strength increases, the performance gap between different code distances gradually narrows. Near the threshold region, the logical fidelity curves approach one another and eventually intersect. Such crossing behavior is characteristic of fault-tolerant quantum error-correcting codes and provides an estimate of the decoding threshold.

The overall behavior observed in Fig. 7.1 is consistent with previously reported results for the Surface-GKP code [7], thereby validating the implementation of the concatenated code construction, syndrome extraction procedure, and closest-point decoder.

7.1.2 Threshold Estimation

To obtain a more accurate estimate of the decoding threshold, additional simulations were performed in a narrow interval around the crossing region observed in Fig. 7.1.

The physical noise strength was restricted to

$$0.596 \leq \sigma \leq 0.607, \tag{7.3}$$

and code distances up to

$$d = 29 \tag{7.4}$$

were considered.

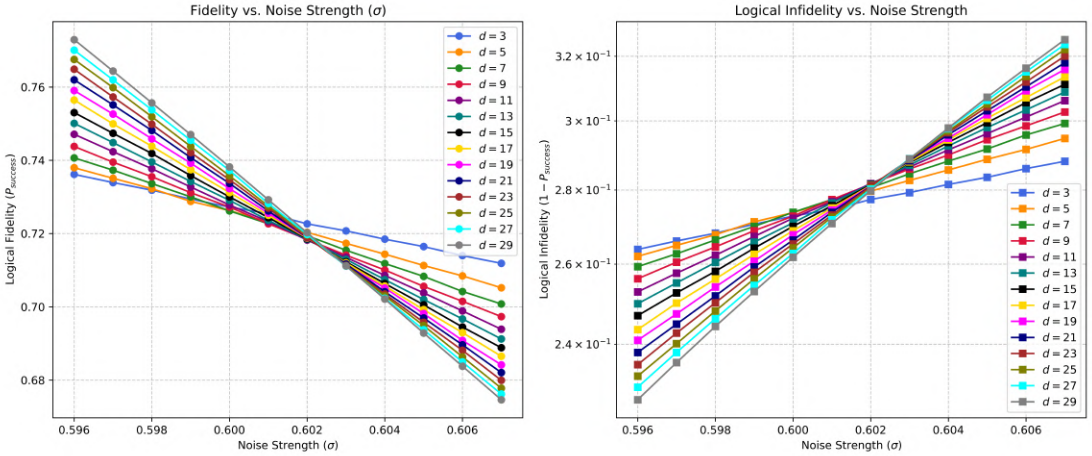


Figure 7.2: Zoomed view of the Rotated-Surface-GKP logical fidelity curves near the threshold region. Results are shown for code distances ranging from $d = 3$ to $d = 29$ with 10^6 trials.

The enlarged view reveals a common crossing point of the logical fidelity curves. This crossing corresponds to the decoding threshold of the Surface-GKP code under independent Gaussian displacement noise.

From the numerical data, the threshold is estimated to occur near

$$\sigma_{\text{th}} \approx 0.602, \tag{7.5}$$

which is in excellent agreement with the threshold reported by Chamberland *et al.* [7].

The successful reproduction of both the logical-fidelity curves and the threshold value provides strong evidence that the simulation framework accurately implements the Surface-GKP decoding procedure. Having established this benchmark, we now proceed to investigate the performance of the proposed Multi-Genus Toric-GKP code.

7.2 Multi-Genus Rotated Toric-GKP Code Performance

Having validated the simulation framework through the reproduction of previously reported Surface-GKP results, we now turn to the primary contribution of this thesis: the Multi-Genus Rotated Toric-GKP code.

The objective of this section is to investigate the logical performance of concatenated GKP codes constructed from higher-genus topological lattices. In particular, we seek to determine whether the increased encoding capability afforded by higher-genus surfaces can be achieved while maintaining favorable error-correction performance and threshold behavior.

The results presented below are organized into two parts. First, a representative set of logical-fidelity curves is presented for a genus-5 code over a broad range of physical noise strengths (σ). Subsequently, high-resolution simulations are used to determine the threshold behavior for several genus.

7.2.1 Representative Logical Fidelity Curves

To illustrate the overall behavior of the Multi-Genus Rotated Toric-GKP code, simulations were first performed for the genus-5 construction over a broad range of physical noise strengths.

The displacement noise parameter was varied over the interval

$$0.40 \leq \sigma \leq 0.80, \tag{7.6}$$

which spans both the low-noise regime and the threshold region.

Figure 7.3 shows the resulting logical fidelity curves for several code distances.

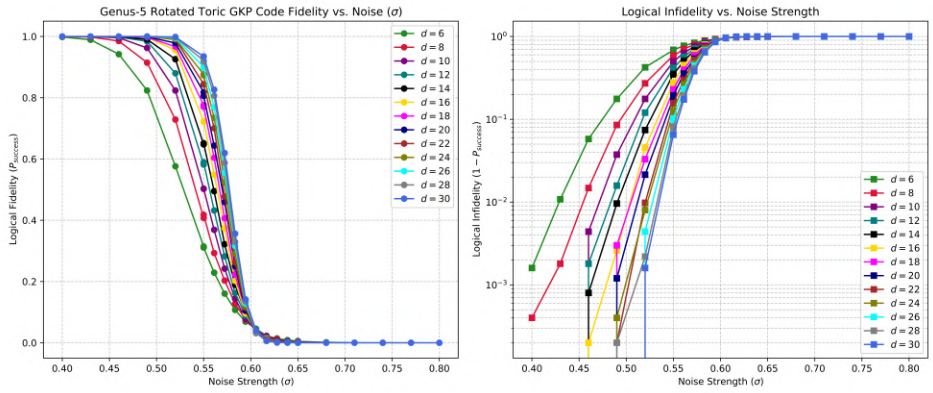


Figure 7.3: Logical fidelity of the genus-5 Rotated Toric-GKP code as a function of the physical displacement noise strength σ. Each point was obtained from 10^4 independent Monte Carlo trials.

The behavior observed in Fig. 7.3 is qualitatively similar to that of the Surface-GKP code. In the low-noise regime, the logical fidelity approaches unity and improves systematically with increasing code distance. As the physical noise strength increases, the logical fidelity decreases and the different code-distance curves gradually converge toward a common crossing region.

The existence of such a crossing region suggests the presence of a fault-tolerance threshold, motivating a more detailed investigation using higher-resolution simulations.

7.2.2 Threshold Estimation for Different Genera

To determine the threshold behavior of the Multi-Genus Rotated Toric-GKP code more accurately, additional simulations were performed in narrow intervals surrounding the crossing regions identified from the full-range fidelity curves.

Unlike the representative genus-5 study presented above, threshold calculations were carried out for multiple genera in order to investigate the dependence of the threshold on the topology of the underlying surface.

For each genus, logical fidelities were computed using high-resolution sampling of the physical noise strength and 10^6 Monte Carlo trials per data point.

The resulting threshold curves are shown in Figs. 7.4–7.7.

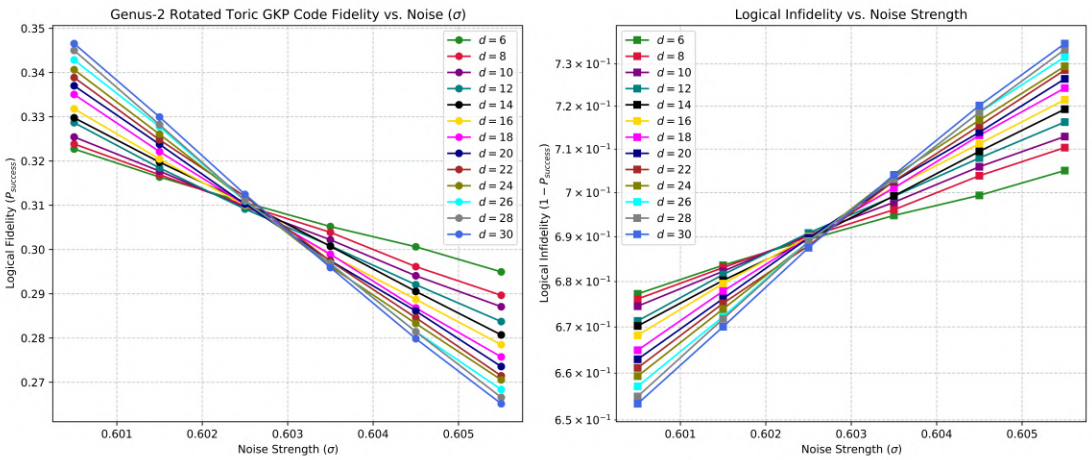


Figure 7.4: Threshold analysis for the genus-2 Rotated Toric-GKP code.

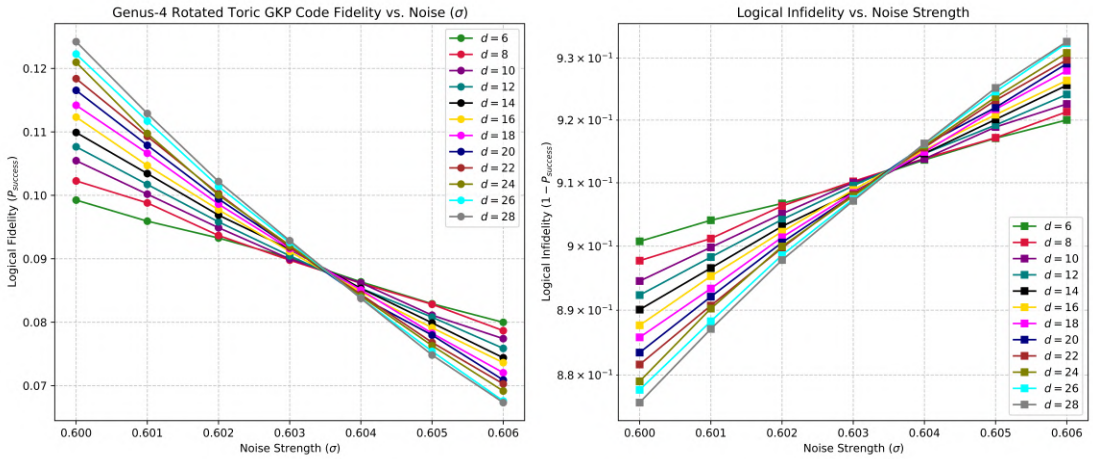


Figure 7.5: Threshold analysis for the genus-4 Rotated Toric-GKP code.

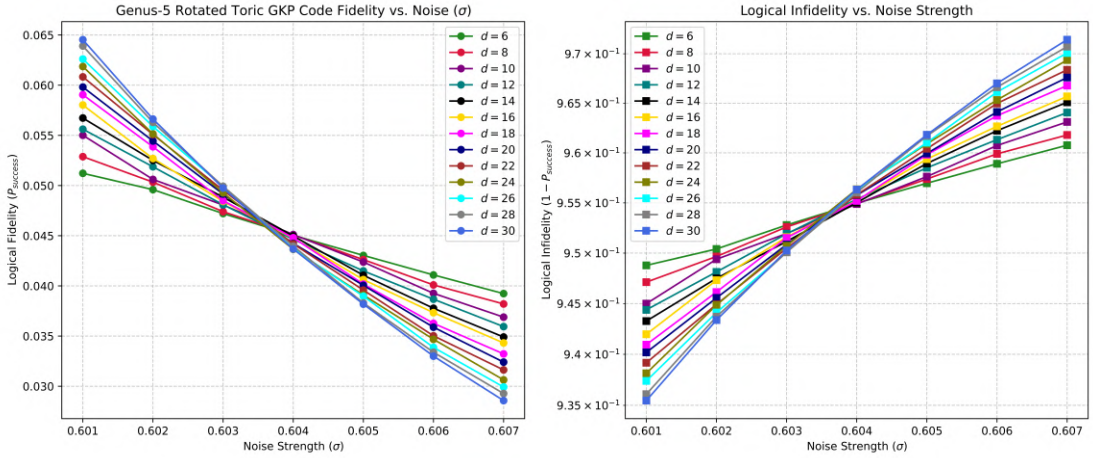


Figure 7.6: Threshold analysis for the genus-5 Rotated Toric-GKP code.

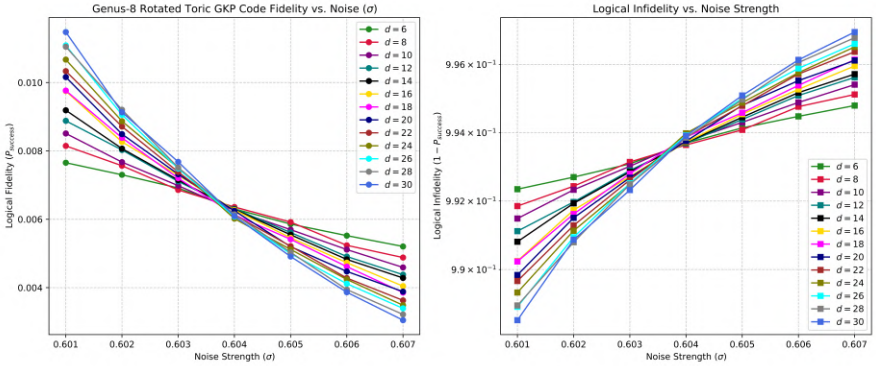


Figure 7.7: Threshold analysis for the genus-8 Rotated Toric-GKP code.

Threshold Estimates

The threshold values obtained from the high-resolution simulations are summarized in Table 7.1.

Table 7.1: Estimated threshold values for the Multi-Genus Rotated Toric-GKP code under independent Gaussian displacement noise.

Genus g	Estimated Threshold σ_{th}
2	≈ 0.6025
4	≈ 0.6035
5	≈ 0.6036
8	≈ 0.6040

Several noteworthy observations can be made from Table 7.1. First, all genera considered exhibit threshold values close to $\sigma \approx 0.60$, demonstrating that the Multi-Genus Rotated Toric-GKP construction preserves the threshold phenomenon characteristic of concatenated topological GKP codes.

Second, the threshold values obtained for all multi-genus constructions are consistently higher than the threshold observed for the rotated Surface-GKP code discussed in the previous section.

For the Surface-GKP code, the threshold was estimated to occur near

$$\sigma_{\text{th}}^{(\text{Surface})} \approx 0.602, \tag{7.7}$$

which is in agreement with the results reported by Chamberland *et al.* [7].

Most importantly, the threshold improvement is achieved simultaneously with a substantial increase in encoding capability. Whereas the rotated surface code encodes a single logical qubit, a genus-g rotated toric code encodes

$$k = 2g \tag{7.8}$$

logical qubits.

Consequently, the genus-8 construction encodes sixteen logical qubits while maintaining a threshold that is slightly higher than that of the rotated Surface-GKP code. Similar behavior is observed for the other genera considered in this study.

These results indicate that increasing the genus of the underlying topological surface does not degrade the fault-tolerance properties of the concatenated code. On the contrary, the simulations suggest that higher-genus constructions can simultaneously provide improved encoding efficiency and modest improvements in threshold performance.

The threshold estimates presented here provide the first indication that Multi-Genus Rotated Toric-GKP codes may offer a favorable trade-off between logical encoding capacity and fault-tolerant performance. A more detailed comparison with the Surface-GKP code is presented in the following section.

7.3 Comparison with the Rotated Surface-GKP Code

The threshold analysis presented in the previous sections demonstrated that the Multi-Genus Rotated Toric-GKP code exhibits higher threshold values compared to those of the Rotated Surface-GKP code. However, threshold values alone do not provide a complete picture of code performance.

A direct comparison between the two code families is complicated by the fact that they encode different numbers of logical qubits. While a rotated surface code encodes a single logical qubit, a genus-g rotated toric code encodes

$$k = 2g \tag{7.9}$$

logical qubits.

To perform a fair comparison, the logical fidelity of the Surface-GKP code was normalized to the same number of logical qubits as the corresponding Multi-Genus Rotated Toric-GKP code. Specifically, for a genus-g toric code, the normalized surface-code fidelity was taken to be

$$F_{\text{surface}}^{(2g)}, \tag{7.10}$$

which represents the probability that all $2g$ independently encoded surface-code logical qubits are successfully recovered.

The relative improvement of the Multi-Genus Rotated Toric-GKP code was then quantified using

$$\Delta F(\%) = 100 \times \frac{F_{\text{toric}} - F_{\text{surface}}^{2g}}{F_{\text{surface}}^{2g}}. \tag{7.11}$$

A positive value of ΔF indicates superior logical performance of the Multi-Genus Rotated Toric-GKP code, while a negative value indicates superior performance of the equivalent collection of independently encoded surface-code logical qubits.

7.3.1 Comparison at Equal Logical Capacity

To investigate whether the higher-genus architecture provides benefits beyond an increased encoding rate, we compare the genus-2 Rotated Toric-GKP code against an equivalent collection of Rotated Surface-GKP logical qubits.

Since a genus-2 rotated toric code encodes four logical qubits, a direct comparison with a single logical qubit surface code would be inappropriate. To account for this difference, the surface-code fidelity is normalized according to

$$F_{\text{surface}}^{4}, \tag{7.12}$$

which corresponds to the probability that four independently encoded surface-code logical qubits are simultaneously recovered successfully.

The relative fidelity improvement is then defined as

$$\Delta F(\%) = 100 \times \frac{F_{\text{toric}} - F^4_{\text{surface}}}{F^4_{\text{surface}}}. \tag{7.13}$$

Positive values of ΔF indicate superior logical performance of the genus-2 Rotated Toric-GKP code. In the next discussions, we have used 10^5 Monte Carlo trials for each data point to estimate fidelity for both Rotated Surface-GKP code and Rotated Toric-GKP code.

Figure 7.8 shows the relative fidelity improvement for several distance pairs $(d, d+1)$, where d denotes the distance of the Rotated Surface-GKP code and $d+1$ denotes the distance of the genus-2 Rotated Toric-GKP code.

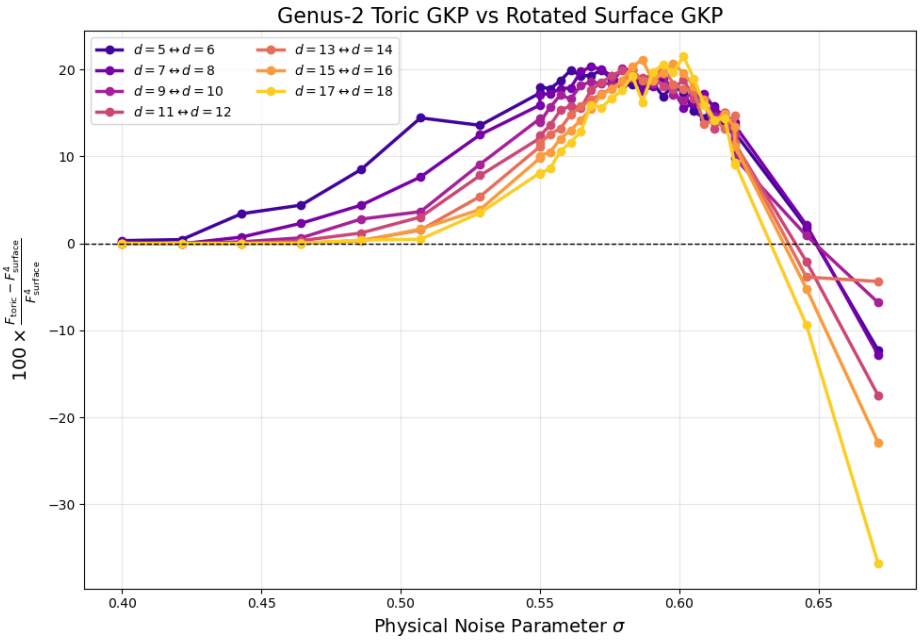


Figure 7.8: Relative fidelity improvement of the genus-2 Rotated Toric-GKP code over the equivalent four-logical-qubit Rotated Surface-GKP construction. The improvement is defined as $100 \times (F_{\text{toric}} - F^4_{\text{surface}})/F^4_{\text{surface}}$.

Several important observations can be made from Fig. 7.8. For all investigated distance pairs, the relative fidelity improvement remains positive throughout the low- and intermediate-noise regimes, indicating that the genus-2 Rotated Toric-GKP code consistently outperforms the equivalent four-logical-qubit Rotated Surface-GKP construction.

As the physical noise strength increases, the improvement grows steadily and reaches its maximum value in the vicinity of the threshold region. The largest gains are observed approximately in the interval

$$0.55 \lesssim \sigma \lesssim 0.62, \tag{7.14}$$

where the relative fidelity improvement approaches 20% for all investigated distance pairs. This behavior is particularly significant because this regime corresponds to the most practically relevant operating region for fault-tolerant quantum computation. The results therefore indicate that the higher-genus architecture provides its greatest advantage precisely where reliable error correction becomes most challenging.

Another notable feature is the consistency of the observed improvement across all considered code distances. Although the magnitude of the enhancement varies slightly from one distance pair to another, all curves exhibit qualitatively similar behavior and attain their maximum values within the same narrow range of physical noise strengths. This suggests that the observed advantage is not a finite-size effect associated with a particular code distance, but rather a systematic feature of the genus-2 Rotated Toric-GKP construction.

7.3.2 Comparison with Higher-Distance Surface-GKP Codes

The comparison presented in the previous subsection was performed against Surface-GKP constructions normalized to the same logical encoding capacity as the genus-2 Rotated Toric-GKP code. While this provides a fair comparison from the perspective of logical-qubit storage, it remains possible that the observed advantage arises simply from the use of larger code distances.

To investigate this possibility, a second comparison was performed using Rotated Surface-GKP codes of higher distance. Specifically, the distance pairs

$$(6,7),\ (8,9),\ (10,11),\ (12,13),\ (14,15),\ (16,17) \tag{7.15}$$

were considered, where the first entry denotes the genus-2 Rotated Toric-GKP code distance and the second entry denotes the corresponding Rotated Surface-GKP code distance.

Since the Surface-GKP code possesses a larger code distance in each pair, this comparison constitutes a significantly more stringent benchmark. In general, one expects a higher-distance code to exhibit improved logical performance. Consequently, any observed advantage of the genus-2 Rotated Toric-GKP construction cannot be attributed solely to distance scaling.

Figure 7.9 shows the resulting relative fidelity improvement as a function of the physical noise strength.

Several observations can be made from Fig. 7.9. Most notably, the relative fidelity improvement remains positive over a broad interval of physical noise strengths despite the fact that the Surface-GKP code possesses a larger code distance in every distance pair considered. This demonstrates that the advantages of the genus-2 Rotated Toric-GKP architecture cannot be attributed solely to distance scaling.

The improvement becomes particularly pronounced in the vicinity of the threshold region, where all investigated distance pairs exhibit enhancements ranging from

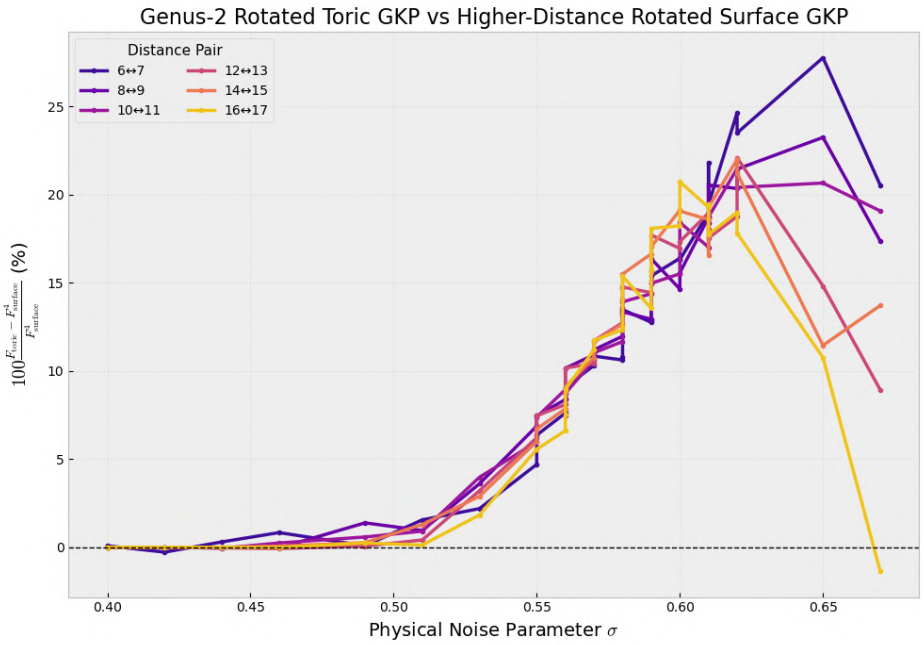


Figure 7.9: Relative fidelity improvement of the genus-2 Rotated Toric-GKP code compared with higher-distance Rotated Surface-GKP codes. The distance pairs $(6, 7)$, $(8, 9)$, $(10, 11)$, $(12, 13)$, $(14, 15)$, and $(16, 17)$ correspond to genus-2 Rotated Toric-GKP and Rotated Surface-GKP codes, respectively. Despite the surface code possessing a larger code distance in every pair, the genus-2 construction exhibits a substantial fidelity advantage throughout the threshold region, reaching improvements exceeding 20% for several distance pairs.

approximately 15% to 28%. Since this regime is the most relevant for practical fault-tolerant operation, the observed behavior provides strong evidence that the higher-genus architecture offers genuine improvements in logical error-correction performance.

Small fluctuations between neighboring data points are visible near the peak-improvement region. These variations arise from the finite Monte Carlo sampling used to estimate logical fidelities and do not alter the overall trend. The essential observation is that all distance pairs display a consistent increase in relative performance as the threshold region is approached.

7.3.3 Implications for Higher-Genus Codes

The detailed fidelity comparisons presented in this section were performed explicitly for the genus-2 Rotated Toric-GKP code. Nevertheless, the threshold analysis discussed earlier suggests that the observed advantages are not unique to this particular genus. As shown in Table 7.1, the Multi-Genus Rotated Toric-GKP codes maintain threshold values that are consistently higher than those of the

Rotated Surface-GKP code across all investigated genera.

This observation is particularly significant because a genus-g rotated toric code encodes

$$k = 2g \tag{7.16}$$

logical qubits. Consequently, increasing the genus leads to a substantial increase in logical encoding capacity while preserving threshold behavior comparable to, and in several cases better than, that of the Rotated Surface-GKP code.

Although a quantitative fidelity comparison analogous to that performed for the genus-2 code has not been carried out for larger genera, the stability of the threshold values provides no indication that increasing the genus introduces a significant degradation in fault-tolerance performance. On the contrary, the available numerical evidence suggests that the benefits observed for the genus-2 construction may persist for higher-genus codes as well.

Taken together, the threshold analysis and the genus-2 fidelity comparisons indicate that Multi-Genus Rotated Toric-GKP codes offer a promising route toward simultaneously increasing encoding efficiency and maintaining strong logical error-correction performance.

7.3.4 Encoding Rate Improvement

In addition to the improvements observed in logical fidelity and threshold behavior, the Multi-Genus Rotated Toric-GKP code provides a significant advantage in encoding efficiency.

A useful metric for quantifying encoding efficiency is the encoding rate,

$$R = \frac{k}{N}, \tag{7.17}$$

where k denotes the number of encoded logical qubits and N is the number of physical qubits of the underlying topological code.

For the rotated surface code,

$$R_{\text{Surface}} = \frac{1}{N} = \frac{1}{d^2}, \tag{7.18}$$

since the code encodes a single logical qubit.

In contrast, the Multi-Genus Rotated Toric code encodes multiple logical qubits while maintaining a comparable local stabilizer structure. As a consequence, the corresponding encoding rate satisfies

$$R_{\text{MGRT}} = 2R_{\text{Surface}}, \tag{7.19}$$

for the code families considered in this work.

Thus, for a comparable number of physical qubits, the Multi-Genus Rotated Toric-GKP construction is capable of storing approximately twice as much logical information as the corresponding Rotated Surface-GKP code.

Importantly, this improvement in encoding rate is achieved without sacrificing fault-tolerance performance. The threshold analysis presented in the previous sections indicates that the Multi-Genus Rotated Toric-GKP codes maintain threshold values comparable to, and in some cases slightly exceeding, those of the Rotated Surface-GKP code.

Taken together, these results suggest that higher-genus topological constructions can simultaneously improve encoding efficiency, logical fidelity, and threshold behavior, making them attractive candidates for large-scale fault-tolerant quantum information processing.

Chapter 8

Conclusion and Future Work

8.1 Summary and Conclusions

The primary objective of this thesis was to investigate whether higher-genus topological structures can be combined with bosonic quantum error correction to improve both encoding efficiency and logical performance. To this end, Multi-Genus Rotated Toric codes were concatenated with square GKP states and studied under Gaussian displacement noise.

A complete simulation framework was developed for constructing the concatenated codes, generating Gaussian displacement errors, extracting syndromes, and performing recovery using the polynomial-time closest-point decoder. The framework was first validated through the reproduction of previously reported results for the Rotated Surface-GKP code, including the logical-fidelity curves and threshold behavior obtained by Chamberland *et al.* [7].

Having established the correctness of the implementation, the methodology was extended to Multi-Genus Rotated Toric-GKP codes. Extensive Monte Carlo simulations revealed that these codes preserve the threshold phenomenon characteristic of concatenated topological GKP codes while simultaneously supporting a significantly larger logical space.

The numerical results indicate that the threshold remains remarkably stable as the genus increases. Threshold estimates of approximately 0.6025, 0.6035, 0.6036, and 0.6040 were obtained for genera $g = 2$, 4, 5, and 8, respectively. These values are higher than the threshold observed for the Rotated Surface-GKP code.

In addition to the threshold analysis, a direct fidelity comparison was performed for the genus-2 construction after normalizing for the number of encoded logical qubits. The results demonstrated that the Multi-Genus Rotated Toric-GKP code can achieve improved logical fidelity over a broad range of physically relevant noise strengths while simultaneously encoding multiple logical qubits within a single topological structure.

Perhaps the most significant outcome of this work is that the increase in logical encoding capability does not appear to come at the expense of fault tolerance.

The simulations suggest that higher-genus topological architectures can maintain the favorable decoding properties of conventional topological GKP codes while providing substantially improved encoding efficiency.

Taken together, these results demonstrate that higher-genus topological constructions constitute a promising direction for scalable bosonic quantum error correction and motivate further exploration of topological structures beyond the conventional surface-code paradigm.

8.2 Future Work

The results presented in this thesis open several promising directions for future research. While the present study demonstrates the viability of Multi-Genus Rotated Toric-GKP codes under Gaussian displacement noise, many important questions remain regarding both their theoretical properties and practical implementation.

One natural extension of this work is the investigation of larger genera and more general topological structures. The simulations considered genera up to $g = 8$, where the threshold behavior remained remarkably stable. Exploring the scaling of logical fidelity, threshold values, and encoding rates for substantially larger genera may provide further insight into the asymptotic properties of higher-genus topological codes.

Another important direction concerns the development of improved decoding algorithms. Throughout this work, decoding was performed using the polynomial-time closest-point decoder based on a reduction to minimum-weight perfect matching. Although this decoder exhibits excellent performance, it is not necessarily optimal. Tensor-network-based decoding techniques have demonstrated considerable success for a variety of topological quantum codes and may offer improved decoding performance for higher-genus lattices. Investigating the interplay between lattice topology and tensor-network contraction methods therefore represents an interesting avenue for future study.

Machine-learning-assisted decoding constitutes another promising research direction. Neural-network and reinforcement-learning decoders have shown competitive performance for several families of stabilizer codes, particularly in situations where the underlying noise model is complex or correlated. Extending such approaches to Multi-Genus Rotated Toric-GKP codes may reveal additional improvements in logical fidelity and threshold behavior beyond those achievable with conventional matching-based decoders.

The present work focused exclusively on topological codes defined on smooth higher-genus surfaces. However, recent developments in topological quantum error correction have highlighted the importance of defects, boundaries, and twists as tools for manipulating logical information. Incorporating twist defects into higher-genus constructions may provide new mechanisms for logical encoding, code deformation, and fault-tolerant gate implementation. The combination of higher-genus topology with twist-based architectures therefore constitutes a particularly

intriguing direction for future investigation.

Another important extension concerns the bosonic layer of the concatenated code. In this thesis, the inner code was restricted to square GKP states. Alternative bosonic encodings, including non-square lattice GKP codes and multimode GKP constructions, may provide enhanced protection against displacement noise and lead to improved overall performance when combined with higher-genus topological codes.

The threshold values obtained in this work are also of particular interest from an information-theoretic perspective. The thresholds observed for the Multi-Genus Rotated Toric-GKP codes approach the regime associated with the best known lower bounds on the quantum capacity of the Gaussian random displacement channel. A more detailed investigation of the relationship between higher-genus topological architectures and fundamental quantum communication limits may therefore yield valuable insights into the ultimate capabilities of bosonic quantum error correction.

Finally, the present study was restricted to phenomenological noise acting on idealized GKP states. Future work should incorporate finite-squeezing effects, imperfect syndrome extraction, measurement noise, and fully fault-tolerant circuit-level simulations. Such investigations will be essential for assessing the practical viability of Multi-Genus Rotated Toric-GKP codes in realistic experimental platforms.

Overall, the results of this thesis suggest that higher-genus topological architectures provide a fertile framework for the development of next-generation bosonic quantum error-correcting codes. Continued exploration of improved decoders, richer topological structures, and more realistic noise models may further enhance their potential for scalable fault-tolerant quantum information processing.

References

[1] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge, UK: Cambridge University Press, 2000.

[2] D. Gottesman. *Stabilizer Codes and Quantum Error Correction*. 1997. arXiv: quant-ph/9705052 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9705052.

[3] B. M. Terhal. "Quantum error correction for quantum memories". In: *Rev. Mod. Phys.* 87 (2 Apr. 2015), pp. 307–346. DOI: 10.1103/RevModPhys.87.307. URL: https://link.aps.org/doi/10.1103/RevModPhys.87.307.

[4] A. Y. Kitaev. "Fault-tolerant quantum computation by anyons". In: *Annals of Physics* 303 (2003), pp. 2–30. DOI: 10.1016/S0003-4916(02)00018-0.

[5] A. G. Fowler et al. "Surface codes: Towards practical large-scale quantum computation". In: *Physical Review A* 86 (2012), p. 032324. DOI: 10.1103/PhysRevA.86.032324.

[6] D. Gottesman, A. Kitaev, and J. Preskill. "Encoding a qubit in an oscillator". In: *Physical Review A* 64.1 (2001). DOI: 10.1103/physreva.64.012310. URL: http://dx.doi.org/10.1103/PhysRevA.64.012310.

[7] K. Noh, C. Chamberland, and F. G. Brandão. "Low-Overhead Fault-Tolerant Quantum Error Correction with the Surface-GKP Code". In: *PRX Quantum* 3 (1 Jan. 2022), p. 010315. DOI: 10.1103/PRXQuantum.3.010315. URL: https://link.aps.org/doi/10.1103/PRXQuantum.3.010315.

[8] E. Dennis et al. "Topological quantum memory". In: *Journal of Mathematical Physics* 43 (2002), pp. 4452–4505. DOI: 10.1063/1.1499754.

[9] A. M. Steane. "Multiple-particle interference and quantum error correction". In: *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 452 (1996), pp. 2551–2577. URL: https://api.semanticscholar.org/CorpusID:8246615.

[10] A. Ketkar et al. “Nonbinary Stabilizer Codes Over Finite Fields”. In: *IEEE Transactions on Information Theory* 52 (2005), pp. 4892–4914. URL: https://api.semanticscholar.org/CorpusID:14648060.

[11] A. Hatcher. *Algebraic Topology*. First edition. New York: Cambridge University Press, 2001.

[12] S. Bravyi and A. Kitaev. “Quantum codes on a lattice with boundary”. In: *arXiv preprint quant-ph/9811052* (1998).

[13] J. Edmonds. “Paths, trees, and flowers”. In: *Canadian Journal of Mathematics* 17 (1965), pp. 449–467.

[14] O. Higgott. “PyMatching: A Python library for decoding quantum error correcting codes”. In: *ACM Transactions on Quantum Computing* 3.3 (2022), pp. 1–27. DOI: 10.1145/3528713.

[15] M. Lin, C. Chamberland, and K. Noh. “Closest Lattice Point Decoding for Multimode Gottesman-Kitaev-Preskill Codes”. In: *PRX Quantum* 4.4 (Dec. 2023). DOI: 10.1103/prxquantum.4.040334. URL: http://dx.doi.org/10.1103/PRXQuantum.4.040334.

[16] D. Aasen et al. *A Topologically Fault-Tolerant Quantum Computer with Four Dimensional Geometric Codes*. 2025. arXiv: 2506.15130 [quant-ph]. URL: https://arxiv.org/abs/2506.15130.

[17] C. C. Gerry and P. L. Knight. *Introductory Quantum Optics*. 2nd ed. Cambridge University Press, 2023.

[18] J. H. Conway and N. J. A. Sloane. *Sphere Packings, Lattices and Groups*. Springer, 1988. DOI: 10.1007/978-1-4757-2016-7.

[19] P. Arkhipov and V. Kolmogorov. *Blossom VI: A Practical Minimum Weight Perfect Matching Algorithm*. 2026. arXiv: 2604.20351 [cs.DS]. URL: https://arxiv.org/abs/2604.20351.